



Master Calcul Haute Performance et Simulation

Mémoire 1 – Sujet mathématiques

Veille, étude et implémentation de modèles de rivière basés sur les équations de Saint-Venant

par Jean-Baptiste Gaillot

sous la direction de Tristan Cambonie

Table des matières

1	Équations générales	1
1.1	Équations de Navier-Stokes	1
1.2	Équations de Saint-Venant en 2D	2
1.3	Équations de Saint-Venant en 1D	4
2	Modèle de rivière pour une section rectangulaire avec hypothèses dynamiques	7
2.1	Problème instationnaire	7
2.1.1	Présentation du problème	7
2.1.2	Linéarisation du problème, et conditions aux limites	10
2.1.3	Solutions exactes	13
2.1.4	Solutions approchées	14
2.1.4.1	Méthodes numériques et schémas numériques	15
2.1.4.2	Discrétisation de l'espace avec la méthode des différences finies	17
2.1.4.3	Discrétisation de l'espace avec la méthode des volumes finis	18
2.1.4.4	Discrétisation du temps avec la méthode des différences finies	18
2.1.4.5	Schéma aux différences finies de Lax-Friedrichs	18
2.1.4.6	Préparation d'un schéma aux volumes finis	21
2.1.4.7	Schéma aux volumes finis de Rusanov	22
2.1.4.8	Comparaison des schémas numériques	38
2.2	Problème stationnaire, et conditions initiales	39
2.2.1	Régime permanent	39
2.2.2	Présentation du problème	39
2.2.3	Solutions exactes	40
2.2.4	Solutions approchées	41
3	Modèle de rivière pour une section rectangulaire avec hypothèses diffusives	44
3.1	Problème instationnaire	44
3.1.1	Présentation du problème	44
3.1.2	Solutions exactes	46
3.1.3	Solutions approchées	48
3.1.3.1	Schéma aux différences finies de Lax-Friedrichs	49
3.1.3.2	Schéma aux volumes finis de Rusanov	49
3.2	Problème stationnaire, et conditions initiales	56
3.2.1	Présentation du problème	56
3.2.2	Solutions approchées	57
4	Modèle de rivière pour une section rectangulaire avec hypothèses cinématiques	58
4.1	Problème instationnaire	58
4.1.1	Présentation du problème	58
4.1.2	Solutions exactes	59
4.1.3	Solutions approchées	65
4.1.3.1	Schéma aux différences finies arrière	65
4.1.3.2	Schéma aux volumes finis arrière	66
4.2	Problème stationnaire, et conditions initiales	74
4.2.1	Présentation du problème	74
4.2.2	Solutions approchées	74

5	Résumé des modèles	76
5.1	Caractéristiques	76
5.2	Erreurs	76
5.3	Performances	77
5.4	Conclusion	79
6	Veille bibliographique, et couplage de modèles	80
7	Application en ligne de commande de la résolution des modèles de rivière	85
7.1	Utilisation	85
7.2	Contraintes	85
7.3	Propriété intellectuelle	86

Informations et introduction

L'objectif de ce projet est d'étudier des modèles de rivière, d'un point de vue mathématique (équations aux dérivées partielles) et informatique (calcul haute performance).

Pour ce projet, des remerciements sont adressés à :

- Tristan Cambonie (docteur en mécanique des fluides), pour son encadrement.
- Kenelm Louetsi (docteur en calcul haute performance et simulation), pour ses avis et ses conseils.

Pour ce stage, des remerciements sont adressés à :

- Alexandre Bredimas (fondateur de *Bluemapping*), pour la confiance accordée.
- Les équipes de *Bluemapping* et de *Strane Innovation*, pour leur accueil et leur sympathie.
- Francesco Bonaldi (enseignant-chercheur au *Laboratoire de Modélisation Pluridisciplinaire et Simulations*), pour sa gestion du Master en Calcul Haute Performance et Simulation, et pour les inspirations qu'il m'a donné en mathématiques appliquées.
- Mathieu Martel (enseignant-chercheur au *Laboratoire de Modélisation Pluridisciplinaire et Simulations*), pour sa gestion du Master en Calcul Haute Performance et Simulation.
- David Defour (enseignant-chercheur à *ESPACE-DEV*), pour son ancienne gestion du Master en Calcul Haute Performance et Simulation, et ses conseils et recommandations.
- Tous les autres enseignant·e·s-chercheur·euse·s de l'Université de Perpignan Via Domitia, qui ont enseigné en Licence de Mathématiques, et en Master en Calcul Haute Performance et Simulation.

L'autre sujet traité lors de ce stage est *Analyse de sensibilité d'un modèle de ruissellement*.

1 Équations générales

Introduction

- Pour modéliser les rivières, il existe plusieurs modèles à équations aux dérivées partielles (voir [10], [8], [1], [13]).
- L'objectif de cette section est de définir les équations générales que l'on peut utiliser pour modéliser des rivières.

1.1 Équations de Navier-Stokes

Introduction

L'objectif de cette section est de définir les équations de Navier-Stokes.

Soient

$$L_x, L_y, L_z, L_t > 0, \quad \Omega_x :=]0, L_x[, \quad \Omega_y :=]0, L_y[, \quad \Omega_z :=]0, L_z[, \quad \Omega_x^* := [0, L_x], \quad \Omega_y^* := [0, L_y], \\ \Omega_z^* := [0, L_z], \quad \Omega_t :=]0, L_t[, \quad \Omega_t^* := [0, L_t],$$

et le problème (P_{NS}) suivant :

Trouver u_x, u_y, u_z et p telles que :

$$\left\{ \begin{array}{ll} \frac{\partial}{\partial t}(u_x) + u_x \frac{\partial}{\partial x}(u_x) + u_y \frac{\partial}{\partial y}(u_x) + u_z \frac{\partial}{\partial z}(u_x) = -\frac{1}{\rho} \frac{\partial}{\partial x}(p) + \nu \left(\frac{\partial^2}{\partial x^2}(u_x) + \frac{\partial^2}{\partial y^2}(u_x) + \frac{\partial^2}{\partial z^2}(u_x) \right) + g_x & \forall (x, y, z, t) \in \Omega_x \times \Omega_y \times \Omega_z \times \Omega_t \\ \frac{\partial}{\partial t}(u_y) + u_x \frac{\partial}{\partial x}(u_y) + u_y \frac{\partial}{\partial y}(u_y) + u_z \frac{\partial}{\partial z}(u_y) = -\frac{1}{\rho} \frac{\partial}{\partial y}(p) + \nu \left(\frac{\partial^2}{\partial x^2}(u_y) + \frac{\partial^2}{\partial y^2}(u_y) + \frac{\partial^2}{\partial z^2}(u_y) \right) + g_y & \forall (x, y, z, t) \in \Omega_x \times \Omega_y \times \Omega_z \times \Omega_t \\ \frac{\partial}{\partial t}(u_z) + u_x \frac{\partial}{\partial x}(u_z) + u_y \frac{\partial}{\partial y}(u_z) + u_z \frac{\partial}{\partial z}(u_z) = -\frac{1}{\rho} \frac{\partial}{\partial z}(p) + \nu \left(\frac{\partial^2}{\partial x^2}(u_z) + \frac{\partial^2}{\partial y^2}(u_z) + \frac{\partial^2}{\partial z^2}(u_z) \right) + g_z & \forall (x, y, z, t) \in \Omega_x \times \Omega_y \times \Omega_z \times \Omega_t, \\ \frac{\partial}{\partial x}(u_x) + \frac{\partial}{\partial y}(u_y) + \frac{\partial}{\partial z}(u_z) = 0 & \forall (x, y, z, t) \in \Omega_x \times \Omega_y \times \Omega_z \times \Omega_t \\ \text{conditions initiales} & \forall (x, y, z) \in \Omega_x \times \Omega_y \times \Omega_z \\ \text{conditions aux limites} & \forall t \in \Omega_t^* \end{array} \right.$$

avec :

$$u_x := u_x(x, y, z, t), \quad u_y := u_y(x, y, z, t), \quad u_z := u_z(x, y, z, t), \quad p := p(x, y, z, t), \\ g_x := g_y := 0, \quad g_z := -g, \quad \nu := \nu(x, y, z), \quad \rho := \rho(x, y, z), \quad \text{et} \quad g := 9.80665.$$

Définitions (nom des variables)

- On appelle et note Ω_x, Ω_y et Ω_z les *domaines de définition des équations en espace*.
- On appelle et note Ω_t le *domaine de définition des équations en temps*.
- On appelle et note x, y , et z les *coordonnées en espace* (en m).
- On appelle et note t le *temps* (en s).
- On appelle et note u_x, u_y , et u_z les *composantes de la vitesse d'écoulement de l'eau* selon les directions x, y et z (en $\text{m} \cdot \text{s}^{-1}$).
- On appelle et note p la *pression* (en $\text{kg} \cdot \text{m}^{-1} \cdot \text{s}^{-2}$).
- On appelle et note ν la *viscosité cinématique* (en $\text{m}^2 \cdot \text{s}^{-1}$).
- On appelle et note ρ la *masse volumique* (en $\text{kg} \cdot \text{m}^{-3}$).
- On appelle et note g_x, g_y et g_z les *composantes de gravité* selon les directions x, y et z (en $\text{m} \cdot \text{s}^{-2}$).
- On appelle et note g la *constante de gravité* (en $\text{m} \cdot \text{s}^{-2}$).

Commentaire

Les fonctions inconnues sont u_x, u_y, u_z et p .

Soient

$$\Omega_{\vec{x}} := \Omega_x \times \Omega_y \times \Omega_z, \quad \Omega_{\vec{x}}^* := \Omega_x^* \times \Omega_y^* \times \Omega_z^*, \quad \vec{x} := \begin{pmatrix} x \\ y \\ z \end{pmatrix}, \quad \vec{u} := \begin{pmatrix} u_x \\ u_y \\ u_z \end{pmatrix}, \quad \vec{g} := \begin{pmatrix} g_x \\ g_y \\ g_z \end{pmatrix},$$
$$\nabla := \begin{pmatrix} \frac{\partial}{\partial x} \\ \frac{\partial}{\partial y} \\ \frac{\partial}{\partial z} \end{pmatrix}, \quad \text{et} \quad \Delta := \frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2} + \frac{\partial^2}{\partial z^2}.$$

Alors, le problème (P_{NS}) se réécrit (sous forme compacte) :

Trouver $\vec{u}$ et p telles que :

$$\left\{ \begin{array}{ll} \frac{\partial}{\partial t} (\vec{u}) + \vec{u} (\nabla \cdot \vec{u}) = -\frac{1}{\rho} \nabla p + \nu \Delta \vec{u} + \vec{g} & \forall (\vec{x}, t) \in \Omega_{\vec{x}} \times \Omega_t \\ \nabla \cdot \vec{u} = 0 & \forall (\vec{x}, t) \in \Omega_{\vec{x}} \times \Omega_t \\ \text{conditions initiales} & \forall \vec{x} \in \Omega_{\vec{x}} \\ \text{conditions aux limites} & \forall t \in \Omega_t^* \end{array} \right. \quad (P_{\text{NS}})$$

Commentaires

- Le problème (P_{NS}) est la résolution des équations de Navier-Stokes en 3D.
- Le modèle par équations de Navier-Stokes est trop général, trop coûteux, et trop complexe pour les rivières. On ne choisira pas ce modèle. Cependant, il représente le modèle général d'un modèle que nous étudierons.

1.2 Équations de Saint-Venant en 2D

Introduction

- À partir des équations de Navier-Stokes, avec certaines hypothèses simplificatrices, on peut définir les équations de Saint-Venant en 2D.
- L'objectif de cette section est de définir les équations de Saint-Venant en 2D.

Théorème (d'après Abdul Hadi et al. 2012 [1])

Soient le problème (P_{NS}) , z_b l'altitude du fond de la rivière, z_s l'altitude de la surface de la rivière,

$$h := h(x, y, t) := z_s - z_b, \quad \vec{u} := \vec{u}(x, y, t) := \begin{pmatrix} u_x \\ u_y \end{pmatrix}, \quad \vec{e}_x := \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \quad \vec{e}_y := \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix},$$
$$\tau_b := \rho \nu \left[\frac{\partial}{\partial z} (\vec{u}) \right]_{z=z_b}, \quad r := r(x, y), \quad S_{f_x} := S_{f_x}(x, y, t) := \frac{r^2 u_x \sqrt{u_x^2 + u_y^2}}{h^{4/3}},$$

$$S_{f_y} := S_{f_y}(x, y, t) := \frac{r^2 u_y \sqrt{u_x^2 + u_y^2}}{h^{4/3}},$$

et les hypothèses suivantes :

$$\frac{\partial}{\partial z}(p) = -\rho g \quad (\text{hypothèse hydrostatique}),$$

$$u_z \ll u_x, u_y \quad (\text{écoulement horizontal}),$$

$$\forall \psi \in \{x, y, z\} : \int_{z_b}^{z_s} \left(\frac{\partial}{\partial t}(u_\psi) + (\vec{u} \cdot \nabla)u_\psi + \frac{1}{\rho} \frac{\partial}{\partial \psi}(p) - \nu \Delta u_\psi - g_\psi \right) dz = 0 \quad (\text{intégration verticale}),$$

et

$$\frac{1}{\rho}(\tau_b \cdot \vec{e}_x) = ghS_{f_x}, \quad \text{et} \quad \frac{1}{\rho}(\tau_b \cdot \vec{e}_y) = ghS_{f_y} \quad (\text{loi de frottement de Manning}).$$

Avec les hypothèses, le problème (P_{NS}) devient le problème (P_{SV2}).

Soient,

$$L_x, L_y, L_t > 0, \quad \Omega_x :=]0, L_x[, \quad \Omega_y :=]0, L_y[, \quad \Omega_x^* := [0, L_x], \quad \Omega_y^* := [0, L_y],$$

et le problème (P_{SV2}) suivant :

Trouver h, u_x et u_y telles que :

$$\left\{ \begin{array}{ll} \frac{\partial}{\partial t}(h) + \frac{\partial}{\partial x}(hu_x) + \frac{\partial}{\partial y}(hu_y) = q & \forall (x, y, t) \in \Omega_x \times \Omega_y \times \Omega_t \\ \frac{\partial}{\partial t}(hu_x) + \frac{\partial}{\partial x}\left(hu_x^2 + \frac{1}{2}gh^2\right) + \frac{\partial}{\partial y}(hu_x u_y) = gh(S_{0_x} - S_{f_x}) + qv_x & \forall (x, y, t) \in \Omega_x \times \Omega_y \times \Omega_t \\ \frac{\partial}{\partial t}(hu_y) + \frac{\partial}{\partial x}(hu_x u_y) + \frac{\partial}{\partial y}\left(hu_y^2 + \frac{1}{2}gh^2\right) = gh(S_{0_y} - S_{f_y}) + qv_y & \forall (x, y, t) \in \Omega_x \times \Omega_y \times \Omega_t \\ \text{condition initiales} & \forall (x, y) \in \Omega_x \times \Omega_y \\ \text{conditions aux limites} & \forall t \in \Omega_t^* \end{array} \right.,$$

avec :

$$q := q(x, y, t), \quad S_{0_x} := S_{0_x}(x, y) := -\frac{\partial}{\partial x}(z(x, y)), \quad S_{0_y} := S_{0_y}(x, y) := -\frac{\partial}{\partial y}(z(x, y)),$$

$$v_x := v_x(x, y, t), \quad \text{et} \quad v_y := v_y(x, y, t).$$

Définitions (nom des variables)

- On appelle et note h la *hauteur d'eau* (en m).
- On appelle et note u_x et u_y les *composantes de la vitesse d'écoulement de l'eau* selon les directions x et y (en $\text{m} \cdot \text{s}^{-1}$).
- On appelle et note q l'*apport du terme source* (en $\text{m}^2 \cdot \text{s}^{-1}$).
- On appelle et note v_x et v_y les *composantes de la vitesse de l'apport du terme source* (en $\text{m} \cdot \text{s}^{-1}$).
- On appelle et note r le *coefficient de rugosité (de Manning)* (en $\text{m}^{-1/3} \cdot \text{s}$).
- On appelle et note z l'*altitude du fond de la rivière* (en m).
- On appelle et note S_{0_x} et S_{0_y} les *composantes de la pente du fond* selon les directions x et y (sans dimension).

- On appelle et note S_{f_x} et S_{f_y} les *composantes de la pente de frottement* selon les directions x et y (sans dimension).

Commentaire

Les fonctions inconnues sont u_x , u_y et h .

Soient

$$\Omega_{\vec{x}} := \Omega_x \times \Omega_y, \quad \Omega_{\vec{x}}^* := \Omega_x^* \times \Omega_y^*, \quad \vec{x} := \begin{pmatrix} x \\ y \end{pmatrix}, \quad \vec{u} := \begin{pmatrix} u_x \\ u_y \end{pmatrix}, \quad \vec{S}_0 := \begin{pmatrix} S_{0_x} \\ S_{0_y} \end{pmatrix}, \quad \vec{S}_f := \begin{pmatrix} S_{f_x} \\ S_{f_y} \end{pmatrix},$$

$$\vec{v} := \begin{pmatrix} v_x \\ v_y \end{pmatrix}, \quad \nabla := \begin{pmatrix} \frac{\partial}{\partial x} \\ \frac{\partial}{\partial y} \end{pmatrix}, \quad \text{et} \quad I_2 := \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}.$$

Alors, le problème (P_{SV2}) se réécrit (sous forme compacte) :

Trouver h et $\vec{u}$ telles que :

$$\left\{ \begin{array}{ll} \frac{\partial}{\partial t}(h) + \nabla \cdot (h\vec{u}) = q & \forall (\vec{x}, t) \in \Omega_{\vec{x}} \times \Omega_t \\ \frac{\partial}{\partial t}(h\vec{u}) + \nabla \cdot \left(h\vec{u} \otimes \vec{u}^T + \frac{1}{2}gh^2 I_2 \right) = g(\vec{S}_0 - \vec{S}_f)h + q\vec{v} & \forall (\vec{x}, t) \in \Omega_{\vec{x}} \times \Omega_t \\ \text{condition initiales} & \forall \vec{x} \in \Omega_{\vec{x}} \\ \text{conditions aux limites} & \forall t \in \Omega_t^* \end{array} \right. \quad (\text{P}_{\text{SV2}})$$

Commentaires

- Un apport de terme source positif peut représenter de la pluie, du ruissellement latéral, ou de l'affluence. Un apport de terme source négatif peut représenter de l'infiltration, du débordement, ou de l'évaporation.
- Le terme source est une densité linéique.
- Le problème (P_{SV2}) est la résolution des équations de Saint-Venant en 2D.
- Le problème (P_{SV2}) est un système d'équations aux dérivées partielles d'ordre 1 non linéaire.
- Les équations de Saint-Venant peuvent être aussi appelées : équations de Barré de Saint-Venant ou équations des eaux peu profondes.
- Les équations de Saint-Venant sont des équations de propagation d'ondes.
- Un cas particulier des équations de Saint-Venant est l'équation des ondes linéarisées (traitées dans le projet *Analyse numérique d'équations aux dérivées partielles par différences finies et implémentation optimisée pour le calcul haute performance*), elle sont utilisées pour modéliser des vibrations de cordes.

1.3 Équations de Saint-Venant en 1D

Introduction

- À partir des équations de Saint-Venant 2D, avec certaines hypothèses simplificatrices, on peut définir les équations de Saint-Venant en 1D.
- L'objectif de cette section est de définir les équations de Saint-Venant en 1D.

Théorème (*admis*)

Soient le problème (P_{SV2}), s l'abscisse curviligne,

$$h := h(s, t), \quad u := u(s, t), \quad \vec{e}_s := \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad \eta \in [0, h], \quad b := b(s, \eta), \quad A := A(s, t) := \int_0^h b \, d\eta,$$

$$\mathcal{G} := \mathcal{G}(s, t) := \int_0^h (h - \eta) b \, d\eta, \quad P := P(s, t), \quad R := R(s, t) := \frac{A}{P},$$

$$r := r(s), \quad S_0 := S_0(s) := -\frac{d}{ds}(z), \quad \text{et} \quad S_f := S_f(s, t) := \frac{r^2 u |u|}{R^{4/3}},$$

et les hypothèses suivantes :

$$u_y \ll u_x \quad (\text{écoulement longitudinal}),$$

$$\frac{\partial}{\partial y}(h) \approx 0 \quad (\text{surface libre uniforme dans la section transverse}),$$

$$h(s, t) := \frac{1}{b(s)} \int_0^{b(s)} h(x, y, t) \, dy, \quad u(s, t) := \frac{1}{b(s)} \int_0^{b(s)} u_x(x, y, t) \, dy,$$

$$q(s, t) := \frac{1}{b(s)} \int_0^{b(s)} q(x, y, t) \, dy, \quad v(s, t) := \frac{1}{b(s)} \int_0^{b(s)} v_x(x, y, t) \, dy,$$

$$\vec{S}_0(x, y) \approx S_0(s) \vec{e}_s \quad (\text{pente du fond selon l'axe longitudinal}),$$

$$\vec{S}_f(x, y, t) \approx S_f(s, t) \vec{e}_s \quad (\text{pente de frottement selon l'axe longitudinal}).$$

Avec les hypothèses, le problème (P_{SV2}) devient le problème (P_{SV1}).

Soient

$$L_s, L_t > 0, \quad \Omega_s :=]0, L_s[, \quad \Omega_s^* := [0, L_s],$$

et le problème (P_{SV1}) suivant :

Trouver h et u telles que :

$$\left\{ \begin{array}{ll} \frac{\partial}{\partial t}(A) + \frac{\partial}{\partial s}(Au) = q & \forall (s, t) \in \Omega_s \times \Omega_t \\ \frac{\partial}{\partial t}(Au) + \frac{\partial}{\partial s}(Au^2 + g\mathcal{G}) = g(S_0 - S_f)A + qv & \forall (s, t) \in \Omega_s \times \Omega_t \\ \text{conditions initiales} & \forall s \in \Omega_s \\ \text{conditions aux limites} & \forall t \in \Omega_t^* \end{array} \right. \quad (\text{P}_{\text{SV1}})$$

Définitions (*nom des variables*)

- On appelle et note s l'*abscisse curviligne* (en m).
- On appelle et note A l'*aire mouillée* (en m²).
- On appelle et note u la *vitesse d'écoulement de l'eau* (en m · s⁻¹).
- On appelle et note $\mathcal{G}$ le *moment statique mouillé* (en m³).
- On appelle et note v la *vitesse de l'apport du terme source* (en m · s⁻¹).
- On appelle et note b la *largeur* (en m).
- On appelle et note S_0 la *pente du fond de la rivière* (sans dimension).

- On appelle et note S_f la *pente de frottement* (sans dimension).
- On appelle et note R le *rayon mouillé* (en m).
- On appelle et note P le *périmètre mouillé* (en m).

Commentaire

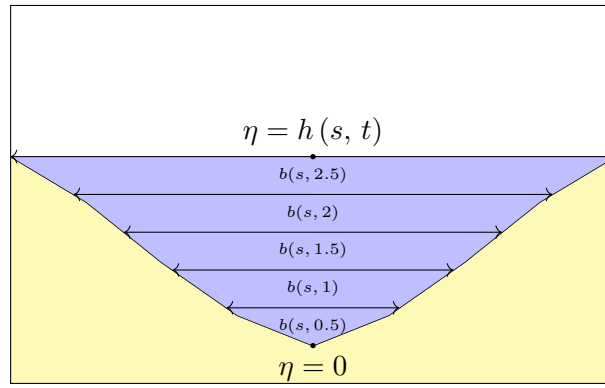
Les fonctions inconnues sont u et h .

Définitions

- On dit que s réalise l'*amont* (de la rivière) lorsque $s = 0$.
- On dit que s réalise l'*aval* (de la rivière) lorsque $s = L_s$.

Illustration

Soit $(s, t) \in \Omega_s^* \times \Omega_t^*$. Alors, on peut illustrer la rivière vue de face :



avec :

$$\blacksquare A(s, t) = \int_0^{h(s, t)} b(s, \eta) \, d\eta.$$

Commentaires

- Pour qualifier la géométrie d'une section de rivière vue de face, l'état de l'art parle de *section de travers*.
- A ce stade, les conditions initiales et les conditions aux limites ne sont pas encore définies explicitement. On les définira ultérieurement.
- Vue de face, on choisit $\eta := 0$ pour la hauteur d'eau la plus profonde. $b(s, \eta)$ représente la largeur totale occupée par de l'eau en (s, η) .
- Le problème (P_{SV1}) est la résolution des équations de Saint-Venant en 1D (ou aussi appelé pseudo-2D), pour une rivière de géométrie quelconque.
- Il existe le modèle par équations de Saint-Venant en 1D qui ne prend pas en compte la géométrie de la rivière. On ne s'intéressera pas à ce modèle.
- Pour toute la suite, on utilisera les équations de Saint-Venant, avec le problème (P_{SV1}) comme base pour les modèles de rivière.
- Pour toute la suite, par simplification, on choisira une section de travers trapézoïdale ou rectangulaire.

2 Modèle de rivière pour une section rectangulaire avec hypothèses dynamiques

Introduction

L'objectif de cette section est d'étudier un modèle de rivière pour une section trapézoïdale ou rectangulaire avec hypothèses dynamiques, associé au problème (P_{SV1}) .

2.1 Problème instationnaire

2.1.1 Présentation du problème

Soient le problème (P_{SV1}) , $(s, t) \in \Omega_s^* \times \Omega_t^*$, et $b(s, \eta) := b_0(s) + 2m(s)\eta$ (avec b_0 la largeur de la base du trapèze, et m la pente du trapèze). Alors :

$$\begin{aligned}
 A &= A(s, t) = \int_0^{h(s, t)} b(s, \eta) \, d\eta = \int_0^{h(s, t)} (b_0(s) + 2m(s)\eta) \, d\eta \\
 &= b_0(s) \int_0^{h(s, t)} d\eta + 2m(s) \int_0^{h(s, t)} \eta \, d\eta = b_0(s) [\eta]_{\eta=0}^{h(s, t)} + m(s) [\eta^2]_{\eta=0}^{h(s, t)} \\
 &= b_0(s) h(s, t) + m(s) h(s, t)^2 = h(s, t) (b_0(s) + m(s) h(s, t)) = h(b_0 + mh), \\
 \mathcal{G} &= \mathcal{G}(s, t) = \int_0^{h(s, t)} (h(s, t) - \eta) b(s, \eta) \, d\eta = \int_0^{h(s, t)} (h(s, t) - \eta) (b_0(s) + 2m(s)\eta) \, d\eta \\
 &= \int_0^{h(s, t)} (h(s, t) b_0(s) + 2m(s) h(s, t) \eta - b_0(s) \eta - 2m(s) \eta^2) \, d\eta \\
 &= h(s, t) b_0(s) \int_0^{h(s, t)} d\eta + 2m(s) h(s, t) \int_0^{h(s, t)} \eta \, d\eta - b_0(s) \int_0^{h(s, t)} \eta \, d\eta - 2m(s) \int_0^{h(s, t)} \eta^2 \, d\eta \\
 &= h(s, t) b_0(s) [\eta]_{\eta=0}^{h(s, t)} + m(s) h(s, t) [\eta^2]_{\eta=0}^{h(s, t)} - \frac{1}{2} b_0(s) [\eta^2]_{\eta=0}^{h(s, t)} - \frac{2}{3} m(s) [\eta^3]_{\eta=0}^{h(s, t)} \\
 &= h(s, t)^2 b_0(s) + m(s) h(s, t)^3 - \frac{1}{2} b_0(s) h(s, t)^2 - \frac{2}{3} m(s) h(s, t)^3 = h(s, t)^2 \left(\frac{1}{2} b_0(s) + \frac{1}{3} m(s) h(s, t) \right) \\
 &= h^2 \left(\frac{1}{2} b_0 + \frac{1}{3} mh \right),
 \end{aligned}$$

et :

$$\begin{aligned}
 P &= P(s, t) = b_0(s) + 2\sqrt{h(s, t)^2 + \left(m(s) [\eta]_{\eta=h(s, t)}\right)^2} \\
 &= b_0(s) + 2\sqrt{h(s, t)^2 + (m(s) h(s, t))^2} = b_0(s) + 2h(s, t) \sqrt{1 + m(s)^2} = b_0 + 2h\sqrt{1 + m^2}.
 \end{aligned}$$

En particulier, si la rivière est de section rectangulaire, alors :

$$m = 0, \quad b(s, \eta) = b_0 := b, \quad A = bh, \quad \mathcal{G} = \frac{1}{2}bh^2, \quad \text{et} \quad P = b + 2h.$$

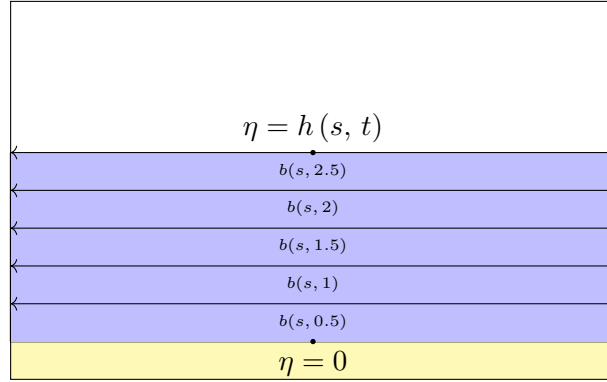
Donc le problème (P_{SV1}) devient le problème (P_{SV1r}) qui s'écrit :

Trouver h et u telles que :

$$\left\{ \begin{array}{ll} b \frac{\partial}{\partial t} (h) + \frac{\partial}{\partial s} (bhu) = q & \forall (s, t) \in \Omega_s \times \Omega_t \\ b \frac{\partial}{\partial t} (hu) + \frac{\partial}{\partial s} \left(bh \left(u^2 + \frac{1}{2}gh \right) \right) = g(S_0 - S_f)bh + qv & \forall (s, t) \in \Omega_s \times \Omega_t \\ \text{conditions initiales} & \forall s \in \Omega_s \\ \text{conditions aux limites} & \forall t \in \Omega_t^* \end{array} \right.$$

Illustration

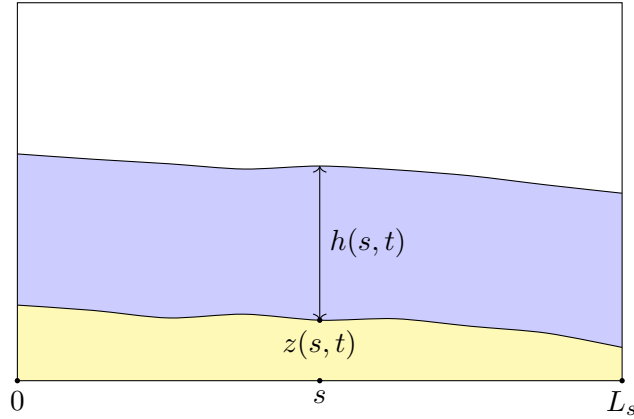
Soit $(s, t) \in \Omega_s^* \times \Omega_t^*$. Alors, on peut illustrer la rivière vue de face :



avec :

$$\blacksquare A(s, t) = b(s) h(s, t).$$

Soit $t \in \Omega_t^*$. Alors, on peut illustrer la rivière vue de profil :



Définitions (termes des équations)

- On appelle les *termes d'inertie* l'expression : $b \frac{\partial}{\partial t} (hu) + \frac{\partial}{\partial s} (bhu^2)$.
- On appelle le *terme de pression* l'expression : $\frac{\partial}{\partial s} \left(\frac{1}{2}gbh^2 \right)$.
- On appelle le *terme d'advection* l'expression : $\frac{\partial}{\partial s} (bhu^2)$.

Définition (*noms des variables*)

On appelle le *débit d'écoulement* (en $\text{m}^3 \cdot \text{s}^{-1}$) l'expression : $Q := Au$.

Définitions (*type des variables*)

- On dit que les variables A et Q sont des *variables conservatives*.
- On dit que les variables h et u sont des *variables physiques*.
- On dit que les variables A et h sont des *variables de masse*.
- On dit que les variables Q et u sont des *variables de quantité de mouvement*.

Soient $h \in \mathbb{R}_+^*$, et $u \in \mathbb{R}$. Alors :

$$\begin{aligned} b \frac{\partial}{\partial t} (h) + \frac{\partial}{\partial s} (bhu) &= \frac{\partial}{\partial t} (A) + \frac{\partial}{\partial s} (Q), \\ b \frac{\partial}{\partial t} (hu) + \frac{\partial}{\partial s} \left(bh \left(u^2 + \frac{1}{2}gh \right) \right) - g(S_0 - S_f)bh \\ &= \frac{\partial}{\partial t} (Q) + \frac{\partial}{\partial s} \left(\frac{Q^2}{A} + \frac{1}{2}g \frac{A^2}{b} \right) - g(S_0 - S_f)A, \end{aligned}$$

et :

$$S_f = \frac{r^2 Q |Q|}{A^2 R^{4/3}}.$$

Donc le problème (P_{SV1r}) se réécrit :

Trouver A et Q telles que :

$$\left\{ \begin{array}{ll} \frac{\partial}{\partial t} (A) + \frac{\partial}{\partial s} (Q) = q & \forall (s, t) \in \Omega_s \times \Omega_t \\ \frac{\partial}{\partial t} (Q) + \frac{\partial}{\partial s} \left(\frac{Q^2}{A} + \frac{1}{2}g \frac{A^2}{b} \right) = g(S_0 - S_f)A + qv & \forall (s, t) \in \Omega_s \times \Omega_t, \\ \text{conditions initiales} & \forall s \in \Omega_s \\ \text{conditions aux limites} & \forall t \in \Omega_t^* \end{array} \right.$$

ou encore :

Trouver A et Q telles que :

$$\boxed{\left\{ \begin{array}{ll} \frac{\partial}{\partial t} (U) + \frac{\partial}{\partial s} (F(U)) = S(U) & \forall (s, t) \in \Omega_s \times \Omega_t \\ \text{conditions initiales} & \forall s \in \Omega_s \\ \text{conditions aux limites} & \forall t \in \Omega_t^* \end{array} \right.}, \quad (P_{SV1r})$$

avec :

$$U := \begin{pmatrix} A \\ Q \end{pmatrix}, \quad F(U) := \begin{pmatrix} Q \\ \frac{Q^2}{A} + \frac{1}{2}g \frac{A^2}{b} \end{pmatrix}, \quad \text{et} \quad S(U) := \begin{pmatrix} q \\ g(S_0 - S_f)A + qv \end{pmatrix}.$$

Définitions (*termes conservatifs*)

- On appelle le *flux conservatif* l'application F .

— On appelle le *terme source* l'application S .

Définition

On dit que le problème (P_{SV1r}) possède des hypothèses *dynamiques*.

Commentaires

- Le nom des hypothèses ne dépend pas de la section de travers, mais il dépend des termes qui modélisent des phénomènes physiques dans l'équation.
- On verra qu'il existe des hypothèses simplifiées.
- Les fonctions inconnues deviennent A et Q . Pour la suite, on utilisera A et Q , ou h et u selon les besoins.

Remarque (condition nécessaire de conservation de la masse)

Si $q := 0$, alors :

$$\begin{aligned}
 & \frac{\partial}{\partial t} (A) + \frac{\partial}{\partial s} (Q) = q \\
 & \Leftrightarrow \frac{\partial}{\partial t} (A(s, t)) = - \frac{\partial}{\partial s} (Q(s, t)) \\
 & \Rightarrow \int_0^{L_s} \frac{\partial}{\partial t} (A(s, t)) \, ds = - \int_0^{L_s} \frac{\partial}{\partial s} (Q(s, t)) \, ds \\
 & \Rightarrow \frac{d}{dt} \left(\int_0^{L_s} A(s, t) \, ds \right) = Q(0, t) - Q(L_s, t) \\
 & \Leftrightarrow \boxed{\frac{d}{dt} (V) = Q(0, t) - Q(L_s, t)}, \tag{2.1}
 \end{aligned}$$

avec :

$$V := V(t) := \int_0^{L_s} A(s, t) \, ds.$$

Donc (2.1) est une condition nécessaire pour que (P_{SV1r}) soit satisfaite.

Définition

On appelle et note V le *volume mouillé de la rivière* (en m^3).

2.1.2 Linéarisation du problème, et conditions aux limites

Introduction

L'objectif de cette section est de linéariser le problème (P_{SV1r}) .

Soient $F_1(U) := Q$, et $F_2(U) := \frac{Q^2}{A} + \frac{1}{2}g\frac{A^2}{b}$. Alors :

$$\frac{\partial}{\partial U} (F(U)) = \begin{pmatrix} \frac{\partial}{\partial A} (F_1(U)) & \frac{\partial}{\partial Q} (F_1(U)) \\ \frac{\partial}{\partial A} (F_2(U)) & \frac{\partial}{\partial Q} (F_2(U)) \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ -\left(\frac{Q}{A}\right)^2 + g\frac{A}{b} & 2\frac{Q}{A} \end{pmatrix} =: J(U) =: J,$$

et :

$$\frac{\partial}{\partial s} (F(U)) = \frac{\partial}{\partial U} (F(U)) \frac{\partial}{\partial s} (U) = J(U) \frac{\partial}{\partial s} (U).$$

Donc :

$$\begin{aligned} & \frac{\partial}{\partial t}(U) + \frac{\partial}{\partial s}(F(U)) = 0 \\ \Leftrightarrow & \boxed{\frac{\partial}{\partial t}(U) + J(U) \frac{\partial}{\partial s}(U) = 0}. \end{aligned} \quad (2.2)$$

Soient χ_J le polynôme caractéristique de J , Sp_J le spectre de J , et $\mathcal{GL}_2(\mathbb{R})$ le groupe des matrices inversibles 2×2 à coefficients réels. Alors :

$$\begin{aligned} \chi_J(\lambda) &= \left| \begin{array}{cc} -\lambda & 1 \\ -\left(\frac{Q}{A}\right)^2 + g\frac{A}{b} & 2\frac{Q}{A} - \lambda \end{array} \right| = \left| \begin{array}{cc} -\lambda & 1 \\ -u^2 + gh & 2u - \lambda \end{array} \right| = (\lambda - u)^2 - gh = 0 \\ &\Leftrightarrow \lambda = u \pm \sqrt{gh}. \\ &\Leftrightarrow \text{Sp}_J = \{u - \sqrt{gh}, u + \sqrt{gh}\} =: \{\lambda_-, \lambda_+\}. \end{aligned}$$

Donc (d'après le théorème de la diagonalisation) :

$$\exists P \in \mathcal{GL}_2(\mathbb{R}) : J = P \begin{pmatrix} \lambda_- & 0 \\ 0 & \lambda_+ \end{pmatrix} P^{-1}.$$

Soit $\text{Ker}(J) := \{v \in \mathbb{R}^2 \mid Jv = 0_{\mathbb{R}^2}\}$ le noyau de la matrice J . Alors, les sous-espaces vectoriels propres de J sont les suivants :

$$\begin{aligned} & \text{Ker}(J - \lambda_- I), \quad \text{et} \quad \text{Ker}(J - \lambda_+ I). \\ J - \lambda_- I &= \begin{pmatrix} -u + \sqrt{gh} & 1 \\ -u^2 + gh & u + \sqrt{gh} \end{pmatrix}, \quad \text{et} \quad J - \lambda_+ I = \begin{pmatrix} -u - \sqrt{gh} & 1 \\ -u^2 + gh & u - \sqrt{gh} \end{pmatrix}, \\ & \begin{pmatrix} -u + \sqrt{gh} & 1 \\ -u^2 + gh & u + \sqrt{gh} \end{pmatrix} v = 0_{\mathbb{R}^2} \\ & \Leftrightarrow (-u^2 + gh \quad u + \sqrt{gh}) v = 0_{\mathbb{R}^2} \\ & \Leftrightarrow \text{Ker}(J - \lambda_- I) = \text{Vect} \left\{ \begin{pmatrix} u + \sqrt{gh} \\ u^2 - gh \end{pmatrix} \right\} = \text{Vect} \left\{ \begin{pmatrix} 1 \\ u - \sqrt{gh} \end{pmatrix} \right\}, \end{aligned}$$

et :

$$\begin{aligned} & \begin{pmatrix} -u - \sqrt{gh} & 1 \\ -u^2 + gh & u - \sqrt{gh} \end{pmatrix} v = 0_{\mathbb{R}^2} \\ & \Leftrightarrow (-u^2 + gh \quad u - \sqrt{gh}) v = 0_{\mathbb{R}^2} \\ & \Leftrightarrow \text{Ker}(J - \lambda_+ I) = \text{Vect} \left\{ \begin{pmatrix} u - \sqrt{gh} \\ u^2 - gh \end{pmatrix} \right\} = \text{Vect} \left\{ \begin{pmatrix} 1 \\ u + \sqrt{gh} \end{pmatrix} \right\}. \end{aligned}$$

Donc (d'après le théorème de la diagonalisation) :

$$\mathbb{R}^2 = \text{Ker}(J - \lambda_- I) \oplus \text{Ker}(J - \lambda_+ I), \quad \text{et} \quad P = \begin{pmatrix} 1 & 1 \\ u - \sqrt{gh} & u + \sqrt{gh} \end{pmatrix}.$$

Soit $c := \sqrt{gh}$. Alors :

$$P = \begin{pmatrix} 1 & 1 \\ u - c & u + c \end{pmatrix}, \quad \det(P) = 2c, \quad \text{et} \quad P^{-1} = \frac{1}{2c} \begin{pmatrix} u + c & -1 \\ -u + c & 1 \end{pmatrix}.$$

Définitions (physique)

- On appelle les *vitesse caractéristique absolue de l'onde* les vitesses auxquelles se transmettent les ondes, relativement au référentiel fixe.
- On appelle la *vitesse caractéristique relative de l'onde* les vitesses auxquelles se transmettent les ondes, relativement au référentiel d'une particule de la rivière.

Théorème (de propagation des ondes caractéristiques, admis)

Soient $J \in M_2(\mathbb{R})$, $U \in \mathbb{R}^2$, et le système d'équations aux dérivées partielles hyperboliques linéarisées suivant :

$$\frac{\partial}{\partial t}(U) + J \frac{\partial}{\partial s}(U) = 0.$$

Alors :

Si J est diagonalisable (avec $\text{Sp}_J = \{\lambda_1, \lambda_2\}$, et $\lambda_1 \neq \lambda_2$), alors : il y a 2 ondes caractéristiques qui ont une vitesse caractéristique d'onde égales à respectivement λ_1 et λ_2 .

Commentaires

- D'après le théorème précédent, les vitesses caractéristiques absolues de l'ondes sont les valeurs λ_- et λ_+ , et les vitesses relatives de l'onde sont les valeurs $-c$ et c .
- Si $c = 0$, alors l'onde se propage à la même vitesse que la vitesse d'écoulement de l'eau.
- Si $u < 0$ et $c < u$, alors l'onde remonte vers l'amont de la rivière.

Définition (physique)

On appelle un *remous dynamique* lorsqu'une onde remonte vers l'amont de la rivière.

Remarques

- Soient $\lambda^* := |u| + \sqrt{gh}$, et T le temps minimal nécessaire à une des ondes pour parcourir le segment Ω_s . Alors :

$$T = \frac{L_s}{\lambda^*}. \quad (2.3)$$

- On utilisera cette égalité lors de la discrétisation des équations (avec des sous-intervalles de Ω_s et de Ω_t), pour définir une condition de stabilité des schémas numériques explicites.

Définition

On appelle le *nombre de Froude* l'expression $\text{Fr} := \frac{|u|}{c}$.

Définitions

Soit $(s, t) \in \Omega_s^* \times \Omega_t^*$. Alors :

- On dit que le point (s, t) suit un *écoulement en régime subcritique* lorsque : $\text{Fr}(s, t) < 1$.
- On dit que le point (s, t) suit un *écoulement en régime critique* lorsque : $\text{Fr}(s, t) = 1$.
- On dit que le point (s, t) suit un *écoulement en régime supercritique* lorsque : $\text{Fr}(s, t) > 1$.

Remarques

Si $\text{Fr} < 1$, alors : $\frac{u}{c} < 1 \Leftrightarrow u - c < 0 \Leftrightarrow \lambda_- < 0$ et $u + c < 2c$.

Si $Fr = 1$, alors : $\frac{u}{c} = 1 \Leftrightarrow u - c = 0 \Leftrightarrow \lambda_- = 0$ et $u + c = 2c > 0 \Leftrightarrow \lambda_+ > 0$.

Si $Fr > 1$, alors : $\frac{c}{\sqrt{gh}} > 1 \Leftrightarrow u - c > 0 \Leftrightarrow \lambda_- > 0$ et $u + c > 2c > 0 \Leftrightarrow \lambda_+ > 0$.

↓ Valeur propre $Fr \rightarrow$	$Fr < 1$	$Fr = 1$	$Fr > 1$
λ_-	< 0	0	> 0
λ_+	indéterminé	> 0	> 0

En pratique, si $Fr < 1$, alors on a généralement $\lambda_+ > 0$.

Si les limites de la rivière ne sont pas des murs, alors on définit le nombre de conditions aux limites de (P_{SV1r}) de la manière suivante :

↓ Valeurs propres Limite $\rightarrow$	Amont	Aval
$\lambda_- < 0$ et $\lambda_+ < 0$	2	0
$\lambda_- < 0$ et $\lambda_+ > 0$	1	1
$\lambda_- = 0$ et $\lambda_+ > 0$	1	1
$\lambda_- > 0$ et $\lambda_+ > 0$	0	2

Si il y a 1 condition aux limites à l'amont de la rivière, et 1 condition aux limites à l'aval de la rivière, alors les choix possibles sont les suivants :

Conditions aux limites	Validité
A à l'amont et A à l'aval	non
A à l'amont et Q à l'aval	oui
Q à l'amont et A à l'aval	oui
Q à l'amont et Q à l'aval	non

2.1.3 Solutions exactes

Introduction

L'objectif de cette section est de présenter des exemples de solutions exactes existantes dans l'état de l'art.

D'après *Hudson 1999 [6]*, il existe une solution exacte de (P_{SV1r}) , avec les paramètres suivants : $\Omega_s :=]0, 1[$, $\Omega_t :=]0, 0.1[$, $b := 1$, et $r := S_0 := q := v := 0$, et les conditions initiales suivantes :

$$h(s, 0) := \begin{cases} 1 & \text{si } 0 \leq s \leq \frac{1}{2} \\ \frac{1}{2} & \text{si } \frac{1}{2} < s \leq 1 \end{cases} \quad \text{et} \quad u(s, 0) := 0,$$

et les conditions aux limites suivantes (pour des limites considérées comme des murs physiques) :

$$u(0, t) := u(L_s, t) := 0.$$

Alors, la solution exacte est la suivante :

$$h(s, t) = \begin{cases} 1 & \text{si } s < \frac{1}{2} - t\sqrt{g} \\ \frac{1}{9g} \left(2\sqrt{g} - \frac{2s-1}{2t} \right)^2 & \text{si } \frac{1}{2} - t\sqrt{g} \leq s \leq (u_2 - c_2)t + \frac{1}{2} \\ \frac{1}{4} \left(\sqrt{1 + \frac{16\psi^2}{g}} - 1 \right) & \text{si } (u_2 - c_2)t + \frac{1}{2} < s \leq \psi t + \frac{1}{2} \\ \frac{1}{2} & \text{si } s > \psi t + \frac{1}{2} \end{cases}, \quad (2.4)$$

et :

$$u(s, t) = \begin{cases} 0 & \text{si } s < \frac{1}{2} - t\sqrt{g} \\ \frac{1}{3t} (2s - 1 + 2t\sqrt{g}) & \text{si } \frac{1}{2} - t\sqrt{g} \leq s \leq (u_2 - c_2)t + \frac{1}{2} \\ u_2 & \text{si } (u_2 - c_2)t + \frac{1}{2} < s \leq \psi t + \frac{1}{2} \\ 0 & \text{si } s > \psi t + \frac{1}{2} \end{cases}, \quad (2.5)$$

avec :

$$u_2 := \psi - \frac{g}{8\psi} \left(1 + \sqrt{1 + \frac{16\psi^2}{g}} \right), \quad c_2 := \sqrt{\frac{g}{4} \left(\sqrt{1 + \frac{16\psi^2}{g}} - 1 \right)},$$

et $\psi := 2.957918120187525$.

D'après *Ippen 1966* [7], il existe une solution exacte de (P_{SV1r}) , avec les paramètres suivants :

$\Omega_s :=]0, 10000[$, $\Omega_t :=]0, 14400[$, $b \equiv 1000$, $r := 0.025$, et $S_0 \equiv q \equiv v \equiv 0$. Alors, la solution exacte est la suivante :

$$h(s, t) = H + a_0 \left(e^{\mu s} \cos(\sigma t - \kappa s) + e^{-\mu s} \cos(\sigma t - \kappa s) \right), \quad (2.6)$$

et :

$$u(s, t) = \frac{a_0}{H} \cdot \frac{\sigma}{\sqrt{\mu^2 + \kappa^2}} \left(e^{\mu s} \cos(\sigma t + \kappa s + \alpha) - e^{-\mu s} \cos(\sigma t - \kappa s + \alpha) \right), \quad (2.7)$$

avec :

$$a_0 := 1, \quad \sigma := \frac{2\pi}{T}, \quad T := 7200, \quad \kappa := \sqrt{\frac{1}{2} \left(\kappa_0^2 + \sqrt{\kappa_0^4 \left(1 + \left(\frac{gM}{\sigma} \right)^2} \right)} \right)}, \quad \kappa_0 := \frac{\sigma}{\sqrt{gH}},$$

$$H := 20, \quad M := \frac{f}{3\pi} \frac{u_{\max}}{gH}, \quad f := \frac{8g}{C^2}, \quad C := \frac{H^{1/6}}{r}, \quad u_{\max} := 1, \quad \mu := \frac{gM}{\sigma} \frac{k_0^2}{2\kappa},$$

et $\alpha := \arctan \left(\frac{\mu}{\kappa} \right)$.

2.1.4 Solutions approchées

Introduction

- En général, pour des paramètres quelconques, il n'existe de solution exacte connue de (P_{SV1r}) . Pour utiliser le modèle, il faut calculer une solution approchée avec une méthode d'analyse numérique.
- L'objectif de cette section est de présenter des solutions approchées.

2.1.4.1 Méthodes numériques et schémas numériques

- Pour trouver une solution approchée de (P_{SV1r}) , on discrétise l'espace et le temps selon différentes méthodes numériques et schémas numériques. Pour discrétiser l'espace, on utilisera la méthode des différences finies ou des volumes finis. Pour discrétiser le temps, on utilisera la méthode des différences finies.
- En espace, on utilise K sous-intervalles. En temps, on utilise N sous-intervalles. On pose $U_{k,n} := U(s_k, t_n)$.
- On utilise les maillages suivants :

↓ Méthode Variable →	Espace	Temps
Différences finies	$s_k := \sum_{\kappa=1}^{k-1} (\Delta_s)_\kappa,$ et $k \in \{1, \dots, K+1\}$	$t_n := \sum_{\kappa=1}^{n-1} (\Delta_t)_\kappa,$ et $n \in \{1, \dots, N+1\}$
Volumes finis	$s_k := \sum_{\kappa=1}^{k-1} ((\Delta_s)_\kappa) + \frac{1}{2} (\Delta_s)_k,$ $s_{k+1/2} := \sum_{\kappa=1}^k (\Delta_s)_\kappa,$ $s_{k-1/2} := \sum_{\kappa=1}^{k-1} (\Delta_s)_\kappa,$ et $k \in \{1, \dots, K\}$	/

- Pour les conditions initiales et les conditions aux limites, on les stocke de manière suivante :

↓ Méthode Variable →	Espace	Temps
Différences finies	s_1 et s_{K+1}	t_0
Volumes finis	Nœuds fantômes $s_{1/2}$ et $s_{K+1/2}$	/

- On calcule la solution approchée sur des nœuds ou des cellules :

↓ Méthode Variable →	Espace	Temps
Différences finies	Nœud s_k	Nœud t_n
Volumes finis	Cellule $C_k := [s_{k-1/2}, s_{k+1/2}]$	/

- En chaque nœud et / ou cellule du maillage espace-temps, on approxime les fonctions A et Q .
- Le maillage en espace est connu avant la simulation, et n'est pas remaillé au cours du temps. Le maillage en temps est adaptatif pendant la simulation. On définira ultérieurement les conditions sur le pas de temps.
- Pour toute la suite, on utilisera ces maillages.

Définitions

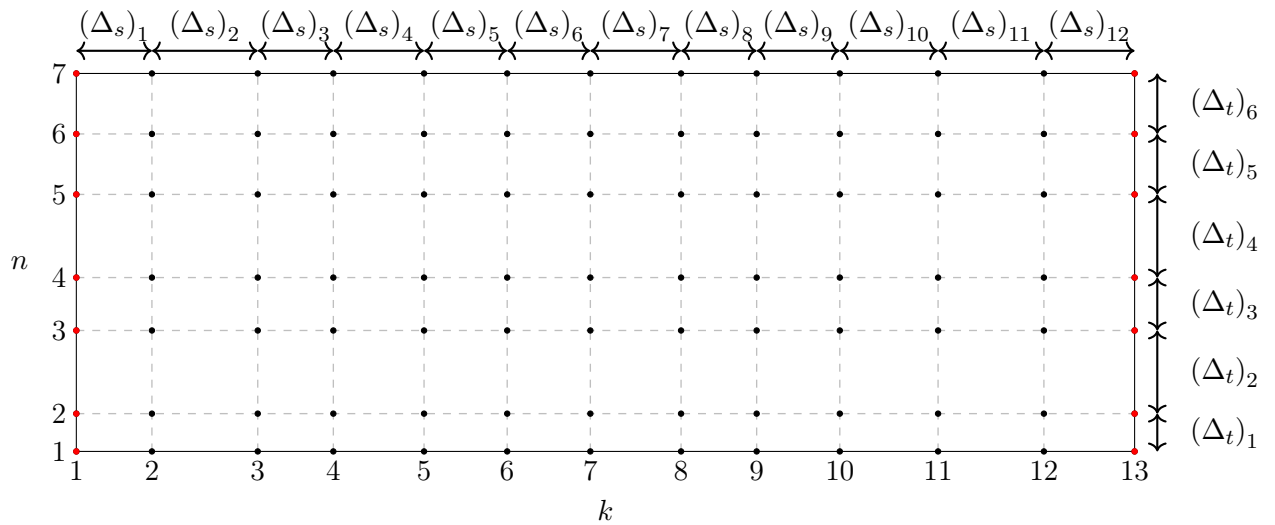
- On appelle un *schéma aux différences finies* un schéma qui utilise une discrétisation aux différences finies.
- On appelle un *schéma aux volumes finis* un schéma qui utilise une discrétisation en espace aux volumes finis
- Pour un schéma aux volumes finis, on appelle une *interface* un point en espace situé sur la frontière d'une cellule.

Commentaires

- On utilise les définitions précédentes indépendamment de la discrétisation en temps.
- Pour un schéma aux différences finies, on calcule la solution approchée en espace sur des nœuds.
- Pour un schéma aux volumes finis, on calcule la solution approchée en espace sur des cellules (chacune représentées par un nœud en leur centre).

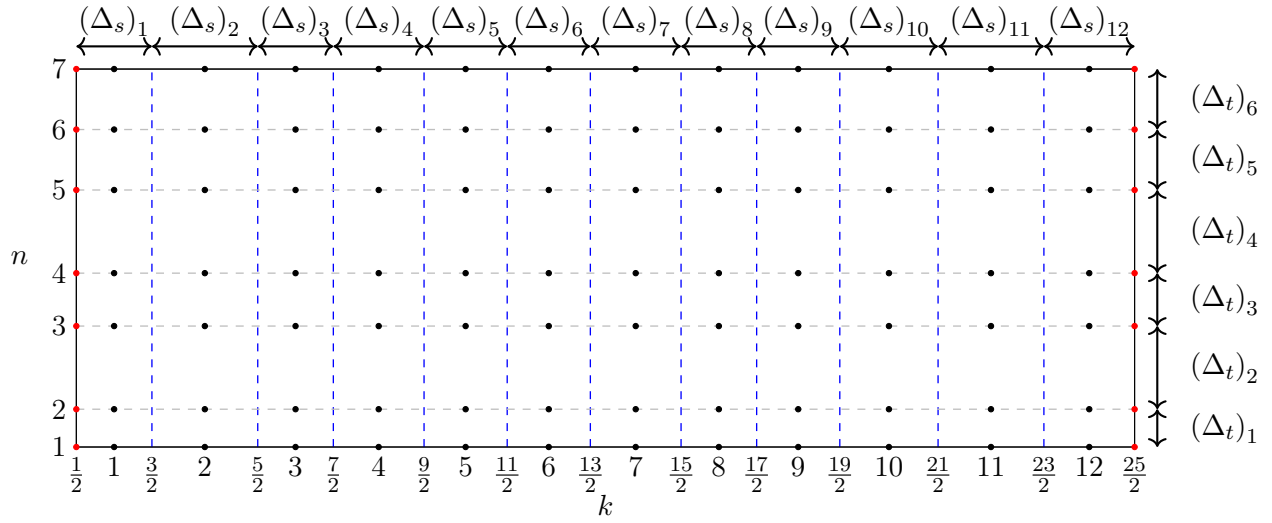
Illustrations

Exemple d'un maillage espace-temps avec la méthode des différences finies en espace, et la méthode des différences finies en temps, avec $K := 12$, et $N := 6$:



- Nœud
- Nœud de limite

Exemple d'un maillage espace-temps avec la méthode des volumes finis en espace, et la méthode des différences finies en temps, avec $K := 12$, et $N := 6$.



- Centre d'une cellule
- Nœud fantôme de limite
- - - Interface

Notation

On note ∞_f un nombre flottant ou bien très grand, ou bien égal à **nan**, ou bien égal à **inf**.

2.1.4.2 Discrétisation de l'espace avec la méthode des différences finies

On a (avec la formule de Taylor en avant et en arrière à l'ordre 2) les égalités suivantes :

$$U_{k+1,n} = U_{k,n} + (\Delta_s)_k \frac{\partial}{\partial s} (U_{k,n}) + O\left((\Delta_s)_k^2\right),$$

et

$$U_{k-1,n} = U_{k,n} - (\Delta_s)_{k-1} \frac{\partial}{\partial s} (U_{k,n}) + O\left((\Delta_s)_{k-1}^2\right).$$

On fait les approximations suivantes :

$$O\left((\Delta_s)_k^2\right) \simeq O\left((\Delta_s)_{k-1}^2\right) \simeq 0.$$

Donc :

$$\begin{aligned} U_{k+1,n} - U_{k-1,n} &\simeq ((\Delta_s)_k + (\Delta_s)_{k-1}) \frac{\partial}{\partial s} (U_{k,n}) \\ \Leftrightarrow \frac{\partial}{\partial s} (U_{k,n}) &\simeq \frac{1}{(\Delta_s)_k + (\Delta_s)_{k-1}} (U_{k+1,n} - U_{k-1,n}), \end{aligned} \quad (2.8)$$

ou encore :

$$\frac{\partial}{\partial s} (U_{k,n}) \simeq \frac{1}{(\Delta_s)_{k-1}} (U_{k,n} - U_{k-1,n}). \quad (2.9)$$

Commentaires

— L'approximation (2.9) n'est utilisable que lorsque les ondes se transmettent de l'amont vers l'aval.

2.1.4.3 Discrétisation de l'espace avec la méthode des volumes finis

On fait l'approximation suivante :

$$\begin{aligned} U_{k,n} &\simeq \frac{1}{(\Delta_s)_k} \int_{C_k} U(s, t) \, ds, \\ \Leftrightarrow \boxed{\int_{C_k} U(s, t) \, ds &\simeq (\Delta_s)_k U_{k,n}}. \end{aligned} \quad (2.10)$$

Commentaire

Cette approximation suppose que la solution est identique en tout point de la cellule C_k .

2.1.4.4 Discrétisation du temps avec la méthode des différences finies

On a (avec la formule de Taylor en avant et en arrière à l'ordre 2) les égalités suivantes :

$$U_{k,n+1} = U_{k,n} + (\Delta_t)_n \frac{\partial}{\partial t} (U_{k,n}) + O\left((\Delta_t)_n^2\right),$$

et

$$U_{k,n-1} = U_{k,n} - (\Delta_t)_{n-1} \frac{\partial}{\partial t} (U_{k,n}) + O\left((\Delta_t)_{n-1}^2\right)$$

On fait les approximations suivantes :

$$O\left((\Delta_t)_n^2\right) \simeq O\left((\Delta_t)_{n-1}^2\right) \simeq 0.$$

Donc :

$$\boxed{\frac{\partial}{\partial t} (U_{k,n}) \simeq \frac{1}{(\Delta_t)_n} (U_{k,n+1} - U_{k,n})}, \quad (2.11)$$

ou encore :

$$\boxed{\frac{\partial}{\partial t} (U_{k,n}) \simeq \frac{1}{(\Delta_t)_n} (U_{k,n} - U_{k,n-1})}. \quad (2.12)$$

Commentaires

- L'approximation (2.11) permettra d'obtenir des schémas explicites.
- L'approximation (2.12) permettra d'obtenir des schémas implicites.

2.1.4.5 Schéma aux différences finies de Lax-Friedrichs

D'après les approximations (2.8) et (2.11), on obtient :

$$\begin{aligned} &\frac{\partial}{\partial t} (U_{k,n}) + \frac{\partial}{\partial s} (F(U_{k,n})) \simeq S(U_{k,n}) \\ \Leftrightarrow &\frac{1}{(\Delta_t)_n} (U_{k,n+1} - U_{k,n}) + \frac{1}{(\Delta_s)_k + (\Delta_s)_{k-1}} (F(U_{k+1,n}) - F(U_{k-1,n})) \simeq S(U_{k,n}) \end{aligned}$$

$$\begin{aligned}
&\Leftrightarrow \frac{1}{(\Delta t)_n} (U_{k,n+1} - U_{k,n}) \simeq -\frac{1}{(\Delta s)_k + (\Delta s)_{k-1}} (F(U_{k+1,n}) - F(U_{k-1,n})) + S(U_{k,n}) \\
&\Leftrightarrow U_{k,n+1} - U_{k,n} \simeq -\frac{(\Delta t)_n}{(\Delta s)_k + (\Delta s)_{k-1}} (F(U_{k+1,n}) - F(U_{k-1,n})) + (\Delta t)_n S(U_{k,n}) \\
&\Leftrightarrow U_{k,n+1} \simeq -\frac{(\Delta t)_n}{(\Delta s)_k + (\Delta s)_{k-1}} (F(U_{k+1,n}) - F(U_{k-1,n})) + U_{k,n} + (\Delta t)_n S(U_{k,n}).
\end{aligned}$$

On fait l'approximation (de Lax-Friedrichs) suivante :

$$U_{k,n} \simeq \frac{(\Delta s)_{k-1} U_{k+1,n} + (\Delta s)_k U_{k-1,n}}{(\Delta s)_k + (\Delta s)_{k-1}}.$$

Donc :

$$\begin{aligned}
&U_{k,n+1} \\
&\simeq -\frac{(\Delta t)_n}{(\Delta s)_k + (\Delta s)_{k-1}} (F(U_{k+1,n}) - F(U_{k-1,n})) \\
&\quad + \frac{(\Delta s)_{k-1} U_{k+1,n} + (\Delta s)_k U_{k-1,n}}{(\Delta s)_{k-1} + (\Delta s)_k} \\
&\quad + (\Delta t)_n S(U_{k,n}). \\
&\Leftrightarrow \left\{ \begin{aligned} &A_{k,n+1} \\ &\simeq -\frac{(\Delta t)_n}{(\Delta s)_k + (\Delta s)_{k-1}} (Q_{k+1,n} - Q_{k-1,n}) \\ &\quad + \frac{1}{(\Delta s)_{k-1} + (\Delta s)_k} ((\Delta s)_{k-1} A_{k+1,n} + (\Delta s)_k A_{k-1,n}) \\ &\quad + (\Delta t)_n q_{k,n} \\ &Q_{k,n+1} \\ &\simeq -\frac{(\Delta t)_n}{(\Delta s)_k + (\Delta s)_{k-1}} \left(\frac{Q_{k+1,n}^2}{A_{k+1,n}} - \frac{Q_{k-1,n}^2}{A_{k-1,n}} + \frac{1}{2} g \left(\frac{A_{k+1,n}^2}{b_{k+1}} - \frac{A_{k-1,n}^2}{b_{k-1}} \right) \right) \\ &\quad + \frac{1}{(\Delta s)_{k-1} + (\Delta s)_k} ((\Delta s)_{k-1} Q_{k+1,n} + (\Delta s)_k Q_{k-1,n}) \\ &\quad + (\Delta t)_n (g((S_0)_k - (S_f)_{k,n}) A_{k,n} + q_{k,n} v_{k,n}) \end{aligned} \right. \quad (2.13)
\end{aligned}$$

Commentaires

- Le schéma numérique obtenu est le schéma de Lax-Friedrichs (avec une correction géométrique pour un maillage en espace non uniforme).
- Le schéma numérique obtenu est explicite.

Proposition (complexité algorithmique)

La complexité algorithmique du schéma (2.13) est égale à $O(KN)$.

Proposition (condition de stabilité, admise)

Alors, le schéma (2.13) est stable lorsque :

$$\forall n \in \{1, \dots, N\} : (\Delta t)_n \leq \text{CFL} \cdot \min \left\{ \frac{(\Delta s)_1}{\lambda_1^*}, \min_{k \in \{2, \dots, K\}} \left\{ \frac{\min \{(\Delta s)_k, (\Delta s)_{k-1}\}}{\lambda_k^*} \right\}, \frac{(\Delta s)_K}{\lambda_{K+1}^*} \right\},$$

avec :

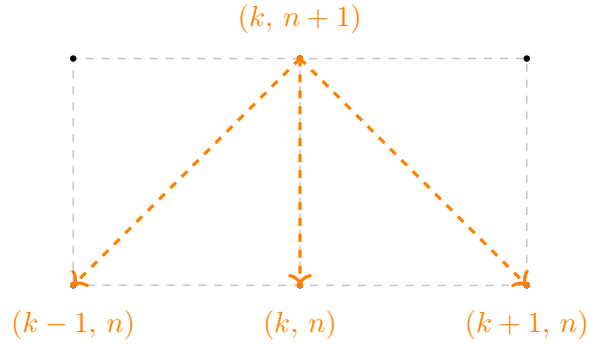
$$\text{CFL} \in]0, 1].$$

Commentaire

Cette condition de stabilité est à l'origine valable pour des schémas aux volumes finis, adaptée pour un schéma aux différences finies. On interprétera cette condition de stabilité lors de la présentation d'un schéma aux volumes finis.

Illustration

Schéma de dépendances du maillage espace-temps :



Résultats (erreurs numériques)

On teste ces implémentations du schéma (2.13), en comparant les résultats numériques avec la solution exacte (2.4) et (2.5) (de Hudson 1999 [6]) comme référence, et on obtient les résultats suivants :

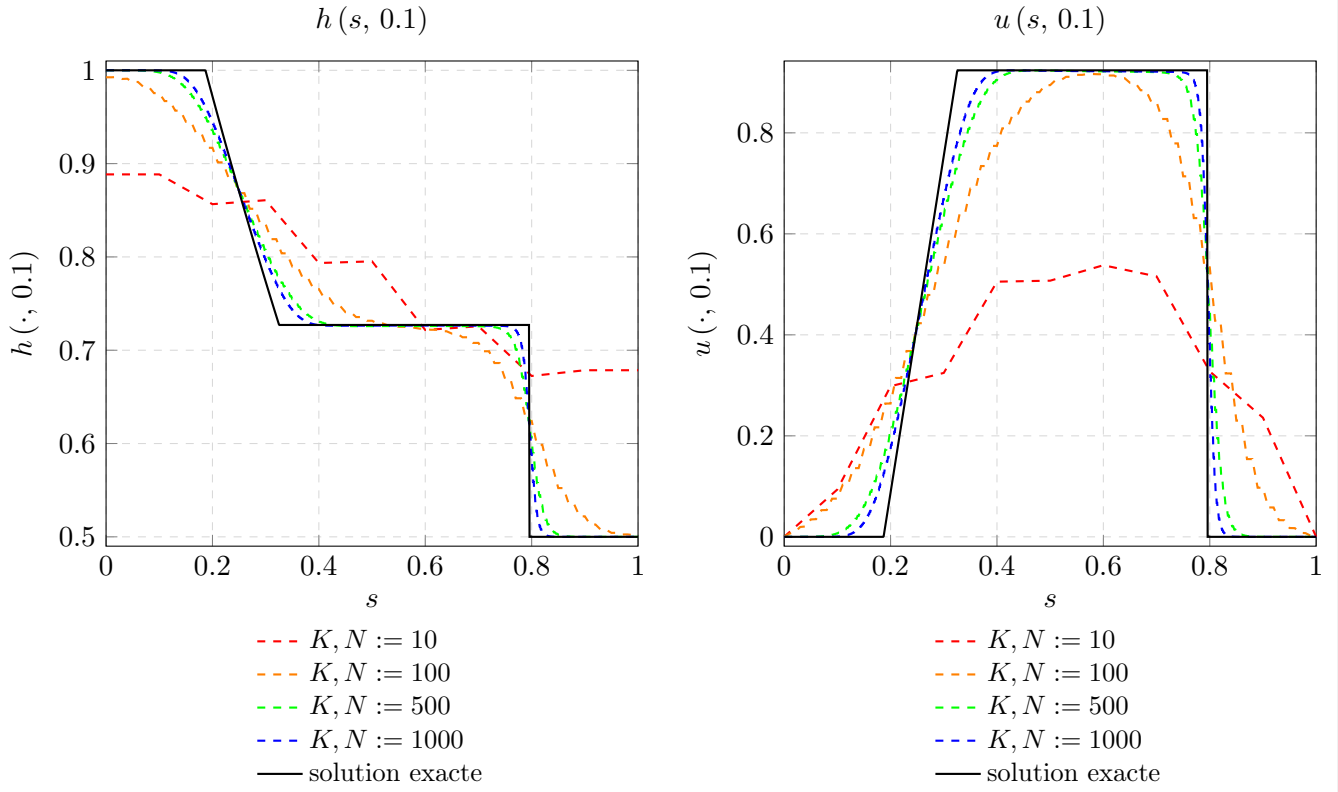
$\|h_{\text{approche}}(\cdot, 0.1) - h_{\text{exact}}(\cdot, 0.1)\|_{L_2}$ en fonction de K et N :

$\downarrow K \quad N \rightarrow$	10	100	500	1000
10	0.117	0.181	0.181	0.181
100	∞_f	0.042	0.075	0.093
500	∞_f	∞_f	0.022	0.030
1000	∞_f	∞_f	0.009	0.015

$\|u_{\text{approche}}(\cdot, 0.1) - u_{\text{exact}}(\cdot, 0.1)\|_{L_2}$ en fonction de K et N :

$\downarrow K \quad N \rightarrow$	10	100	500	1000
10	0.326	0.629	0.632	0.632
100	∞_f	0.161	0.266	0.339
500	∞_f	∞_f	0.083	0.114
1000	∞_f	∞_f	0.037	0.059

et :



Ces résultats ont été obtenus avec un maillage en espace régulier. On obtient des résultats similaires lorsque le maillage en espace n'est pas régulier.

Interprétations

- Le schéma (2.13) semble consistant en espace et en temps.
- Les erreurs sont satisfaisantes.

Résultats (*instabilités numérique*)

On teste ces implémentations du schéma (2.13), en recherchant des instabilités numériques, et on obtient les résultats suivants :

- Il existe des paramètres tels que les résultats numériques divergent (même lorsque $\text{CFL} \leq 1$).
- Le type de divergence est des oscillations en damier selon l'espace-temps qui progressent pour atteindre la valeur ∞_f , principalement observé sur les variables de quantité de mouvement.
- Pour des paramètres tels que les résultats numériques divergent (même lorsque $\text{CFL} \leq 1$), diminuer CFL peut faire converger les résultats numériques, mais il n'y a pas de règle connue sur la valeur de CFL pour garantir la convergence des résultats numériques.

2.1.4.6 Préparation d'un schéma aux volumes finis

$$\begin{aligned}
 & \frac{\partial}{\partial t} (U) + \frac{\partial}{\partial s} (F(U)) = S(U) \\
 \Rightarrow & \int_{C_k} \left(\frac{\partial}{\partial t} (U) + \frac{\partial}{\partial s} (F(U)) \right) ds = \int_{C_k} S(U) ds \\
 \Leftrightarrow & \frac{\partial}{\partial t} \left(\int_{C_k} U ds \right) + \int_{C_k} \frac{\partial}{\partial s} (F(U)) ds = \int_{C_k} S(U) ds,
 \end{aligned}$$

et (d'après le théorème de Green) :

$$\int_{C_k} \frac{\partial}{\partial s} (F(U)) \, ds = [F(U)]_{s=s_{k-1/2}}^{s_{k+1/2}} = F(U_{k+1/2,n}) - F(U_{k-1/2,n}).$$

Donc :

$$\begin{aligned} & \frac{\partial}{\partial t} (U) + \frac{\partial}{\partial s} (F(U)) = S(U) \\ \Leftrightarrow & \frac{\partial}{\partial t} \left(\int_{C_k} U \, ds \right) + F(U_{k+1/2,n}) - F(U_{k-1/2,n}) \simeq \int_{C_k} S(U) \, ds. \end{aligned}$$

D'après l'approximation (2.10), on obtient :

$$\begin{aligned} & \frac{\partial}{\partial t} (U_{k,n}) + \frac{\partial}{\partial s} (F(U_{k,n})) = S(U_{k,n}) \\ \Leftrightarrow & (\Delta_s)_k \frac{\partial}{\partial t} (U_{k,n}) + F(U_{k+1/2,n}) - F(U_{k-1/2,n}) \simeq (\Delta_s)_k S(U_{k,n}) \\ \Leftrightarrow & \boxed{\frac{\partial}{\partial t} (U_{k,n}) + \frac{1}{(\Delta_s)_k} (F(U_{k+1/2,n}) - F(U_{k-1/2,n})) \simeq S(U_{k,n})}. \end{aligned} \quad (2.14)$$

Remarques

— Soit $t \in \Omega_t^*$. Alors, sous forme discrète :

$$V_n = \sum_{k=1}^K (\Delta_s)_k A_{k,n}.$$

— Sous forme discrète, vérifier (2.1) revient à vérifier :

$$\begin{aligned} & \frac{d}{dt} (V) = Q(0, t) - Q(L_s, t) \\ \Leftrightarrow & \frac{d}{dt} (V_n) = Q_{1/2,n} - Q_{K+1/2,n}. \end{aligned}$$

D'après l'approximation (2.12), on obtient :

$$\begin{aligned} & \frac{d}{dt} (V_n) = Q_{1/2,n} - Q_{K+1/2,n} \\ \Leftrightarrow & \frac{1}{(\Delta t)_n} (V_n - V_{n-1}) \simeq Q_{1/2,n} - Q_{K+1/2,n} \\ \Leftrightarrow & V_n - V_{n-1} \simeq (\Delta t)_n (Q_{1/2,n} - Q_{K+1/2,n}) \\ \Leftrightarrow & \boxed{V_n - V_{n-1} - (\Delta t)_n (Q_{1/2,n} - Q_{K+1/2,n}) \simeq 0}. \end{aligned} \quad (2.15)$$

— Lors de l'implémentation, on se servira de cette égalité pour vérifier la conservation de la masse.

2.1.4.7 Schéma aux volumes finis de Rusanov

Soient $\lambda_{k-1/2,n}^* := \max \{ \lambda_{k-1,n}^*, \lambda_{k,n}^* \}$, et $\lambda_{k+1/2,n}^* := \max \{ \lambda_{k,n}^*, \lambda_{k+1,n}^* \}$. Alors, on fait les approximations (de Rusanov, voir [11]) suivantes :

$$F(U_{k+1/2,n}) \simeq \frac{1}{2} \left((F(U_{k,n}) + F(U_{k+1,n})) - \lambda_{k+1/2,n}^* (U_{k+1,n} - U_{k,n}) \right),$$

et

$$F(U_{k-1/2,n}) := \frac{1}{2} \left((F(U_{k-1,n}) + F(U_{k,n})) - \lambda_{k-1/2,n}^* (U_{k,n} - U_{k-1,n}) \right).$$

Donc (d'après l'approximation (2.14)) :

$$\begin{aligned} \frac{\partial}{\partial t}(U_{k,n}) + \frac{\partial}{\partial s}(F(U_{k,n})) &= S(U_{k,n}) \\ &\Leftrightarrow \frac{\partial}{\partial t}(U_{k,n}) \\ + \frac{1}{2} \frac{1}{(\Delta_s)_k} &\left((F(U_{k,n}) + F(U_{k+1,n}) - \lambda_{k+1/2,n}^* (U_{k+1,n} - U_{k,n})) - (F(U_{k-1,n}) + F(U_{k,n}) - \lambda_{k-1/2,n}^* (U_{k,n} - U_{k-1,n})) \right) \\ &\simeq S(U_{k,n}). \end{aligned}$$

D'après l'approximation (2.11), on obtient :

$$\begin{aligned} \frac{\partial}{\partial t}(U_{k,n}) + \frac{\partial}{\partial s}(F(U_{k,n})) &= S(U_{k,n}) \\ &\Leftrightarrow \frac{1}{(\Delta_t)_n} (U_{k,n+1} - U_{k,n}) \\ + \frac{1}{2} \frac{1}{(\Delta_s)_k} &\left((F(U_{k,n}) + F(U_{k+1,n}) - \lambda_{k+1/2,n}^* (U_{k+1,n} - U_{k,n})) - (F(U_{k-1,n}) + F(U_{k,n}) - \lambda_{k-1/2,n}^* (U_{k,n} - U_{k-1,n})) \right) \\ &\simeq S(U_{k,n}). \\ &\Leftrightarrow U_{k,n+1} - U_{k,n} \\ + \frac{1}{2} \frac{(\Delta_t)_n}{(\Delta_s)_k} &\left((F(U_{k,n}) + F(U_{k+1,n}) - \lambda_{k+1/2,n}^* (U_{k+1,n} - U_{k,n})) - (F(U_{k-1,n}) + F(U_{k,n}) - \lambda_{k-1/2,n}^* (U_{k,n} - U_{k-1,n})) \right) \\ &\simeq (\Delta_t)_n S(U_{k,n}). \\ &\Leftrightarrow U_{k,n+1} \\ \simeq -\frac{1}{2} \frac{(\Delta_t)_n}{(\Delta_s)_k} &\left((F(U_{k,n}) + F(U_{k+1,n}) - \lambda_{k+1/2,n}^* (U_{k+1,n} - U_{k,n})) - (F(U_{k-1,n}) + F(U_{k,n}) - \lambda_{k-1/2,n}^* (U_{k,n} - U_{k-1,n})) \right) \\ &\quad + U_{k,n} \\ &\quad + (\Delta_t)_n S(U_{k,n}). \\ &\Leftrightarrow \begin{pmatrix} A_{k,n+1} \\ Q_{k,n+1} \end{pmatrix} \\ \simeq -\frac{1}{2} \frac{(\Delta_t)_n}{(\Delta_s)_k} &\left(\left(\frac{Q_{k,n}^2}{A_{k,n}} + \frac{1}{2} g \frac{A_{k,n}^2}{b_k} \right) + \left(\frac{Q_{k+1,n}^2}{A_{k+1,n}} + \frac{1}{2} g \frac{A_{k+1,n}^2}{b_{k+1}} \right) - \lambda_{k+1/2,n}^* \left(\begin{pmatrix} A_{k+1,n} \\ Q_{k+1,n} \end{pmatrix} - \begin{pmatrix} A_{k,n} \\ Q_{k,n} \end{pmatrix} \right) - \left(\left(\frac{Q_{k-1,n}^2}{A_{k-1,n}} + \frac{1}{2} g \frac{A_{k-1,n}^2}{b_{k-1}} \right) + \left(\frac{Q_{k,n}^2}{A_{k,n}} + \frac{1}{2} g \frac{A_{k,n}^2}{b_k} \right) + \lambda_{k-1/2,n}^* \left(\begin{pmatrix} A_{k,n} \\ Q_{k,n} \end{pmatrix} - \begin{pmatrix} A_{k-1,n} \\ Q_{k-1,n} \end{pmatrix} \right) \right) \\ &\quad + \begin{pmatrix} A_{k,n} \\ Q_{k,n} \end{pmatrix} \\ &\quad + (\Delta_t)_n \begin{pmatrix} q_{k,n} \\ g((S_0)_k - (S_f)_{k,n}) A_{k,n} + q_{k,n} v_{k,n} \end{pmatrix}. \\ &\Leftrightarrow \begin{pmatrix} A_{k,n+1} \\ Q_{k,n+1} \end{pmatrix} \\ \simeq -\frac{1}{2} \frac{(\Delta_t)_n}{(\Delta_s)_k} &\left(\left(\frac{Q_{k+1,n}^2}{A_{k+1,n}} + \frac{1}{2} g \frac{A_{k+1,n}^2}{b_{k+1}} - \left(\frac{Q_{k-1,n}^2}{A_{k-1,n}} + \frac{1}{2} g \frac{A_{k-1,n}^2}{b_{k-1}} \right) \right) - \lambda_{k+1/2,n}^* \left(\begin{pmatrix} A_{k+1,n} \\ Q_{k+1,n} \end{pmatrix} - \begin{pmatrix} A_{k,n} \\ Q_{k,n} \end{pmatrix} \right) + \lambda_{k-1/2,n}^* \left(\begin{pmatrix} A_{k,n} \\ Q_{k,n} \end{pmatrix} - \begin{pmatrix} A_{k-1,n} \\ Q_{k-1,n} \end{pmatrix} \right) \right) \\ &\quad + \begin{pmatrix} A_{k,n} \\ Q_{k,n} \end{pmatrix} \\ &\quad + (\Delta_t)_n \begin{pmatrix} q_{k,n} \\ g((S_0)_k - (S_f)_{k,n}) A_{k,n} + q_{k,n} v_{k,n} \end{pmatrix} \end{aligned}$$

$$\Leftrightarrow \left\{ \begin{array}{l} A_{k,n+1} \\ \simeq -\frac{1}{2} \frac{(\Delta_t)_n}{(\Delta_s)_k} \left(Q_{k+1,n} - Q_{k-1,n} - \lambda_{k+1/2,n}^* (A_{k+1,n} - A_{k,n}) + \lambda_{k-1/2,n}^* (A_{k,n} - A_{k-1,n}) \right) \\ + A_{k,n} \\ + (\Delta_t)_n q_{k,n} \\ Q_{k,n+1} \\ \simeq -\frac{1}{2} \frac{(\Delta_t)_n}{(\Delta_s)_k} \left(\frac{Q_{k+1,n}^2}{A_{k+1,n}} - \frac{Q_{k-1,n}^2}{A_{k-1,n}} + \frac{1}{2} g \left(\frac{A_{k+1,n}^2}{b_{k+1}} - \frac{A_{k-1,n}^2}{b_{k-1}} \right) - \lambda_{k+1/2,n}^* (Q_{k+1,n} - Q_{k,n}) + \lambda_{k-1/2,n}^* (Q_{k,n} - Q_{k-1,n}) \right) \\ + Q_{k,n} \\ + (\Delta_t)_n (g((S_0)_k - (S_f)_{k,n}) A_{k,n} + q_{k,n} v_{k,n}) \end{array} \right. \quad (2.16)$$

Commentaires

- Le schéma numérique obtenu est le schéma de Rusanov.
- On dit que l'approximation de Rusanov est une *méthode de flux*.
- D'après l'état de l'art, il existe l'approximation de Godunov, qui est plus précise que l'approximation de Rusanov, mais nécessite de résoudre un problème de Riemann à chaque interface.

Proposition (complexité algorithmique)

La complexité algorithmique du schéma (2.16) est égale à $O(KN)$.

Proposition (condition de stabilité, admise)

Le schéma (2.16) est stable lorsque :

$$\forall n \in \{1, \dots, N\} : (\Delta_t)_n \leq \text{CFL} \cdot \min_{k \in \{1, \dots, K\}} \left\{ \frac{(\Delta_s)_k}{\lambda_k^*} \right\},$$

avec :

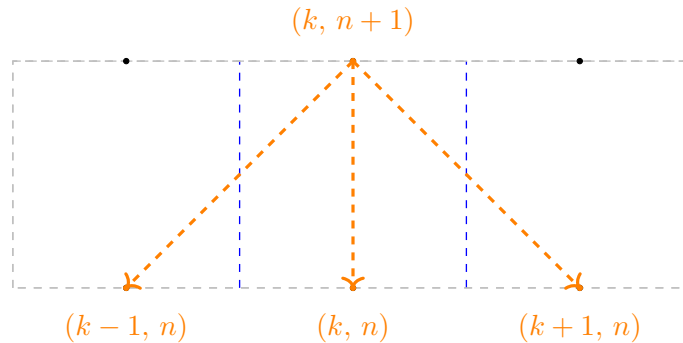
$$\text{CFL} \in]0, 1].$$

Commentaires

- Cette proposition impose que toutes les ondes caractéristiques ne puissent pas sauter de plus d'une cellule en espace par itération, elle se base sur l'égalité (2.3), appliqué à une distance de Δ_s au lieu de L_s .
- Si cette inégalité n'est pas respectée, alors certaines cellules ne recevront aucune onde caractéristique, ce qui est physiquement faux, et peut mener à des instabilités numériques.
- $\forall k \in \{1, \dots, K\}, \forall n \in \{1, \dots, N\} : (\Delta_t)_n$ et $(\Delta_s)_k$ sont liés de manière linéaire.

Illustration

Schéma de dépendances du maillage espace-temps :



Implémentation (résumé, voir [lien cliquable](#))

Les signatures et corps de fonctions qui appliquent le schéma (2.16) sont les suivantes :

```
static inline __attribute__((always_inline))
void calculer_Delta_t_n(
    double CFL,
    int K,
    int n,
    double *Delta_s,
    double *b,
    double *A,
    double *Q,
    double *Delta_t
){
    if (CFL <= 0){
        return;
    }

    double Delta_t_n_local;
    double Delta_t_n;

    Delta_t_n = DBL_MAX;

    for (int k = 1 ; k < K + 1 ; k ++){

        Delta_t_n_local = (
            Delta_s[k] /
            (
                fabs(Q[idx(k, n)] / A[idx(k, n)]) +
                sqrt(g * A[idx(k, n)] / b[k])
            )
        );
        if (Delta_t_n_local < Delta_t_n){
            Delta_t_n = Delta_t_n_local;
        }
    }

    Delta_t[n] = CFL * Delta_t_n;

    return;
}
```

```
static inline __attribute__((always_inline))
void calculer_P_n(
    int K,
    int n,
    double *b,
    double *A,
    double *P_n
){
    for (int k = 0 ; k < K + 2 ; k ++){
        P_n[k] = b[k] + 2 * A[idx(k, n)] / b[k];
    }
}
```

```

    }

    return;
}

```

```

static inline __attribute__((always_inline))
void calculer_R_n(
    int K,
    int n,
    double *P_n,
    double *A,
    double *R_n
){
    for (int k = 0 ; k < K + 2 ; k ++){
        R_n[k] = A[idx(k, n)] / P_n[k];
    }

    return;
}

```

```

static inline __attribute__((always_inline))
void calculer_S_f_n(
    int K,
    int n,
    double *b,
    double *r,
    double *R_n,
    double *A,
    double *Q,
    double *S_f_n
){
    for (int k = 0 ; k < K + 2 ; k ++){
        S_f_n[k] = (
            Q[idx(k, n)] * fabs(Q[idx(k, n)]) * pow(r[k], 2) /
            pow(A[idx(k, n)], 2) * pow(R_n[k], 4.0 / 3.0)
        );
    }

    return;
}

```

```

static inline __attribute__((always_inline))
void calculer_lambda_star_n(
    int K,
    int n,
    double *b,
    double *A,
    double *Q,
    double *lambda_star_n
){

```

```

double lambda_star_gauche;
double lambda_star_droite;

for (int k = 0 ; k < K + 1 ; k ++){

    lambda_star_gauche = (
        fabs(Q[idx(k, n)] / A[idx(k, n)])
        + sqrt(g * A[idx(k, n)] / b[k])
    );
    lambda_star_droite = (
        fabs(Q[idx(k + 1, n)] / A[idx(k + 1, n)])
        + sqrt(g * A[idx(k + 1, n)] / b[k + 1])
    );
    lambda_star_n[k] = max(
        lambda_star_gauche,
        lambda_star_droite
    );

}

return;
}

```

```

static inline __attribute__((always_inline))
void appliquer_schema_A(
    int K,
    double *Delta_s,
    double *Delta_t,
    int n,
    double *lambda_star_n,
    double *q,
    double *A,
    double *Q
){

    for (int k = 1 ; k < K + 1 ; k ++){

        A[idx(k, n + 1)] = (
            -0.5 * (Delta_t[n] / Delta_s[k]) * (
                Q[idx(k + 1, n)] -
                Q[idx(k - 1, n)] -
                lambda_star_n[k] * (A[idx(k + 1, n)] - A[idx(k, n)]) +
                lambda_star_n[k - 1] * (A[idx(k, n)] - A[idx(k - 1, n)])
            ) +
            A[idx(k, n)] +
            Delta_t[n] * q[idx(k, n)]
        );

    }

    return;
}

```

```

static inline __attribute__((always_inline))
void appliquer_schema_Q(
    int K,
    double *Delta_s,
    double *Delta_t,
    int n,
    double *S_f_n,
    double *lambda_star_n,
    double *b,
    double *S_0,
    double *q,
    double *v,
    double *A,
    double *Q
){
    for (int k = 1 ; k < K + 1 ; k ++){
        Q[idx(k, n + 1)] = (
            -0.5 * (Delta_t[n] / Delta_s[k]) * (
                pow(Q[idx(k + 1, n)], 2) / A[idx(k + 1, n)] -
                pow(Q[idx(k - 1, n)], 2) / A[idx(k - 1, n)] +
                0.5 * g * (
                    pow(A[idx(k + 1, n)], 2) / b[k + 1] -
                    pow(A[idx(k - 1, n)], 2) / b[k - 1]
                ) -
                lambda_star_n[k] * (Q[idx(k + 1, n)] - Q[idx(k, n)]) +
                lambda_star_n[k - 1] * (Q[idx(k, n)] - Q[idx(k - 1, n)])
            ) +
            Q[idx(k, n)] +
            Delta_t[n] * (
                g *
                (S_0[k] - S_f_n[k]) *
                A[idx(k, n)] + q[idx(k, n)] * v[idx(k, n)]
            )
        );
    }
    return;
}

```

```

static inline __attribute__((always_inline))
void appliquer_condition_limite(
    int type_condition_limite_A,
    int type_condition_limite_Q,
    int K,
    int n,
    double *A,
    double *Q
);

```

```

void resoudre_volume_fini_rusanov(
    int type_condition_limite_A,

```



```

int type_condition_limite_Q,
double CFL,
int K,
double *Delta_s,
int N,
double L_t,
double *Delta_t,
double *t,
double *b,
double *r,
double *S_0,
double *q,
double *v,
double *A,
double *Q,
double *t_final_ptr
);

```

Remarques

- On utilise la macro `idx` pour accéder aux tableaux 2D linéarisés :

```
# define idx(k, n) ((n) * (K + 2) + (k))
```

- On a 3 versions de l'implémentation des fonctions qui appliquent le schéma (2.16) :
 - Séquentielle : [lien cliquable](#)
 - Parallèle avec OpenMP : [lien cliquable](#)
 - Parallèle avec MPI : [lien cliquable](#)
- Chaque version donne des résultats numériques identiques (à non-associativité des nombres flottants près).

Implémentation (résumé, voir [lien cliquable](#))

Des fonctions pour gérer la parallélisation avec MPI sont les suivantes :

```

void creer_topologie(){

    int tore = 0;
    dimension_communicateur_1d = 0;

    MPI_Dims_create(
        nombre_processus,
        1,
        &dimension_communicateur_1d
    );
    MPI_Cart_create(
        MPI_COMM_WORLD,
        1,
        &dimension_communicateur_1d,
        &tore,
        0,
        &communicateur_1d
    );

    MPI_Barrier(communicateur_1d);
}

```

```

    return;
}

```

```

void calculer_information_topologie(){

    MPI_Cart_coords(
        communicateur_1d,
        numero_processus,
        1,
        &numero_processus_communicateur_1d
    );
    MPI_Cart_shift(
        communicateur_1d,
        0,
        1,
        &(voisin[0]),
        &(voisin[1])
    );

    nombre_bord_libre = 2;
    for (int i = 0 ; i < 2 ; i ++){
        if (voisin[i] == -1){
            nombre_bord_libre --;
        }
    }

    MPI_Barrier(communicateur_1d);

    return;
}

```

```

void calculer_information_processus(
    int K,
    int *K_divise_ptr,
    int *k_debut_ptr,
    int *k_fin_ptr
){

    int K_divise;
    int k_debut;
    int k_fin;

    k_debut = (
        (numero_processus_communicateur_1d * K) /
        dimension_communicateur_1d
    );
    k_fin = (
        ((numero_processus_communicateur_1d + 1) * K) /
        dimension_communicateur_1d - 1
    );
    K_divise = k_fin - k_debut + 1;
}

```

```

    *K_divise_ptr = K_divise;
    *k_debut_ptr = k_debut;
    *k_fin_ptr = k_fin;

    MPI_Barrier(communiquateur_1d);

    return;
}

```

```

void calculer_information_borne_boucle_espace(
    int K_divise,
    int k_debut,
    int k_fin,
    int *k_boucle_debut_ptr,
    int *k_boucle_fin_ptr
){
    int k_boucle_debut;
    int k_boucle_fin;

    k_boucle_debut = 1;
    if (voisin[0] == -1){
        k_boucle_debut ++;
    }
    k_boucle_fin = K_divise + 1;
    if (voisin[1] == -1){
        k_boucle_fin --;
    }

    *k_boucle_debut_ptr = k_boucle_debut;
    *k_boucle_fin_ptr = k_boucle_fin;

    return;
}

```

```

void calculer_information_regroupement_resultat(
    int K,
    int **deplacements_ptr,
    int **nombre_elements_recus_ptr
);

```

```

void initialiser_tableau_divise_1d(
    int K,
    int K_divise,
    int k_debut,
    double *U,
    double *U_divise
);

```

```

void initialiser_tableau_divise_2d(
    int K,
    int K_divise,

```

```
int k_debut ,
double *U_initial ,
double *U_divise
);
```

```
void communiquer_cellule_fantome(
int K_divise ,
double *U_divise
){

// Envoi gauche , reception droite
MPI_Sendrecv(
    &(U_divise[1]),
    1,
    MPI_DOUBLE ,
    voisin[0] ,
    etiquette ,
    &(U_divise[K_divise + 1]),
    1,
    MPI_DOUBLE ,
    voisin[1] ,
    etiquette ,
    communicateur_1d ,
    &statut
);

// Envoi droite , reception gauche
MPI_Sendrecv(
    &(U_divise[K_divise]),
    1,
    MPI_DOUBLE ,
    voisin[1] ,
    etiquette ,
    &(U_divise[0]),
    1,
    MPI_DOUBLE ,
    voisin[0] ,
    etiquette ,
    communicateur_1d ,
    &statut
);

return;
}
```

```
void communiquer_cellule_fantome_double(
int K_divise ,
double *U_divise ,
double *V_divise
);
```

```
void regrouper_resultats(
int K ,
int N ,
```

```

int K_divise ,
int *deplacements ,
int *nombre_elements_recus ,
double *A_initial ,
double *Q_initial ,
double *A_divise ,
double *Q_divise ,
double **A_ptr ,
double **Q_ptr
);

```

Remarque

Pour implémenter les fonctions qui gèrent la parallélisation avec MPI et l'adaptation pour OpenMP et MPI des fonctions qui appliquent le schéma (2.16), on utilise la même méthode que dans le sujet *Analyse numérique d'équations aux dérivées partielles par différences finies et implémentation optimisée pour le calcul haute performance*.

Résultats (erreurs numériques)

On teste ces implémentations du schéma (2.16), en comparant les résultats numériques avec la solution exacte (2.4) et (2.5) (de Hudson 1999 [6]) comme référence, et on obtient les résultats suivants :

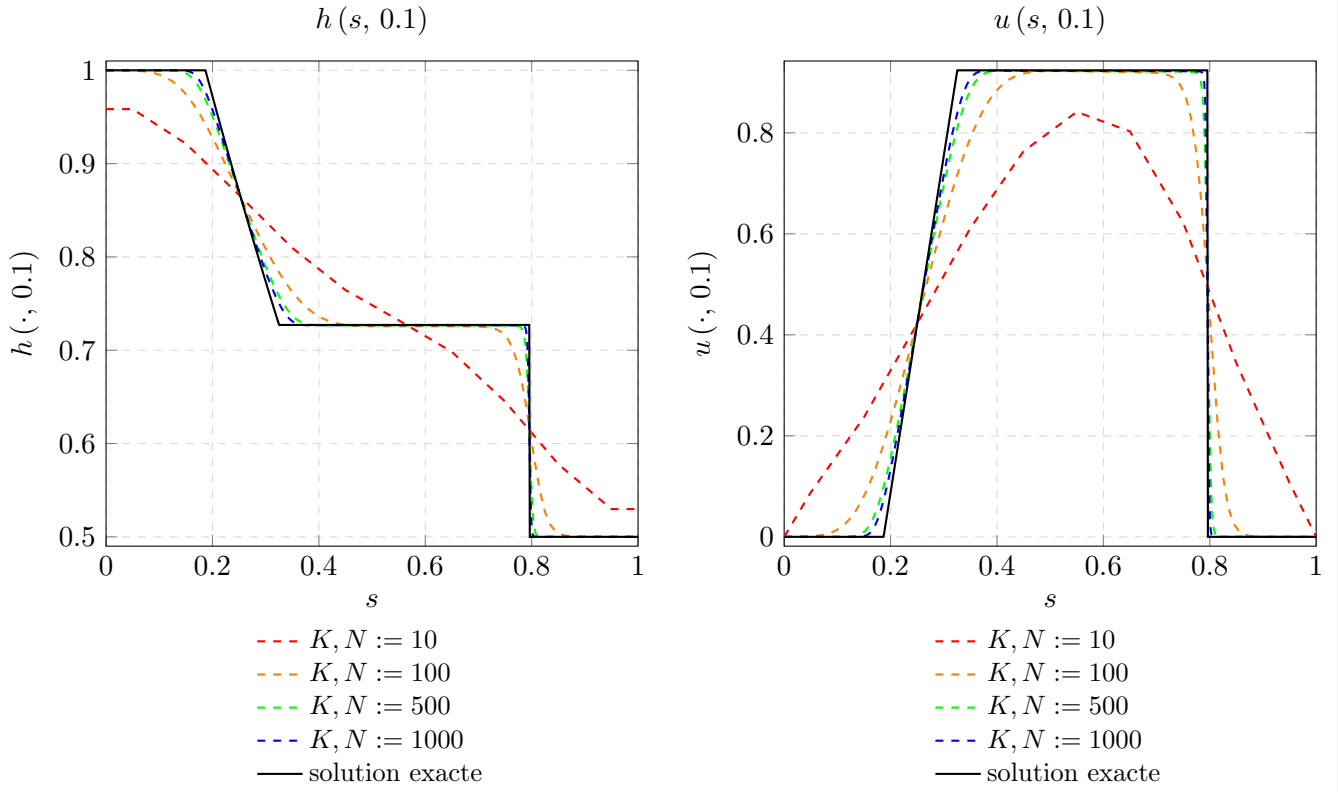
$\|h_{\text{approche}}(\cdot, 0.1) - h_{\text{exact}}(\cdot, 0.1)\|_{L_2}$ en fonction de K et N :

$\downarrow K \quad N \rightarrow$	10	100	500	1000
10	0.055	0.061	0.061	0.061
100	∞_f	0.024	0.027	0.027
500	∞_f	∞_f	0.011	0.012
1000	∞_f	∞_f	0.006	0.007

$\|u_{\text{approche}}(\cdot, 0.1) - u_{\text{exact}}(\cdot, 0.1)\|_{L_2}$ en fonction de K et N :

$\downarrow K \quad N \rightarrow$	10	100	500	1000
10	0.207	0.222	0.223	0.223
100	∞_f	0.091	0.100	0.101
500	∞_f	∞_f	0.041	0.044
1000	∞_f	∞_f	0.023	0.028

et :



Ces résultats ont été obtenus avec un maillage en espace régulier. On obtient des résultats similaires lorsque le maillage en espace n'est pas régulier.

Interprétations

- Le schéma (2.16) semble consistant en espace et en temps.
- Les erreurs sont satisfaisantes.

Résultats (*instabilités numérique*)

On teste ces implémentations du schéma (2.16), en recherchant des instabilités numériques, et on obtient les résultats suivants :

- Il existe des paramètres tels que les résultats numériques divergent (même lorsque $\text{CFL} \leq 1$).
- Le type de divergence est des oscillations en damier selon l'espace-temps qui progressent pour atteindre la valeur ∞_f , principalement observé sur les variables de quantité de mouvement.
- Pour des paramètres tels que les résultats numériques divergent (même lorsque $\text{CFL} \leq 1$), diminuer CFL peut faire converger les résultats numériques, mais il n'y a pas de règle connue sur la valeur de CFL pour garantir la convergence des résultats numériques.

Notations

- On note $\mathcal{T}$ le temps d'exécution.
- On note n le nombre de threads ou de processus.

Définitions

- On appelle le *facteur d'accélération* l'expression :

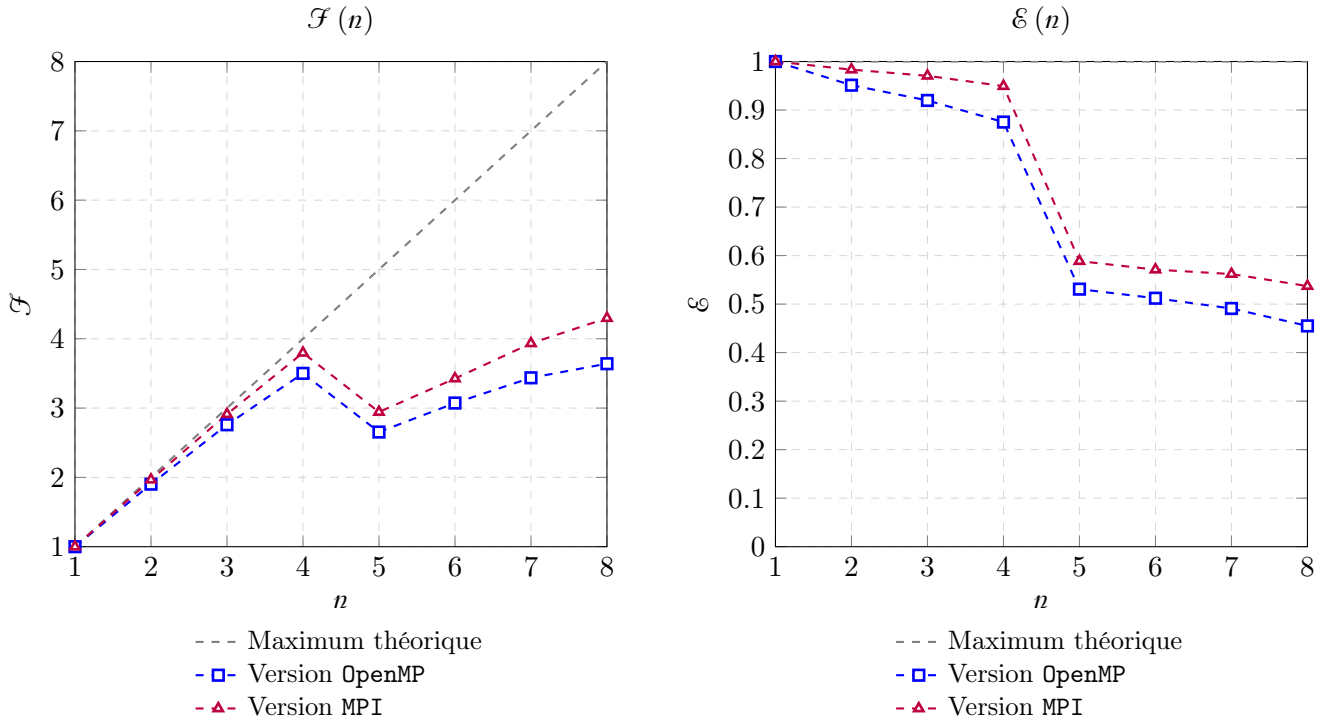
$$\mathcal{F}(n) := \frac{\mathcal{T}(1)}{\mathcal{T}(n)}.$$

— On appelle l'*efficacité de parallélisation* l'expression :

$$\varepsilon(n) := \frac{\mathcal{F}(n)}{n}.$$

Résultats (parallélisation)

On teste ces implémentations du schéma (2.16) (pour les paramètres suivants : $K, N := 15000$), en mesurant et comparant les facteurs d'accélération et efficacité de parallélisation entre les versions, et on obtient les résultats suivants (moyennés sur 10 exécutions) :



Interprétations

- Pour les versions `OpenMP` et `MPI`, l'efficacité de parallélisation est excellente lorsque $n \in \{2, 3, 4\}$ (entre 88% et 98%).
- Pour la version `OpenMP`, si $n = 4$, alors l'efficacité de parallélisation est égale à 88%.
- Pour la version `MPI`, si $n = 4$, alors l'efficacité de parallélisation est égale à 95%.
- Pour les versions `OpenMP` et `MPI`, l'efficacité de parallélisation est strictement décroissante.
- Pour les versions `OpenMP` et `MPI`, l'efficacité de parallélisation chute entre $n = 4$ et $n = 5$ (perte de 34% à 36%). On peut expliquer cela par l'architecture de la machine qui a 4 cœurs physiques et 8 threads logiques (avec 2 threads par cœur).
- L'efficacité de parallélisation est la meilleure pour la version `MPI` (3% à 8% de plus par rapport à la version `OpenMP`).
- Les résultats sont prometteurs en vue d'une parallélisation sur un cluster de calcul.

Définition

On appelle le *coefficient du modèle de temps d'exécution* l'expression : $\mathcal{W} := \mathcal{W}(K, N) := \frac{\mathcal{F}(K, N)}{KN}$.

Remarque

Le coefficient du modèle de temps d'exécution dépend de l'architecture de la machine.

Résultats (*temps d'exécution*)

On teste l'implémentation du schéma (2.16) en version séquentielle, en mesurant les temps d'exécution, et on obtient les résultats suivants (moyennés sur 10 exécutions) :

Temps d'exécution (en s) en fonction de K et N :

$\downarrow K \quad N \rightarrow$	3000	6000	9000	12000	15000
3000	0.32	0.65	0.98	1.29	1.61
6000	0.64	1.29	1.93	2.57	3.21
9000	0.97	1.95	2.91	3.87	4.86
12000	1.30	2.60	3.88	5.16	6.46
15000	1.63	3.25	4.87	6.55	8.08

Coefficient du modèle de temps d'exécution en fonction de K et N :

$\downarrow K \quad N \rightarrow$	3000	6000	9000	12000	15000
3000	3.56×10^{-8}	3.61×10^{-8}	3.63×10^{-8}	3.58×10^{-8}	3.58×10^{-8}
6000	3.56×10^{-8}	3.58×10^{-8}	3.57×10^{-8}	3.57×10^{-8}	3.57×10^{-8}
9000	3.59×10^{-8}	3.61×10^{-8}	3.59×10^{-8}	3.58×10^{-8}	3.60×10^{-8}
12000	3.61×10^{-8}	3.61×10^{-8}	3.59×10^{-8}	3.58×10^{-8}	3.59×10^{-8}
15000	3.62×10^{-8}	3.61×10^{-8}	3.61×10^{-8}	3.64×10^{-8}	3.59×10^{-8}

Interprétations

- La matrice des temps d'exécution est pratiquement symétrique.
- La complexité algorithmique est vérifiée.
- Pour l'architecture de la machine, on peut établir la loi suivante :

$$\mathcal{T}(K, N) \simeq 3.59 \cdot 10^{-8} \cdot KN.$$

Définition

On appelle l'*intensité arithmétique* le rapport entre le nombre d'opérations arithmétiques flottantes et le nombre d'octets d'accès mémoire (hors variables locales).

Remarques

- Dans les implémentations, le nombre d'octets d'un accès mémoire est égal à 8.
- Moins l'intensité arithmétique est élevée, plus le programme est limité par la mémoire. Plus l'intensité arithmétique est élevée, plus le programme est limité par les unités arithmétiques et logiques du C. P. U..

Résultats (*opérations arithmétiques flottantes, accès mémoires, et temps d'exécution des fonctions*)

On teste l'implémentation du schéma (2.16) en version séquentielle, en mesurant les opérations arithmétiques flottantes, les accès mémoires et le temps d'exécution des fonctions (avec le paquet **gprof**), et on obtient les résultats suivants (moyennés sur 5 exécutions) :

Propriété → ↓ Fonction	Nombre d'opérations arithmétiques flottantes									Nombre d'accès mémoire	Intensité arithmétique	Temps d'exécution (en %)
	+	-	*	/	fabs	sqrt	pow	<, >, ==, ...	Total			
calculer _Delta_t_n	1		1	3	1	1		1	8	5	0.2	9.2
calculer _P_n	1		1	1					3	4	0.1	5.3
calculer _R_n				1					1	3	< 0.1	3.8
calculer _S_f_n			4	2	1		1		8	6	0.2	3.3
calculer _lambda_star_n	2		2	4	2	2		1	13	9	0.2	13.3
appliquer _schema_A	3	4	5	1					13	14	0.1	28.1
appliquer _schema_Q	5	6	14	5					30	24	0.2	37.0
appliquer _condition_limite								5	5	8	0.1	0.0
autres	1								1	2	0.1	0.0
itération complète	13	10	27	17	4	3	1	7	82	75	0.1	100.0

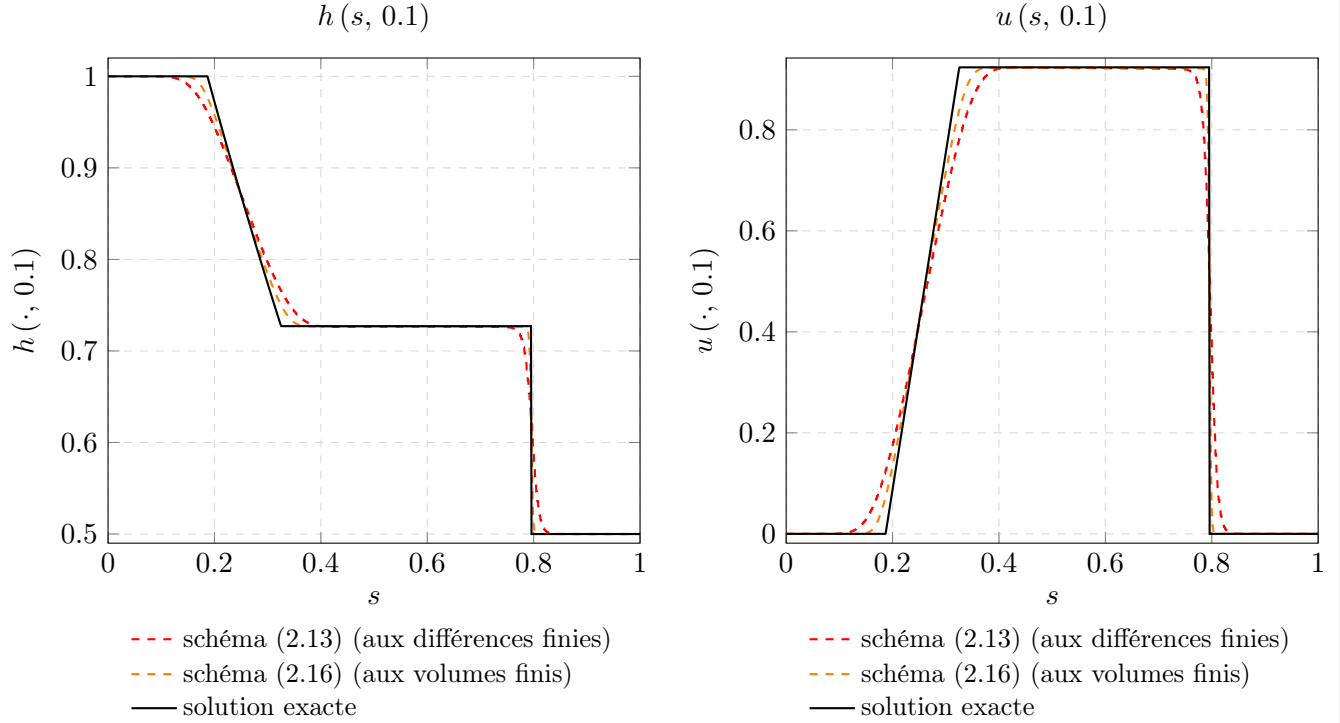
Interprétations

- Les fonctions qui ont le temps d'exécution le plus élevé sont **appliquer_schema_Q** (37.0%), et **appliquer_schema_A** (28.1%). Ces 2 fonctions représentent 65.1% du temps d'exécution.
- Les fonctions qui ont le nombre d'opérations arithmétiques flottantes le plus élevé sont **appliquer_schema_Q** (30), **appliquer_schema_A** (13), et **calculer_lambda_star_n** (13).
- Les fonctions qui ont le nombre d'accès mémoire le plus élevé sont **appliquer_schema_Q** (24), et **appliquer_schema_A** (14).
- On estime que les fonctions **appliquer_schema_Q**, et **appliquer_schema_A** représentent une grande partie du temps d'exécution.
- On estime que les fonctions **calculer_lambda_star_n**, et **calculer_Delta_t_n** représentent une partie modérée du temps d'exécution.
- On estime que les fonctions **calculer_P_n**, **calculer_R_n**, et **calculer_S_f_n** représentent une petite partie du temps d'exécution.
- On estime que la fonction **appliquer_condition_limite** représente une partie négligeable du temps d'exécution.
- L'intensité arithmétique est très faible. Le temps des accès mémoire représente probablement une grande partie du temps d'exécution.

2.1.4.8 Comparaison des schémas numériques

Résultats (erreurs numériques)

On compare les résultats numériques des schémas (2.13) et (2.16) avec la solution exacte (2.4) et (2.5) (de Hudson 1999 [6], avec les paramètres suivants : $K, N := 1000$). on obtient les résultats suivants :



Soient E_h le rapport de l'erreur sur la fonction h entre le schéma (2.13) et (2.16), et E_u le rapport de l'erreur sur la fonction u entre le schéma (2.13) et (2.16). Alors, on compare les résultats numériques des schémas (2.13) et (2.16) avec la solution exacte (2.4) et (2.5) (de Hudson 1999 [6]). On obtient les résultats suivants :

E_h en fonction de K et N :

$\downarrow K \quad N \rightarrow$	10	100	500	1000
10	2.127	2.967	2.967	2.967
100	/	1.750	2.778	3.444
500	/	/	2.000	2.500
1000	/	/	1.500	2.143

E_u en fonction de K et N :

$\downarrow K \quad N \rightarrow$	10	100	500	1000
10	1.575	2.833	2.834	2.834
100	/	1.769	2.660	3.356
500	/	/	2.024	2.591
1000	/	/	1.609	2.107

Interprétation

Le schéma (2.16) (aux volumes finis) permet d'obtenir une erreur environ 1.5 à 3.5 fois inférieure que le schéma (2.13) (aux différences finies).

Remarques

- D'après l'état de l'art, les schémas aux volumes finis sont réputés pour davantage conserver les flux, et mieux capturer les ondes de chocs, ce que vérifie cet exemple.
- Pour l'application du modèle en conditions réelles, on choisira le schéma (2.16) (aux volumes finis).

2.2 Problème stationnaire, et conditions initiales

2.2.1 Régime permanent

Définition (physique)

Soit $t \in \Omega_t^*$. Alors, on dit que la rivière suit un *écoulement en régime permanent* lorsque :

$$\forall s \in \Omega_s : \left(\frac{\partial}{\partial t} (A(s, t)) = 0 \quad \text{et} \quad \exists Q_C \in \mathbb{R} : Q_C = Q(s) \right).$$

Remarque

Soit $t \in \Omega_t^*$. Alors, si la rivière suit un écoulement en régime permanent et $\forall s \in \Omega_s, \forall t \in [t, t + \Delta_t] : q(s, t) = 0$, alors :

$$\forall s \in \Omega_s, \forall t + \delta_t \in [t, t + \Delta_t] : (A(s, t + \delta_t) = A(s, t) \quad \text{et} \quad Q(s, t + \delta_t) = Q_C).$$

2.2.2 Présentation du problème

Si $\frac{\partial}{\partial t} (A) := \frac{\partial}{\partial t} (Q) := 0$, $\frac{\partial}{\partial s} := \frac{d}{ds}$, $A := A(s)$, et $q \equiv v \equiv 0$ alors :

$$\begin{aligned} \frac{\partial}{\partial t} (A) + \frac{\partial}{\partial s} (Q) &= q \\ \Rightarrow \frac{d}{ds} (Q) &= 0 \\ \Leftrightarrow Q &=: Q_C \in \mathbb{R}, \end{aligned}$$

et :

$$\begin{aligned} \frac{\partial}{\partial t} (Q) + \frac{\partial}{\partial s} \left(\frac{Q^2}{A} + \frac{1}{2} g \frac{A^2}{b} \right) &= g (S_0 - S_f) A + qv \\ \Rightarrow \frac{d}{ds} \left(Q_C^2 \frac{1}{A} + \frac{1}{2} g \frac{A^2}{b} \right) - g (S_0 - S_f) A &= 0 \\ \Leftrightarrow Q_C^2 \frac{d}{ds} \left(\frac{1}{A} \right) + \frac{1}{2} g \frac{d}{ds} \left(\frac{A^2}{b} \right) - g (S_0 - S_f) A &= 0 \\ \Leftrightarrow Q_C^2 \frac{d}{ds} \left(\frac{1}{A} \right) + g \left(\frac{1}{2} \frac{d}{ds} \left(\frac{A^2}{b} \right) - (S_0 - S_f) A \right) &= 0 \\ \Leftrightarrow -\frac{Q_C^2}{A^2} \frac{d}{ds} (A) + g \left(\frac{1}{2} \frac{1}{b^2} \left(2bA \frac{d}{ds} (A) - A^2 \frac{d}{ds} (b) \right) - (S_0 - S_f) A \right) &= 0 \\ \Leftrightarrow \left(-\frac{Q_C^2}{A^2} + g \frac{A}{b} \right) \frac{d}{ds} (A) + g \left(-\frac{1}{2} \frac{A}{b^2} \frac{d}{ds} (b) - S_0 + S_f \right) A &= 0 \\ \Leftrightarrow \frac{d}{ds} (A) + g \left(-\frac{1}{2} \frac{A}{b^2} \frac{d}{ds} (b) - S_0 + S_f \right) A \left(-\frac{Q_C^2}{A^2} + g \frac{A}{b} \right)^{-1} &= 0 \\ \Leftrightarrow \frac{d}{ds} (A) = g \left(\frac{1}{2} \frac{A}{b^2} \frac{d}{ds} (b) + S_0 - S_f \right) A \left(-\frac{Q_C^2}{A^2} + g \frac{A}{b} \right)^{-1} \\ \Leftrightarrow A' = g \left(\frac{1}{2} \frac{A}{b^2} b' + S_0 - S_f \right) A \left(-\frac{Q_C^2}{A^2} + g \frac{A}{b} \right)^{-1} \end{aligned}$$

$$\Leftrightarrow A' = \varphi(A, s),$$

avec :

$$A' := \frac{d}{ds}(A) \quad b' := \frac{d}{ds}(b), \quad \varphi(A, s) := g \left(\frac{1}{2} \frac{A}{b^2} b' + S_0 - S_f \right) A \left(-\frac{Q_C^2}{A^2} + g \frac{A}{b} \right)^{-1},$$

$$\text{et } S_f(A) := \frac{r^2 Q_C |Q_C|}{A^2 R^{4/3}}.$$

Soit le problème $(P_{SV1r}^{\text{initial}})$ suivant :

Trouver A telle que :

$$\left\{ \begin{array}{l} A' = \varphi(A, s) \quad \forall s \in \Omega_s \\ Q_C \in \mathbb{R} \\ \text{conditions aux limites} \end{array} \right. . \quad (P_{SV1r}^{\text{initial}})$$

Commentaires

- Le but de résoudre l'équation en régime permanent avec les hypothèses posées permet de trouver des conditions initiales qui représentent une situation physique réaliste.
- En pratique, on utilise la solution du problème $(P_{SV1r}^{\text{initial}})$ comme condition initiale pour résoudre le problème (P_{SV1r}) .
- Le problème $(P_{SV1r}^{\text{initial}})$ est une équation différentielle ordinaire d'ordre 1 non linéaire, et est un problème de Cauchy.
- En régime permanent, A ne dépend pas du temps. Pour la suite, lorsqu'on sera en régime permanent, A deviendra une fonction à 1 variable.

2.2.3 Solutions exactes

Introduction

L'objectif de cette section est de présenter des exemples de solutions exactes existantes dans l'état de l'art.

D'après *Delestre et al. 2012 [3]*, il existe une solution exacte avec les paramètres suivants : $\Omega_s :=]0, 25[$,

$$b \equiv 1, r \equiv 0, z(s) := \begin{cases} \frac{1}{5} - \frac{1}{20}(s-10)^2 & \text{si } 8 \leq s \leq 12 \\ 0 & \text{sinon} \end{cases}, \text{ et } Q_C := 4.42, \text{ et la condition aux limites}$$

suivante : $A(L_s) = h(L_s) := 2.0$. Alors, la solution exacte est telle que :

$$h(s)^3 + \left(z(s) - \frac{1}{2g} \frac{Q_C^2}{h(L_s)^2} - h(L_s) \right) h(s)^2 + \frac{1}{2g} Q_C^2 = 0. \quad (2.17)$$

Commentaires

- La solution exacte reste implicite et nécessite la résolution d'équations polynomiales de degré 3.
- On utilisera la méthode de décomposition "QR" pour approcher cette solution exacte, et on considérera que la solution obtenue est exacte.

D'après *Delestre et al. 2012 [3]*, il existe une solution exacte avec les paramètres suivants : $\Omega_s :=]0, 200[$, $b(s) := 10 - 5e^{-10((1/200)s-1/2)^2}$, $r \equiv 0.03$,

$$S_0(s) := 1 - \frac{Q_C^2}{gh(s)^3 b(s)^2} h'(s) + Q_C^2 r^2 \frac{(b(s) + 2h(s))^{4/3}}{h(s)^{10/3} b(s)^{10/3}} - \frac{Q_C^2 b'(s)}{gh(s)^2 b(s)^3},$$

et $Q_C := 20$, et la condition aux limites suivante : $A(L_s) = b(L_s) h(L_s) := 9.589575 \cdot 0.902921 = 8.658629$, avec $h(L_s) := 0.902921$. Alors, la solution exacte est :

$$h(s) = 0.9 + 0.3e^{-20((1/200)s-1/2)^2}. \quad (2.18)$$

2.2.4 Solutions approchées

Introduction

- En général, pour des paramètres quelconques, il n'existe de solution exacte connue de $(P_{SV1r}^{\text{initial}})$. Pour utiliser le modèle, il faut calculer une solution approchée avec une méthode d'analyse numérique.
- L'objectif de cette section est de présenter des solutions approchées.

Soit le schéma numérique suivant :

$$A_{k+1} \simeq A_k + \frac{1}{6} (\Delta_s)_k (C_1 + 2C_2 + 2C_3 + C_4), \quad (2.19)$$

avec :

$$\begin{aligned} C_1 &:= \varphi(A_k, s_k), \quad C_2 := \varphi\left(A_k + \frac{1}{2} (\Delta_s)_k C_1, s_k + \frac{1}{2} (\Delta_s)_k\right), \\ C_3 &:= \varphi\left(A_k + \frac{1}{2} (\Delta_s)_k C_2, s_k + \frac{1}{2} (\Delta_s)_k\right), \quad \text{et} \quad C_4 := \varphi(A_k + (\Delta_s)_k C_3, s_k + (\Delta_s)_k). \end{aligned}$$

Commentaire

Le schéma numérique défini est le schéma de Runge-Kutta 4, et est explicite.

Proposition (complexité algorithmique)

La complexité algorithmique du schéma (2.19) est égale à $O(K)$.

Implémentation

Soient $\varepsilon > 0$ suffisamment petit, $\mathcal{E} \in \mathbb{N}^*$ suffisamment grand, $Q_C \in \mathbb{R}$ constant, et $A_{\text{amont}} \in \mathbb{R}$ la condition limite à l'amont. Alors, un algorithme qui applique le schéma (2.19) est le suivant :

function RESOUDRE_RUNGE_KUTTA_4($K, \Delta_s, A_{\text{amont}}, Q_C, b, b', r, S_0$)

$A_0 \leftarrow A_{\text{amont}}$

for $k \leftarrow 0$ **to** $K - 1$ **do**

$(\Delta_s)_k \leftarrow (\Delta_s)_k$

$A_k \leftarrow A_k$

$$((S_f)_k)_1 \leftarrow \frac{r_k^2 Q_C |Q_C| \left(b_k + 2 \frac{A_k}{b_k}\right)^{4/3}}{A_k^{10/3}}$$

$$(C_k)_1 \leftarrow \varphi(A_k, Q_C, b_k, b'_k, (S_0)_k, ((S_f)_k)_1)$$

$$(A_k)_2 \leftarrow A_k + \frac{1}{2} (\Delta_s)_k (C_k)_1$$

$$((S_f)_k)_2 \leftarrow \frac{r_k^2 Q_C |Q_C| \left(b_k + 2 \frac{(A_k)_2}{b_k}\right)^{4/3}}{(A_k^{10/3})_2}$$

$$(C_k)_2 \leftarrow \varphi((A_k)_2, Q_C, b_k, b'_k, (S_0)_k, ((S_f)_k)_2)$$

$$(A_k)_3 \leftarrow A_k + \frac{1}{2} (\Delta_s)_k (C_k)_2$$

```


$$((S_f)_k)_3 \leftarrow \frac{r_k^2 Q_C |Q_C| \left(b_k + 2 \frac{(A_k)_3}{b_k}\right)^{4/3}}{\left(A_k^{10/3}\right)_3}$$


$$(C_k)_3 \leftarrow \varphi((A_k)_3, Q_C, b_k, b'_k, (S_0)_k, ((S_f)_k)_3)$$


$$(A_k)_4 \leftarrow A_k + (\Delta_s)_k (C_k)_3$$


$$((S_f)_k)_4 \leftarrow \frac{r_k^2 Q_C |Q_C| \left(b_k + 2 \frac{(A_k)_4}{b_k}\right)^{4/3}}{\left(A_k^{10/3}\right)_4}$$


$$(C_k)_4 \leftarrow \varphi((A_k)_4, Q_C, b_k, b'_k, (S_0)_k, ((S_f)_k)_4)$$


$$A_{k+1} \leftarrow A_k + \frac{(\Delta_s)_k}{6} ((C_k)_1 + 2(C_k)_2 + 2(C_k)_3 + (C_k)_4)_k$$

end for
return  $A$ 
end function

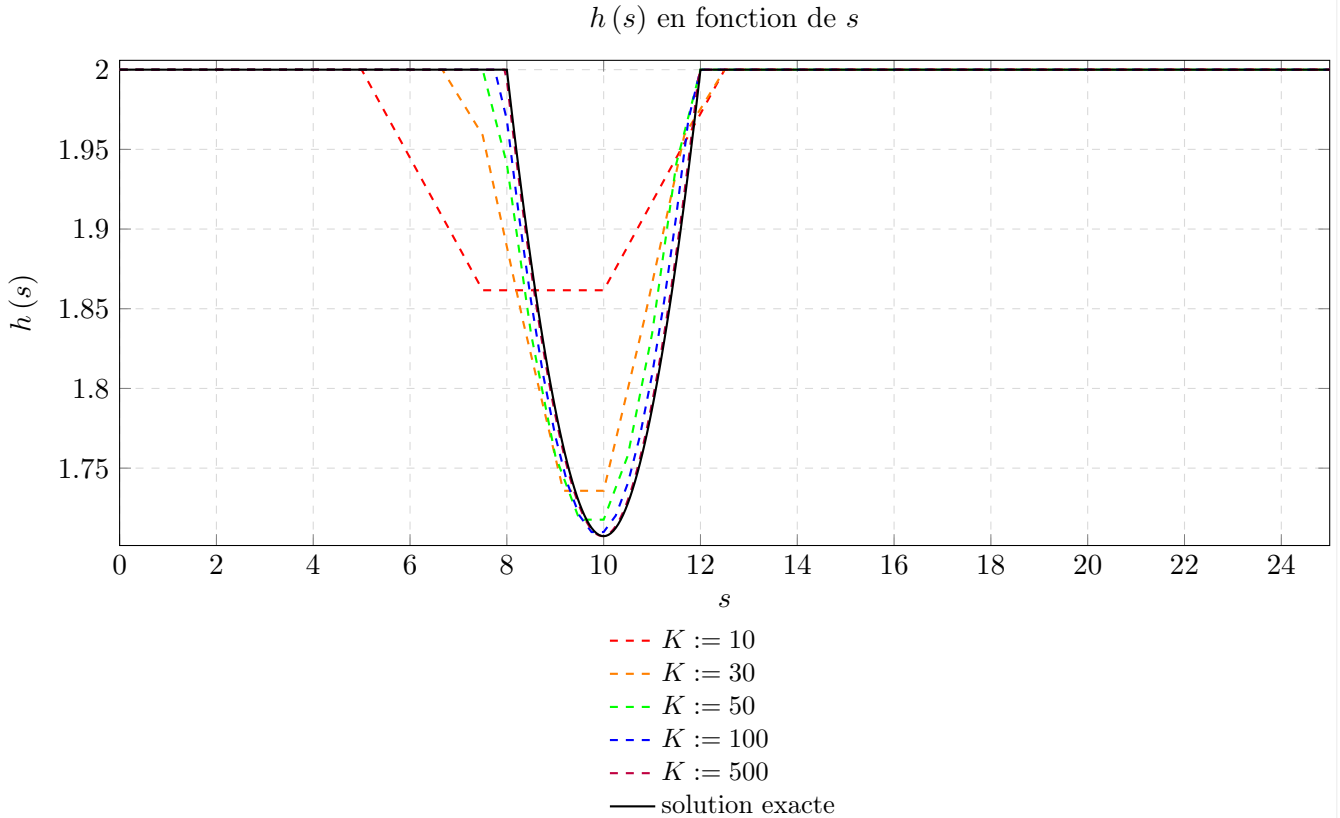
```

Résultats (erreurs numériques)

On teste cette implémentation du schéma (2.19), en comparant les résultats numériques avec la solution exacte (2.17), et on obtient les résultats suivants :

K	5	10	20	30	40	50	100	500
$\ h_{\text{approche}} - h_{\text{exact}}\ _{\infty}$	0.154	0.154	0.087	0.079	0.071	0.601	0.032	0.007

et :



On teste cette implémentation du schéma (2.19), en comparant les résultats numériques avec la solution exacte (2.18), et les résultats numériques divergents.

Interprétations

- Pour la solution exacte (2.17), le schéma (2.19) semble consistant en espace.
- Pour la solution exacte (2.17), les erreurs sont satisfaisantes.

Résultats (*instabilités numérique*)

On teste ces implémentations du schéma (2.19), en recherchant des instabilités numériques, et on obtient les résultats suivants :

- En général, les résultats numériques divergent. On peut expliquer cela par le fait que le problème $(P_{SV1r}^{\text{initial}})$ est dit *raide*.
- Pour tenter de supprimer les instabilité numériques, on a testé une implémentation déjà faite avec la bibliothèque `scipy`, cela n'a pas résolu le problème.
- En pratique, on ne pourra pas utiliser cette implémentation. On verra en section 4 une méthode alternative pour obtenir des conditions initiales utilisables.

3 Modèle de rivière pour une section rectangulaire avec hypothèses diffusives

Introduction

L'objectif de cette section est d'étudier un modèle de rivière pour une section rectangulaire avec hypothèses diffusives, en partant du problème (P_{SV1r}) .

3.1 Problème instationnaire

3.1.1 Présentation du problème

Soit l'hypothèse suivante :

$$b \frac{\partial}{\partial t} (hu) + \frac{\partial}{\partial s} (bhu^2) := 0 \quad (\text{absence d'inertie}).$$

Avec l'hypothèse, le problème (P_{SV1r}) devient le problème $(P_{SV1r\star})$:

Trouver h et u telles que :

$$\begin{cases} b \frac{\partial}{\partial t} (h) + \frac{\partial}{\partial s} (bhu) = q & \forall (s, t) \in \Omega_s \times \Omega_t \\ \frac{1}{2} g \frac{\partial}{\partial s} (bh^2) = g(S_0 - S_f)bh + qv & \forall (s, t) \in \Omega_s \times \Omega_t, \\ \text{conditions initiales} & \forall s \in \Omega_s \\ \text{conditions aux limites} & \forall t \in \Omega_t^* \end{cases}$$

ou encore :

Trouver A et Q telles que :

$$\begin{cases} \frac{\partial}{\partial t} (A) + \frac{\partial}{\partial s} (Q) = q & \forall (s, t) \in \Omega_s \times \Omega_t \\ \frac{1}{2} g \frac{\partial}{\partial s} \left(\frac{A^2}{b} \right) = g(S_0 - S_f)A + qv & \forall (s, t) \in \Omega_s \times \Omega_t, \\ \text{conditions initiales} & \forall s \in \Omega_s \\ \text{conditions aux limites} & \forall t \in \Omega_t^* \end{cases}$$

$$\begin{aligned} & \frac{1}{2} g \frac{\partial}{\partial s} \left(\frac{A^2}{b} \right) = g(S_0 - S_f)A + qv \\ \Leftrightarrow & \frac{1}{2} g \left(2 \frac{A}{b} \frac{\partial}{\partial s} (A) - \frac{A^2}{b^2} \frac{d}{ds} (b) \right) - g(S_0 - S_f)A = qv \\ \Leftrightarrow & \frac{1}{2} \left(2 \frac{1}{b} \frac{\partial}{\partial s} (A) - \frac{A}{b^2} \frac{d}{ds} (b) \right) - (S_0 - S_f) = \frac{qv}{gA} \\ \Leftrightarrow & \frac{1}{b} \frac{\partial}{\partial s} (A) - \frac{1}{2} \frac{A}{b^2} \frac{d}{ds} (b) - (S_0 - S_f) = \frac{qv}{gA} \\ \Leftrightarrow & \frac{1}{b} \frac{\partial}{\partial s} (A) - \frac{1}{2} \frac{A}{b^2} \frac{d}{ds} (b) = S_0 - S_f + \frac{qv}{gA} \\ \Leftrightarrow & \frac{\partial}{\partial s} (A) - \frac{1}{2} \frac{A}{b} \frac{d}{ds} (b) = b \left(S_0 - S_f + \frac{qv}{gA} \right), \end{aligned}$$

et :

$$S_f = \frac{r^2 u |u|}{R^{4/3}} = \frac{r^2 P^{4/3} u |u|}{A^{4/3}} = \frac{r^2 P^{4/3} Q |Q|}{A^{10/3}}.$$

Donc :

$$\begin{aligned} & \frac{1}{2} g \frac{\partial}{\partial s} \left(\frac{A^2}{b} \right) - g (S_0 - S_f) A = qv \\ \Leftrightarrow & \frac{\partial}{\partial s} (A) - \frac{1}{2} \frac{A}{b} \frac{d}{ds} (b) = b \left(S_0 - \frac{r^2 P^{4/3} Q |Q|}{A^{10/3}} + \frac{qv}{gA} \right) \\ \Leftrightarrow & \frac{\partial}{\partial s} (A) - \frac{1}{2} \frac{A}{b} \frac{d}{ds} (b) = b \left(S_0 + \frac{qv}{gA} \right) - b \frac{r^2 P^{4/3} Q |Q|}{A^{10/3}} \\ \Leftrightarrow & \frac{\partial}{\partial s} (A) - \frac{1}{2} \frac{A}{b} \frac{d}{ds} (b) - b \left(S_0 + \frac{qv}{gA} \right) = -b \frac{r^2 P^{4/3} Q |Q|}{A^{10/3}} \\ \Leftrightarrow & Q |Q| = \frac{A^{10/3}}{br^2 P^{4/3}} \left(-\frac{\partial}{\partial s} (A) + \frac{1}{2} \frac{A}{b} \frac{d}{ds} (b) + b \left(S_0 + \frac{qv}{gA} \right) \right). \end{aligned}$$

Proposition

Soient $Q, X \in \mathbb{R}$, et $\text{sgn} : \begin{cases} \mathbb{R} & \rightarrow & \{-1, 1\} \\ x & \mapsto & \begin{cases} 1 & \text{si } x \geq 0 \\ -1 & \text{sinon} \end{cases} \end{cases}$. Alors :

$$Q |Q| = X \Leftrightarrow Q = \text{sgn}(X) \sqrt{|X|}.$$

Démonstration

($\Rightarrow$) Si $Q |Q| = X$, alors : $\text{sgn}(X) \sqrt{|X|} = \text{sgn}(Q |Q|) \sqrt{|Q |Q|} = \text{sgn}(Q^2) \sqrt{Q^2} = \sqrt{Q^2} = Q$.
— Si $Q \geq 0$, alors : $\text{sgn}(Q |Q|) \sqrt{|Q |Q|} = \text{sgn}(Q^2) \sqrt{Q^2} = \sqrt{Q^2} = Q$.
— Si $Q < 0$, alors : $\text{sgn}(Q |Q|) \sqrt{|Q |Q|} = \text{sgn}(-Q^2) \sqrt{|Q^2|} = -|Q| = -(-Q) = Q$.
($\Leftarrow$) Si $Q = \text{sgn}(X) \sqrt{|X|}$, alors : $Q |Q| = \text{sgn}(X) \sqrt{|X|} \left| \text{sgn}(X) \sqrt{|X|} \right| = \text{sgn}(X) \sqrt{|X|} \sqrt{|X|}$
 $= \text{sgn}(X) \left(\sqrt{|X|} \right)^2 = \text{sgn}(X) |X| = X$.

Donc :

$$Q |Q| = X \Leftrightarrow Q = \text{sgn}(X) \sqrt{|X|}.$$

Remarque

Soit $f_{X,Q} : \begin{cases} \mathbb{R} & \rightarrow & \mathbb{R} \\ Q & \mapsto & Q |Q| \end{cases}$. Alors : $f_{X,Q}$ est bijective et $f_{X,Q}^{-1} : \begin{cases} \mathbb{R} & \rightarrow & \mathbb{R} \\ X & \mapsto & \text{sgn}(X) \sqrt{|X|} \end{cases}$.

Soient $Q_1 := \frac{A^{10/3}}{br^2 P^{4/3}} > 0$, et $Q_2 := -\frac{\partial}{\partial s} (A) + \frac{1}{2} \frac{A}{b} \frac{d}{ds} (b) + b \left(S_0 + \frac{qv}{gA} \right)$. Alors :

$$Q |Q| = Q_1 Q_2 \Leftrightarrow Q = \text{sgn}(Q_2) \sqrt{Q_1} \sqrt{|Q_2|}.$$

Donc :

$$\frac{1}{2} g \frac{\partial}{\partial s} \left(\frac{A^2}{b} \right) - g (S_0 - S_f) A = qv$$

$$\Leftrightarrow Q = \operatorname{sgn}(Q_2) \sqrt{Q_1} \sqrt{|Q_2|}.$$

Donc le problème $(P_{SV1r\star})$ se réécrit :

Trouver A et Q telles que :

$$\left\{ \begin{array}{ll} \frac{\partial}{\partial t}(A) + \frac{\partial}{\partial s}(Q) = q & \forall (s, t) \in \Omega_s \times \Omega_t \\ Q = \operatorname{sgn}(Q_2) \sqrt{Q_1} \sqrt{|Q_2|} & \forall (s, t) \in \Omega_s \times \Omega_t \\ \text{conditions initiales} & \forall s \in \Omega_s \\ \text{conditions aux limites} & \forall t \in \Omega_t^* \end{array} \right. . \quad (P_{SV1r\star})$$

Définition

On dit que le problème $(P_{SV1r\star})$ possède des hypothèses *diffusives*.

Commentaire

En comparaison avec (P_{SV1r}) , est une unique équation où l'inconnue est A , et où Q est donnée par une expression analytique qui dépend de A .

Remarque

Le terme $\frac{\partial}{\partial s}(Q)$ peut se réécrire comme $D(A) \frac{\partial^2}{\partial s^2}(A) + f(A)$ (avec $D(A)$ un terme de diffusion, et $f(A)$ un terme source), donc $\frac{\partial}{\partial t}(A) + \frac{\partial}{\partial s}(Q) = q$ est une équation de diffusion non linéaire.

3.1.2 Solutions exactes

Introduction

L'objectif de cette section est de présenter des exemples de solutions exactes.

Proposition

$$\forall (s, t) \in \Omega_s^* \times \Omega_t^* : \frac{\partial}{\partial t}(A) + \frac{\partial}{\partial s}(Q) = \frac{\partial}{\partial t}(A) + c(A) \frac{\partial}{\partial s}(A),$$

avec :

$$c(A) := \frac{d}{dA}(Q(A)).$$

Démonstration

On a $Q = Q(A)$. Alors :

$$\begin{aligned} \frac{\partial}{\partial t}(A) + \frac{\partial}{\partial s}(Q) &= 0 \\ \Leftrightarrow \frac{\partial}{\partial t}(A) + \frac{\partial}{\partial s}(Q(A)) &= 0. \end{aligned}$$

De plus,

$$\frac{\partial}{\partial s}(Q(A(s, t))) = \frac{d}{dA}(Q(A(s, t))) \frac{\partial}{\partial s}(A(s, t)).$$

Donc :

$$\begin{aligned} \frac{\partial}{\partial t}(A) + \frac{\partial}{\partial s}(Q) &= 0 \\ \Leftrightarrow \frac{\partial}{\partial t}(A) + \frac{d}{dA}(Q(A)) \frac{\partial}{\partial s}(A) &= 0 \\ \Leftrightarrow \frac{\partial}{\partial t}(A) + c(A) \frac{\partial}{\partial s}(A) &= 0, \end{aligned}$$

avec :

$$c(A) := \frac{d}{dA}(Q(A)).$$

Proposition

Si $Q > 0$, alors :

$$c(A) = \frac{\frac{A^{7/3}}{br^2 P(A)^{7/3}} \left(\frac{1}{3} \left(-\frac{\partial}{\partial s}(A) + \frac{1}{2} \frac{A}{b} \frac{d}{ds}(b) + b \left(S_0 + \frac{qv}{gA} \right) \right) \left(10P(A) - 8\frac{1}{b}A \right) + AP(A) \left(-\frac{\partial^2}{\partial s^2}(A) \left(\frac{\partial}{\partial s}(A) \right)^{-1} + \frac{1}{2} \frac{1}{b} \frac{d}{ds}(b) - b \frac{qv}{gA^2} \right) \right)}{2 \left(\sqrt{\frac{A^{10/3}}{br^2 P(A)^{4/3}} \left(-\frac{\partial}{\partial s}(A) + \frac{1}{2} \frac{A}{b} \frac{d}{ds}(b) + b \left(S_0 + \frac{qv}{gA} \right) \right)} \right)}. \quad (3.1)$$

Démonstration

Si $Q > 0$, et $Q := Q(A) := \sqrt{\frac{A^{10/3}}{br^2 P(A)^{4/3}} \left(-\frac{\partial}{\partial s}(A) + \frac{1}{2} \frac{A}{b} \frac{d}{ds}(b) + b \left(S_0 + \frac{qv}{gA} \right) \right)}$

$= \sqrt{\frac{1}{br^2} \frac{A^{10/3}}{P(A)^{4/3}} \left(-\frac{\partial}{\partial s}(A) + \frac{1}{2} \frac{A}{b} \frac{d}{ds}(b) + b \left(S_0 + \frac{qv}{gA} \right) \right)}$, alors :

$$\begin{aligned} \frac{d}{dA}(Q(A)^2) &= \frac{1}{br^2} \left(\left(-\frac{\partial}{\partial s}(A) + \frac{1}{2} \frac{A}{b} \frac{d}{ds}(b) + b \left(S_0 + \frac{qv}{gA} \right) \right) \frac{d}{dA} \left(\frac{A^{10/3}}{P(A)^{4/3}} \right) + \frac{A^{10/3}}{P(A)^{4/3}} \frac{d}{dA} \left(-\frac{\partial}{\partial s}(A) + \frac{1}{2} \frac{A}{b} \frac{d}{ds}(b) + b \left(S_0 + \frac{qv}{gA} \right) \right) \right) \\ &= \frac{d}{dA}(Q(A)^2) = \frac{1}{br^2} \left(\left(-\frac{\partial}{\partial s}(A) + \frac{1}{2} \frac{A}{b} \frac{d}{ds}(b) + b \left(S_0 + \frac{qv}{gA} \right) \right) d_1 + \frac{A^{10/3}}{P(A)^{4/3}} d_2 \right), \end{aligned}$$

avec :

$$d_1 := \frac{d}{dA} \left(\frac{A^{10/3}}{P(A)^{4/3}} \right), \quad \text{et} \quad d_2 := \frac{d}{dA} \left(-\frac{\partial}{\partial s}(A) + \frac{1}{2} \frac{A}{b} \frac{d}{ds}(b) + b \left(S_0 + \frac{qv}{gA} \right) \right).$$

D'une part :

$$d_1 = \frac{d}{dA} \left(\frac{A^{10/3}}{P(A)^{4/3}} \right) = \frac{P(A)^{4/3} \frac{d}{dA}(A^{10/3}) - A^{10/3} \frac{d}{dA}(P(A)^{4/3})}{P(A)^{8/3}},$$

et :

$$\frac{d}{dA}(P(A)^{4/3}) = \frac{4}{3} P(A)^{1/3} \frac{d}{dA}(P(A)) = \frac{4}{3} P(A)^{1/3} \frac{d}{dA} \left(b + 2\frac{A}{b} \right) = \frac{8}{3} \frac{1}{b} P(A)^{1/3}.$$

Donc :

$$d_1 = \frac{P(A)^{4/3} \frac{10}{3} A^{7/3} - A^{10/3} \frac{8}{3} \frac{1}{b} P(A)^{1/3}}{P(A)^{8/3}} = \frac{1}{3} \left(\frac{A}{P(A)} \right)^{7/3} \left(10P(A) - 8\frac{1}{b}A \right).$$

D'autre part :

$$d_2 = \frac{d}{dA} \left(-\frac{\partial}{\partial s}(A) + \frac{1}{2} \frac{A}{b} \frac{d}{ds}(b) + b \left(S_0 + \frac{qv}{gA} \right) \right) = \frac{d}{dA} \left(-\frac{\partial}{\partial s}(A) \right) + \frac{d}{dA} \left(\frac{1}{2} \frac{A}{b} \frac{d}{ds}(b) \right) + \frac{d}{dA} \left(b \left(S_0 + \frac{qv}{gA} \right) \right),$$

et :

$$\begin{aligned} \frac{\partial}{\partial s} \left(\frac{\partial}{\partial s}(A) \right) &= \frac{d}{dA} \left(\frac{\partial}{\partial s}(A) \right) \frac{\partial}{\partial s}(A) \\ \Leftrightarrow \frac{\partial^2}{\partial s^2}(A) &= \frac{d}{dA} \left(\frac{\partial}{\partial s}(A) \right) \frac{\partial}{\partial s}(A) \\ \Leftrightarrow \frac{d}{dA} \left(\frac{\partial}{\partial s}(A) \right) &= \frac{\partial^2}{\partial s^2}(A) \left(\frac{\partial}{\partial s}(A) \right)^{-1}. \end{aligned}$$

Donc :

$$d_2 = -\frac{\partial^2}{\partial s^2}(A) \left(\frac{\partial}{\partial s}(A) \right)^{-1} + \frac{1}{2} \frac{1}{b} \frac{d}{ds}(b) - b \frac{qv}{gA^2}.$$

Donc :

$$\begin{aligned} \frac{d}{dA} (Q(A)^2) &= \left(\left(-\frac{\partial}{\partial s}(A) + \frac{1}{2} \frac{A}{b} \frac{d}{ds}(b) + b \left(S_0 + \frac{qv}{gA} \right) \right) \frac{1}{3} \left(\frac{A}{P(A)} \right)^{7/3} \left(10P(A) - 8\frac{1}{b}A \right) + \frac{A^{10/3}}{P(A)^{4/3}} \left(-\frac{\partial^2}{\partial s^2}(A) \left(\frac{\partial}{\partial s}(A) \right)^{-1} + \frac{1}{2} \frac{1}{b} \frac{d}{ds}(b) - b \frac{qv}{gA^2} \right) \right) \\ &= \frac{A^{7/3}}{br^2P(A)^{7/3}} \left(\frac{1}{3} \left(-\frac{\partial}{\partial s}(A) + \frac{1}{2} \frac{A}{b} \frac{d}{ds}(b) + b \left(S_0 + \frac{qv}{gA} \right) \right) \left(10P(A) - 8\frac{1}{b}A \right) + AP(A) \left(-\frac{\partial^2}{\partial s^2}(A) \left(\frac{\partial}{\partial s}(A) \right)^{-1} + \frac{1}{2} \frac{1}{b} \frac{d}{ds}(b) - b \frac{qv}{gA^2} \right) \right). \end{aligned}$$

De plus :

$$\begin{aligned} \frac{d}{dA} (Q(A)^2) &= 2Q(A) \frac{d}{dA} (Q(A)) = 2Q(A) c(A) \\ \Leftrightarrow c(A) &= \frac{1}{2Q(A)} \frac{d}{dA} (Q(A)^2). \end{aligned}$$

Donc :

$$c(A) = \frac{\frac{A^{7/3}}{br^2P(A)^{7/3}} \left(\frac{1}{3} \left(-\frac{\partial}{\partial s}(A) + \frac{1}{2} \frac{A}{b} \frac{d}{ds}(b) + b \left(S_0 + \frac{qv}{gA} \right) \right) \left(10P(A) - 8\frac{1}{b}A \right) + AP(A) \left(-\frac{\partial^2}{\partial s^2}(A) \left(\frac{\partial}{\partial s}(A) \right)^{-1} + \frac{1}{2} \frac{1}{b} \frac{d}{ds}(b) - b \frac{qv}{gA^2} \right) \right)}{2 \left(\sqrt{\frac{A^{10/3}}{br^2P(A)^{4/3}} \left(-\frac{\partial}{\partial s}(A) + \frac{1}{2} \frac{A}{b} \frac{d}{ds}(b) + b \left(S_0 + \frac{qv}{gA} \right) \right)} \right)}.$$

Commentaires

- c dépend des dérivées partielles de A de manière non linéaire, donc c n'est pas une vitesse de transport.
- On verra que pour le prochain problème, on obtiendra c qui est une vitesse de transport.

3.1.3 Solutions approchées

Introduction

- En général, pour des paramètres quelconques, il n'existe de solution exacte connue de $(P_{SV1r}^{\text{initial}})$. Pour utiliser le modèle, il faut calculer une solution approchée avec une méthode d'analyse numérique.
- L'objectif de cette section est de présenter des solutions approchées.

Remarque

On va réutiliser les schéma aux différences finies de Lax-Friedrichs et aux volumes finis de Rusanov obtenus pour le problème (P_{SV1r}), uniquement appliqués à l'équation de la conservation de la masse. Q reste déterminé par l'expression exacte. On verra que pour le prochain problème, on utilisera d'autres schémas.

3.1.3.1 Schéma aux différences finies de Lax-Friedrichs

D'après (2.13), on obtient :

$$A_{k,n+1} \simeq -\frac{(\Delta_t)_n}{(\Delta_s)_k + (\Delta_s)_{k-1}} (Q_{k+1,n} - Q_{k-1,n}) + \frac{1}{(\Delta_s)_{k-1} + (\Delta_s)_k} ((\Delta_s)_{k-1} A_{k+1,n} + (\Delta_s)_k A_{k-1,n}) + (\Delta_t)_n q_{k,n}. \quad (3.2)$$

Proposition (complexité algorithmique)

La complexité algorithmique du schéma (3.2) est égale à $O(KN)$.

3.1.3.2 Schéma aux volumes finis de Rusanov

D'après (2.16), on obtient :

$$A_{k,n+1} \simeq -\frac{1}{2} \frac{(\Delta_t)_n}{(\Delta_s)_k} \left(Q_{k+1,n} - Q_{k-1,n} - \lambda_{k+1/2,n}^* (A_{k+1,n} - A_{k,n}) + \lambda_{k-1/2,n}^* (A_{k,n} - A_{k-1,n}) \right) + A_{k,n} + (\Delta_t)_n q_{k,n}. \quad (3.3)$$

Proposition (complexité algorithmique)

La complexité algorithmique du schéma (3.3) est égale à $O(KN)$.

Implémentation (résumé, voir [lien cliquable](#))

Les signatures et corps de fonctions qui appliquent le schéma (3.3) sont les suivantes :

```
static inline __attribute__((always_inline))
void calculer_Delta_t_n(
    double CFL,
    int K,
    int n,
    double *Delta_s,
    double *b,
    double *A,
    double *Q,
    double *Delta_t
);
```

```
static inline __attribute__((always_inline))
void calculer_P_n(
    int K,
    int n,
    double *b,
    double *A,
    double *P_n
);
```

```
static inline __attribute__((always_inline))
void calculer_R_n(
    int K,
    int n,
    double *P_n,
```

```

    double *A,
    double *R_n
);

```

```

static inline __attribute__((always_inline))
void calculer_S_f_n(
    int K,
    int n,
    double *b,
    double *r,
    double *A,
    double *Q,
    double *S_f_n
);

```

```

static inline __attribute__((always_inline))
void calculer_lambda_star_n(
    int K,
    int n,
    double *b,
    double *A,
    double *Q,
    double *lambda_star_n
)

```

```

static inline __attribute__((always_inline))
void appliquer_schema_A(
    int K,
    double *Delta_s,
    double *Delta_t,
    int n,
    double *lambda_star_n,
    double *q,
    double *A,
    double *Q
){
    for (int k = 1 ; k < K + 1 ; k ++){
        A[idx(k, n + 1)] = (
            -0.5 * (Delta_t[n] / Delta_s[k]) * (
                Q[idx(k + 1, n)] -
                Q[idx(k - 1, n)] -
                lambda_star_n[k] * (A[idx(k + 1, n)] - A[idx(k, n)]) +
                lambda_star_n[k - 1] * (A[idx(k, n)] - A[idx(k - 1, n)])
            ) +
            A[idx(k, n)] +
            Delta_t[n] * q[idx(k, n)]
        );
    }

    return;
}

```

```

static inline __attribute__((always_inline))
void appliquer_schema_Q(
    int K,
    double *Delta_s,
    double *Delta_t,
    int n,
    double *S_f_n,
    double *lambda_star_n,
    double *b,
    double *b_prime,
    double *r,
    double *S_0,
    double *q,
    double *v,
    double *A,
    double *Q
){

    double d_sur_ds_A_n_plus_1_k;
    double Q_1_n_plus_1_k;
    double Q_2_n_plus_1_k;
    double sgn_Q_2_n_plus_1_k;

    for (int k = 1 ; k < K + 1 ; k ++){

        if (k == 1){
            d_sur_ds_A_n_plus_1_k = (
                (A[idx(k + 1, n + 1)] - A[idx(k, n + 1)]) /
                (0.5 * (Delta_s[k] + Delta_s[k + 1]))
            );
        }
        else if (k == K){
            d_sur_ds_A_n_plus_1_k = (
                (A[idx(k, n + 1)] - A[idx(k - 1, n + 1)]) /
                (0.5 * (Delta_s[k - 1] + Delta_s[k]))
            );
        }
        else{
            d_sur_ds_A_n_plus_1_k = (
                (A[idx(k + 1, n + 1)] - A[idx(k - 1, n + 1)]) /
                (0.5 * Delta_s[k - 1] + Delta_s[k] + 0.5 * Delta_s[k + 1])
            );
        }

        Q_1_n_plus_1_k = (
            pow(A[idx(k, n + 1)], 10.0 / 3.0) /
            (
                b[k] * pow(r[k], 2) *
                pow((b[k] + 2.0 * A[idx(k, n + 1)] / b[k]), 4.0 / 3.0)
            )
        );
    }
}

```

```

    Q_2_n_plus_1_k = (
        -d_sur_ds_A_n_plus_1_k +
        0.5 * (A[idx(k, n + 1)] / b[k]) * b_prime[k] +
        b[k] * (
            S_0[k] +
            (q[idx(k, n + 1)] * v[idx(k, n + 1)]) /
            (g * A[idx(k, n + 1)])
        )
    );

    if (Q_2_n_plus_1_k > 0){
        sgn_Q_2_n_plus_1_k = 1;
    }
    else{
        sgn_Q_2_n_plus_1_k = -1;
    }

    Q[idx(k, n + 1)] = (
        sgn_Q_2_n_plus_1_k *
        sqrt(Q_1_n_plus_1_k) *
        sqrt(fabs(Q_2_n_plus_1_k))
    );

}

return;

```

```

}

```

```

static inline __attribute__((always_inline))
void appliquer_condition_limite(
    int type_condition_limite_A,
    int type_condition_limite_Q,
    int K,
    int n,
    double *A,
    double *Q
);

```

```

void resoudre_volume_fini_rusanov(
    int type_condition_limite_A,
    int type_condition_limite_Q,
    double CFL,
    int K,
    double *Delta_s,
    int N,
    double L_t,
    double *Delta_t,

```



```

double *t,
double *b,
double *b_prime,
double *r,
double *S_0,
double *q,
double *v,
double *A,
double *Q,
double *t_final_ptr
);

```

Remarques

- On a 3 versions de l'implémentation des fonctions qui appliquent le schéma (3.3) :
 - Séquentielle : [lien cliquable](#)
 - Parallèle avec OpenMP : [lien cliquable](#)
 - Parallèle avec MPI : [lien cliquable](#)
- Chaque version donne des résultats numériques identiques (à non-associativité des nombres flottants près).
- Les fonctions pour gérer la parallélisation avec MPI sont identiques à celles du paragraphe 2.1.4.7.
- Par manque de temps, on n'a testé l'implémentation du schéma 3.3 en comparant les résultats numériques avec une solution exacte, car aucune solution exacte exploitable n'a été trouvée.

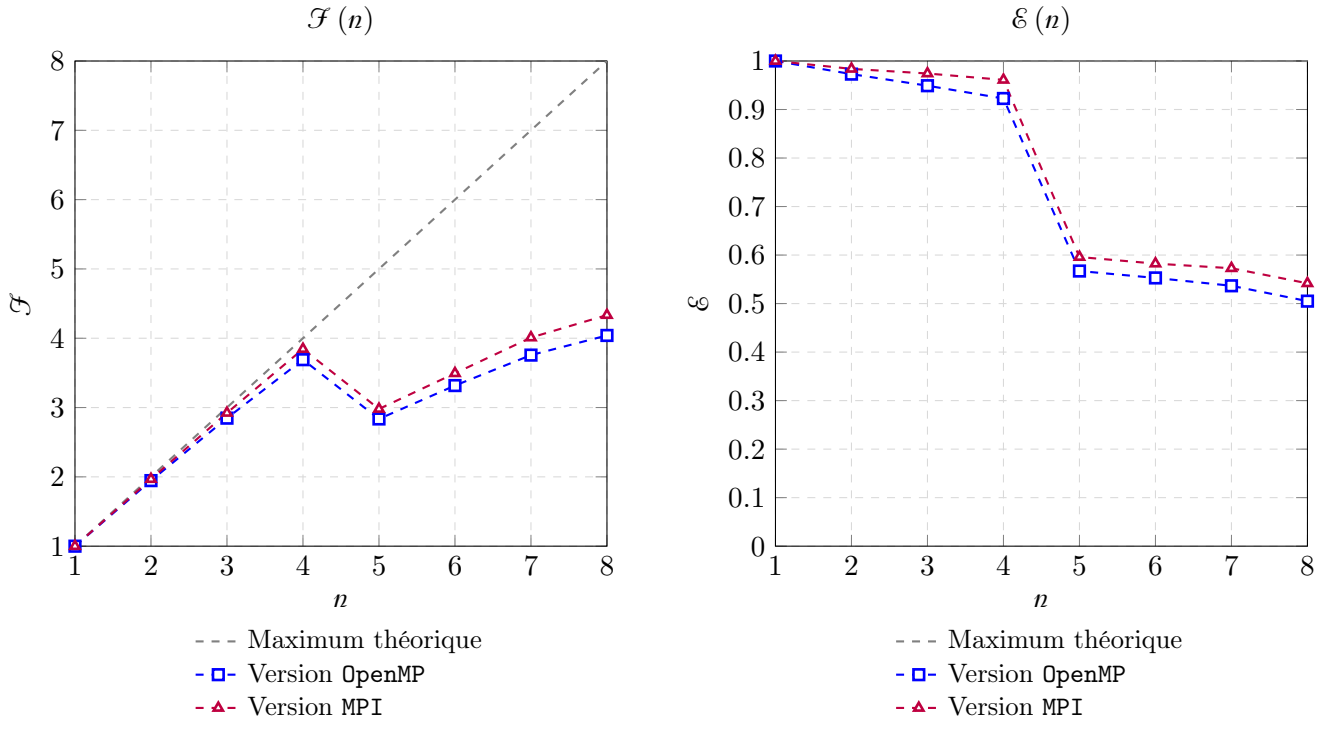
Résultats (*instabilités numérique*)

On teste ces implémentations du schéma (3.3), en recherchant des instabilités numériques, et on obtient les résultats suivants :

- Aucun paramètres tels que les résultats numériques divergent (lorsque $CFL \leq 1$) n'a été trouvé.
- Il existe parfois des légères oscillations en damier selon l'espace-temps, qui ne progressent pas au fur et à mesure des itérations.

Résultats (*parallélisation*)

On teste ces implémentations du schéma (3.3) (pour les paramètres suivants : $K, N := 15000$), en mesurant et comparant les facteurs d'accélération et efficacité de parallélisation entre les versions, et on obtient les résultats suivants (moyennés sur 10 exécutions) :



Interprétations

- Pour les versions `OpenMP` et `MPI`, l'efficacité de parallélisation est excellente lorsque $n \in \{2, 3, 4\}$ (entre 92% et 98%).
- Pour la version `OpenMP`, si $n = 4$, alors l'efficacité de parallélisation est égale à 92%.
- Pour la version `MPI`, si $n = 4$, alors l'efficacité de parallélisation est égale à 96%.
- Pour les versions `OpenMP` et `MPI`, l'efficacité de parallélisation est strictement décroissante.
- Pour les versions `OpenMP` et `MPI`, l'efficacité de parallélisation chute entre $n = 4$ et $n = 5$ (perte de 36% à 37%). On peut expliquer cela par l'architecture de la machine qui a 4 cœurs physiques et 8 threads logiques (avec 2 threads par cœur).
- L'efficacité de parallélisation est la meilleure pour la version `MPI` (1% à 4% de plus par rapport à la version `OpenMP`).
- Les résultats sont prometteurs en vue d'une parallélisation sur un cluster de calcul.

Résultats (temps d'exécution)

On teste l'implémentation du schéma (3.3) en version séquentielle, en mesurant les temps d'exécution, et on obtient les résultats suivants (moyennés sur 10 exécutions) :

Temps d'exécution (en s) en fonction de K et N :

$\downarrow K \quad N \rightarrow$	3000	6000	9000	12000	15000
3000	0.51	1.03	1.54	2.06	2.56
6000	1.03	2.06	3.08	4.09	5.13
9000	1.54	3.09	4.62	6.14	7.68
12000	2.05	4.12	6.17	8.19	10.25
15000	2.57	5.14	7.72	10.30	12.86

Coefficient du modèle de temps d'exécution en fonction de K et N :

$\downarrow K \quad N \rightarrow$	3000	6000	9000	12000	15000
3000	5.67×10^{-8}	5.72×10^{-8}	5.70×10^{-8}	5.72×10^{-8}	5.69×10^{-8}
6000	5.72×10^{-8}	5.72×10^{-8}	5.70×10^{-8}	5.68×10^{-8}	5.70×10^{-8}
9000	5.70×10^{-8}	5.72×10^{-8}	5.70×10^{-8}	5.69×10^{-8}	5.69×10^{-8}
12000	5.69×10^{-8}	5.72×10^{-8}	5.71×10^{-8}	5.69×10^{-8}	5.69×10^{-8}
15000	5.71×10^{-8}	5.71×10^{-8}	5.72×10^{-8}	5.72×10^{-8}	5.72×10^{-8}

Interprétations

- La matrice des temps d'exécution est pratiquement symétrique.
- La complexité algorithmique est vérifiée.
- Pour l'architecture de la machine, on peut établir la loi suivante :

$$\mathcal{T}(K, N) \simeq 5.71 \cdot 10^{-8} \cdot KN.$$

Résultats (opérations arithmétiques flottantes, accès mémoires, et temps d'exécution des fonctions)

On teste l'implémentation du schéma (3.3) en version séquentielle, en mesurant les opérations arithmétiques flottantes, les accès mémoires et le temps d'exécution des fonctions (avec le paquet **gprof**), et on obtient les résultats suivants (moyennés sur 5 exécutions) :

Propriété → ↓ Fonction	Nombre d'opérations arithmétiques flottantes									Nombre d'accès mémoire	Intensité arithmétique	Temps d'exécution (en %)
	+	-	*	/	fabs	sqrt	pow	<, >, ==, ...	Total			
calculer _Delta_t_n	1		1	3	1	1		1	8	5	0.2	8.8
calculer _P_n	1		1	1					3	4	0.1	4.7
calculer _R_n				1					1	3	< 0.1	3.6
calculer _S_f_n			4	2	1		1		8	6	0.2	2.8
calculer _lambda_star_n	2		2	4	2	2		1	13	9	0.2	13.0
appliquer _schema_A	3	4	5	1					13	14	0.1	21.2
appliquer _schema_Q	6	1	12	7	1	2	2	2	33	20	0.2	46.0
appliquer _condition_limite								5	5	8	0.1	0.0
autres	1								1	2	0.1	0.0
itération complète	14	5	25	19	5	5	3	9	85	71	0.1	100.0

Interprétations

- Les fonctions qui ont le temps d'exécution le plus élevé sont `appliquer_schema_Q` (46.0%), et `appliquer_schema_A` (21.2%). Ces 2 fonctions représentent 67.2% du temps d'exécution.
- Les fonctions qui ont le nombre d'opérations arithmétiques flottantes le plus élevé sont `appliquer_schema_Q` (33), `appliquer_schema_A` (13), et `calculer_lambda_star_n` (13).
- Les fonctions qui ont le nombre d'accès mémoire le plus élevé sont `appliquer_schema_Q` (20), et `appliquer_schema_A` (14).
- On estime que les fonctions `appliquer_schema_Q`, et `appliquer_schema_A` représentent une grande partie du temps d'exécution.
- On estime que les fonctions `calculer_lambda_star_n`, et `calculer_Delta_t_n` représentent une partie modérée du temps d'exécution.
- On estime que les fonctions `calculer_P_n`, `calculer_R_n`, et `calculer_S_f_n` représentent une petite partie du temps d'exécution.
- On estime que la fonction `appliquer_condition_limite` représente une partie négligeable du temps d'exécution.
- L'intensité arithmétique est très faible. Le temps des accès mémoire représente probablement une grande partie du temps d'exécution.

3.2 Problème stationnaire, et conditions initiales

3.2.1 Présentation du problème

Si $\frac{\partial}{\partial t}(A) := 0$, $\frac{\partial}{\partial s} := \frac{d}{ds}$, $A := A(s)$, et $q \equiv v \equiv 0$ alors :

$$\begin{aligned}\frac{\partial}{\partial t}(A) + \frac{\partial}{\partial s}(Q) &= q \\ \Rightarrow \frac{d}{ds}(Q) &= 0 \\ \Leftrightarrow Q &=: Q_C \in \mathbb{R},\end{aligned}$$

et :

$$\begin{aligned}Q_C &= \text{sgn}(Q_2) \sqrt{Q_1} \sqrt{|Q_2|} \\ \Leftrightarrow Q_C |Q_C| &= -\frac{A^{10/3}}{br^2 P^{4/3}} \left(\frac{d}{ds}(A) - \frac{1}{2} \frac{A}{b} \frac{d}{ds}(b) - bS_0 \right) \\ \Leftrightarrow Q_C |Q_C| &= -\frac{A^{10/3}}{br^2 P^{4/3}} \left(\frac{d}{ds}(A) \right) + \frac{A^{10/3}}{br^2 P^{4/3}} \left(\frac{1}{2} \frac{A}{b} \frac{d}{ds}(b) + bS_0 \right) \\ \Leftrightarrow -\frac{A^{10/3}}{br^2 P^{4/3}} \left(\frac{d}{ds}(A) \right) &= Q_C |Q_C| - \frac{A^{10/3}}{br^2 P^{4/3}} \left(\frac{1}{2} \frac{A}{b} \frac{d}{ds}(b) + bS_0 \right) \\ \Leftrightarrow \frac{d}{ds}(A) &= -\frac{br^2 P^{4/3}}{A^{10/3}} \left(Q_C |Q_C| - \frac{A^{10/3}}{br^2 P^{4/3}} \left(\frac{1}{2} \frac{A}{b} \frac{d}{ds}(b) + bS_0 \right) \right) \\ \Leftrightarrow \frac{d}{ds}(A) &= -\frac{bQ_C |Q_C| r^2 P^{4/3}}{A^{10/3}} + \frac{1}{2} \frac{A}{b} \frac{d}{ds}(b) + bS_0 \\ \Leftrightarrow A' &= -\frac{bQ_C |Q_C| r^2 P^{4/3}}{A^{10/3}} + \frac{1}{2} \frac{A}{b} b' + bS_0\end{aligned}$$

$$\Leftrightarrow A' = \varphi(A, s),$$

avec :

$$A' := \frac{d}{ds}(A) \quad b' := \frac{d}{ds}(b), \quad \text{et} \quad \varphi(A, s) := -\frac{bQ_C |Q_C| r^2 P^{4/3}}{A^{10/3}} + \frac{1}{2} \frac{A}{b} b' + bS_0.$$

Soit le problème $(P_{SV1r\star}^{\text{initial}})$ suivant :

Trouver A telle que :

$$\left\{ \begin{array}{l} A' = \varphi(A, s) \quad \forall s \in \Omega_s \\ Q_C \in \mathbb{R} \\ \text{conditions aux limites} \end{array} \right. . \quad (P_{SV1r\star}^{\text{initial}})$$

Commentaires

Le problème $(P_{SV1r\star}^{\text{initial}})$ est équation différentielle ordinaire d'ordre 1 non linéaire, et est un problème de Cauchy.

3.2.2 Solutions approchées

Remarque

On utilise le schéma (2.19), avec la définition de φ de $(P_{SV1r\star}^{\text{initial}})$.

Résultats (*instabilités numérique*)

On teste ces implémentations du schéma (2.19), en recherchant des instabilités numériques, et on obtient les résultats suivants :

- En général, les résultats numériques divergent. On peut expliquer cela par le fait que le problème $(P_{SV1r}^{\text{initial}})$ est dit *raide*.
- Pour tenter de supprimer les instabilité numériques, on a testé une implémentation déjà faite avec la bibliothèque `scipy`, cela n'a pas résolu le problème.
- En pratique, on ne pourra pas utiliser cette implémentation. On verra en section 4 une méthode alternative pour obtenir des conditions initiales utilisables.
- On obtient exactement les mêmes types d'instabilités numériques que pour le problème $(P_{SV1r\star})$.

4 Modèle de rivière pour une section rectangulaire avec hypothèses cinématiques

Introduction

L'objectif de cette section est d'étudier un modèle de rivière pour une section rectangulaire avec hypothèses cinématiques, en partant du problème $(P_{SV1r\star})$.

4.1 Problème instationnaire

4.1.1 Présentation du problème

Soient les hypothèses suivante :

$$\frac{\partial}{\partial s} \left(\frac{1}{2} g b h^2 \right) := 0 \quad (\text{absence de pression}),$$

$$S_f := S_0 \quad (\text{la pente de frottement est égale à la pente du fond de la rivière}),$$

et

$$u \geq 0 \quad (\text{les particules ne peuvent pas se déplacer vers l'amont}).$$

Si $S_f := S_0$, $S_0 \geq 0$, et $u \geq 0$, alors :

$$S_f = S_0 \Leftrightarrow \frac{r^2 u^2}{R^{4/3}} = S_0 \Leftrightarrow u = \frac{R^{2/3} \sqrt{S_0}}{r}.$$

Avec les hypothèses, le problème $(P_{SV1r\star})$ devient le problème $(P_{SV1r\star\star})$:

Trouver h et u telles que :

$$\begin{cases} b \frac{\partial}{\partial t} (h) + \frac{\partial}{\partial s} (b h u) = q & \forall (s, t) \in \Omega_s \times \Omega_t \\ u = \frac{\sqrt{S_0} R^{2/3}}{r} & \forall (s, t) \in \Omega_s \times \Omega_t, \\ \text{conditions initiales} & \forall s \in \Omega_s \\ \text{conditions aux limites} & \forall t \in \Omega_t^* \end{cases}$$

ou encore :

Trouver A et Q telles que :

$$\boxed{\begin{cases} \frac{\partial}{\partial t} (A) + \frac{\partial}{\partial s} (Q) = q & \forall (s, t) \in \Omega_s \times \Omega_t \\ Q = \frac{\sqrt{S_0} A R^{2/3}}{r} & \forall (s, t) \in \Omega_s \times \Omega_t, \\ \text{conditions initiales} & \forall s \in \Omega_s \\ \text{conditions aux limites} & \forall t \in \Omega_t^* \end{cases}}. \quad (P_{SV1r\star\star})$$

Définition

On dit que le problème $(P_{SV1r\star\star})$ possède des hypothèses *cinématiques*.

Commentaires

- En comparaison avec (P_{SV1r}) , est une unique équation où l'inconnue est A , et où Q est donnée par une expression analytique qui dépend de A .
- Q suit la *loi de Manning*.

4.1.2 Solutions exactes

Introduction

L'objectif de cette section est de présenter des exemples de solutions exactes.

Proposition

$$\forall (s, t) \in \Omega_s^* \times \Omega_t^* : \frac{\partial}{\partial t} (A) + \frac{\partial}{\partial s} (Q) = \frac{\partial}{\partial t} (A) + c(A) \frac{\partial}{\partial s} (A),$$

avec :

$$c(A) := \frac{d}{dA} (Q(A)).$$

Démonstration

On a $Q = Q(A)$. Alors :

$$\begin{aligned} \frac{\partial}{\partial t} (A) + \frac{\partial}{\partial s} (Q) &= 0 \\ \Leftrightarrow \frac{\partial}{\partial t} (A) + \frac{\partial}{\partial s} (Q(A)) &= 0. \end{aligned}$$

De plus,

$$\frac{\partial}{\partial s} (Q(A(s, t))) = \frac{d}{dA} (Q(A(s, t))) \frac{\partial}{\partial s} (A(s, t)).$$

Donc :

$$\begin{aligned} \frac{\partial}{\partial t} (A) + \frac{\partial}{\partial s} (Q) &= 0 \\ \Leftrightarrow \frac{\partial}{\partial t} (A) + \frac{d}{dA} (Q(A)) \frac{\partial}{\partial s} (A) &= 0 \\ \Leftrightarrow \frac{\partial}{\partial t} (A) + c(A) \frac{\partial}{\partial s} (A) &= 0, \end{aligned}$$

avec :

$$c(A) := \frac{d}{dA} (Q(A)).$$

Proposition

$$\boxed{c(A) = \frac{\sqrt{S_0}}{r} \left(R(A)^{2/3} + \frac{2}{3} \frac{bA^{2/3}}{P(A)^{5/3}} \right)}. \quad (4.1)$$

Démonstration

Si $Q(A) := \frac{\sqrt{S_0}AR(A)^{2/3}}{r} = \frac{\sqrt{S_0}}{r}AR(A)^{2/3}$, alors :

$$\begin{aligned} c(A) &= \frac{d}{dA}(Q(A)) = \frac{\sqrt{S_0}}{r} \left(R(A)^{2/3} \frac{d}{dA}(A) + A \frac{d}{dA}(R(A)^{2/3}) \right) \\ &= \frac{\sqrt{S_0}}{r} \left(R(A)^{2/3} + A \frac{d}{dA}(R(A)^{2/3}) \right), \end{aligned}$$

et :

$$\frac{d}{dA}(R(A)^{2/3}) = \frac{2}{3} \left(\frac{A}{b + 2\frac{A}{b}} \right)^{-1/3} \frac{b}{\left(b + 2\frac{A}{b}\right)^2} = \frac{2}{3} \frac{bR(A)^{-1/3}}{P(A)^2}.$$

Donc :

$$c(A) = \frac{\sqrt{S_0}}{r} \left(R(A)^{2/3} + \frac{2}{3} \frac{bAR(A)^{-1/3}}{P(A)^2} \right) = \frac{\sqrt{S_0}R(A)^{-1/3}}{r} \left(R(A) + \frac{2}{3} \frac{bA}{P(A)^2} \right).$$

Remarque

Le problème $(P_{SV1r\star\star})$ est une équation de transport non linéaire avec une vitesse de transport $c(A)$.

Proposition (*existence d'une solution exacte*)

$$\boxed{\exists A_0 \in C(\mathbb{R}), \forall (s, t) \in \Omega_s^* \times \Omega_t^* : A(s, t) = A_0(-c(A(s, t))t + s)}. \quad (4.2)$$

Démonstration

On cherche une courbe $\mathcal{C} := \{(s(t), t) \mid t \in \Omega_t\}$ telle que $\forall t \in \Omega_t : A(s(t), t) = k$. Soit $(s(t), t) \in \mathcal{C}$, alors :

$$\begin{aligned} A(s(t), t) &= k \\ \Leftrightarrow \frac{d}{dt}(A(s(t), t)) &= 0 \\ \Leftrightarrow \left(\frac{\partial}{\partial s}(A) \right)(s(t), t) \frac{d}{dt}(s(t)) + \left(\frac{\partial}{\partial t}(A) \right)(s(t), t) &= 0. \\ \Leftrightarrow \left(\frac{\partial}{\partial t}(A) \right)(s(t), t) + \left(\frac{\partial}{\partial s}(A) \right)(s(t), t) \frac{d}{dt}(s(t)) &= 0. \end{aligned}$$

Donc (d'après $(P_{SV1r\star\star})$) :

$$c(A(s(t), t)) = \frac{d}{dt}(s(t)).$$

A est constant, donc $c(A)$ est constant, donc $\frac{d}{dt}(s(t))$ est constant, donc :

$$s(t) = c(A(s(t), t))t + s(0).$$

Soit $(s, t) \in \Omega_s^* \times \Omega_t^*$, alors :

$$\exists s_0 \in \Omega_s : s = c(A(s(t), t))t + s_0.$$

Soit $A_0(s_0) := A(s_0, 0)$, alors :

$$\begin{aligned} s &= c(A_0(s_0))t + s_0 \\ \Leftrightarrow s_0 &= -c(A_0(s_0))t + s \\ \Rightarrow A(s, t) &= A_0(-c(A(s, t))t + s). \end{aligned}$$

On vérifie que $A_0(-c(A(s, t))t + s)$ est solution de (P_{SV1r**}) . Soit $c^*(s, t) := -c(A(s, t)t + s)$. Alors :

$$\frac{\partial}{\partial t}(A_0(-c(A(s, t))t + s)) = \frac{\partial}{\partial t}(A_0(c^*(s, t))) = \frac{d}{dc^*}(A_0(c^*(s, t))) \frac{\partial}{\partial t}(c^*(s, t)),$$

et :

$$\begin{aligned} \frac{\partial}{\partial t}(c^*(s, t)) &= \frac{\partial}{\partial t}(-c(A(s, t))t + s) = -\frac{\partial}{\partial t}(c(A(s, t))t) + \frac{\partial}{\partial t}(s) \\ &= -\frac{d}{dA}(c(A(s, t))) \frac{\partial}{\partial t}(A(s, t))t - c(A(s, t)). \end{aligned}$$

Donc :

$$\frac{\partial}{\partial t}(A_0(c^*)) = \frac{d}{dc^*}(A_0(c^*)) \left(-\frac{d}{dA}(c(A)) \frac{\partial}{\partial t}(A)t - c(A) \right).$$

De plus,

$$\frac{\partial}{\partial s}(A_0(-c(A(s, t))t + s)) = \frac{\partial}{\partial s}(A_0(c^*(s, t))) = \frac{d}{dc^*}(A_0(c^*(s, t))) \frac{\partial}{\partial s}(c^*(s, t)),$$

et :

$$\begin{aligned} \frac{\partial}{\partial s}(c^*(s, t)) &= \frac{\partial}{\partial s}(-c(A(s, t))t + s) = -\frac{\partial}{\partial s}(c(A(s, t))t) + \frac{\partial}{\partial s}(s) \\ &= -\frac{d}{dA}(c(A(s, t))) \frac{\partial}{\partial s}(A(s, t))t + 1. \end{aligned}$$

Donc :

$$\frac{\partial}{\partial s}(A_0(c^*)) = \frac{d}{dc^*}(A_0(c^*)) \left(-\frac{d}{dA}(c(A)) \frac{\partial}{\partial s}(A)t + 1 \right).$$

Enfin :

$$\begin{aligned} &\frac{d}{dc^*}(A_0(c^*)) \left(-\frac{d}{dA}(c(A)) \frac{\partial}{\partial t}(A)t - c(A) \right) + c(A) \frac{d}{dc^*}(A_0(c^*)) \left(-\frac{d}{dA}(c(A)) \frac{\partial}{\partial s}(A)t + 1 \right) \\ &= -\frac{d}{dc^*}(A_0(c^*)) \left(\frac{d}{dA}(c(A)) \frac{\partial}{\partial t}(A)t + c(A) + c(A) \frac{d}{dA}(c(A)) \frac{\partial}{\partial s}(A)t - c(A) \right) \\ &= -\frac{d}{dc^*}(A_0(c^*)) \frac{d}{dA}(c(A))t \underbrace{\left(\frac{\partial}{\partial t}(A) + c(A) \frac{\partial}{\partial s}(A) \right)}_{=0} = 0. \end{aligned}$$

Donc $A_0(-c(A(s, t))t + s)$ est solution de (P_{SV1r**}) .

Donc :

$$\exists A_0 \in C(\mathbb{R}), \forall (s, t) \in \Omega_s^* \times \Omega_t^* : A(s, t) = A_0(-c(A(s, t))t + s).$$

Exemple

Par simplification, on suppose que $Q(A) := \alpha A^2$, et $A_0(s) := \gamma(s+1)$ avec $\alpha, \gamma \in \mathbb{R}$. Alors :

$$c(A) = 2\alpha A,$$

et :

$$\begin{aligned} A(s, t) &= \gamma(-c(A(s, t))t + s + 1) = \gamma(-2\alpha A(s, t)t + s + 1) \\ \Rightarrow A(s, t) &= \frac{\gamma(s+1)}{1+2\alpha\gamma t} \quad (\text{pour } t \neq -\frac{1}{2\alpha\gamma}). \end{aligned}$$

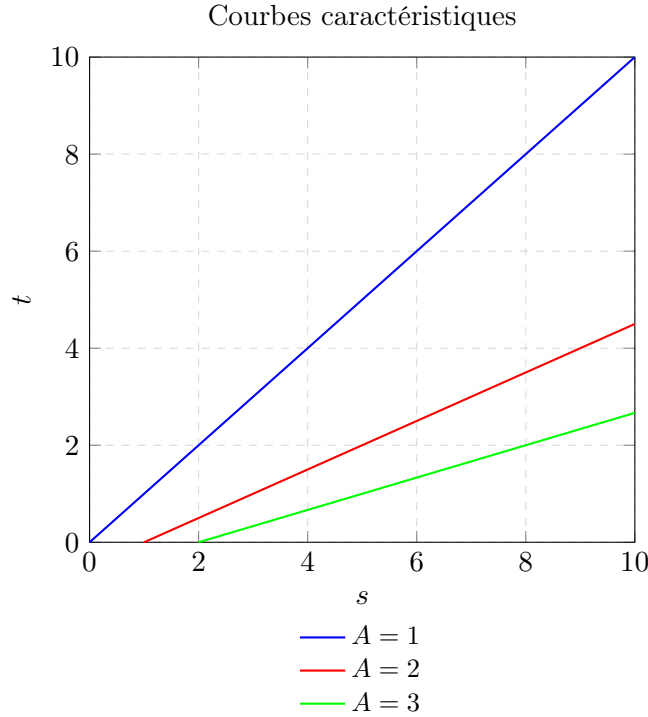
Si $\alpha := \frac{1}{2}$, et $\gamma := 1$, alors :

$$c(A) = A,$$

et :

$$\boxed{A(s, t) = \frac{s+1}{t+1}}. \quad (4.3)$$

On trace quelques courbes caractéristiques :



Les courbes caractéristiques sont des droites.

Commentaires

- Lorsque les expressions de Q et A_0 sont sous une forme analytique suffisamment simple, alors on a une solution exacte explicite pour A .
- En général, on a une solution exacte implicite pour A , et les courbes caractéristiques sont des courbes polynomiales.

- Pour le problème $(P_{SV1r\star})$, l'expression de $c(A)$ ne permet pas d'obtenir une solution exacte explicite. On verra que pour le prochain problème, l'expression de $c(A)$ sera suffisamment simple pour utiliser une méthode d'analyse numérique qui permet d'obtenir une solution explicite.

D'après (4.2), on trouve une solution exacte de $(P_{SV1r\star\star})$, avec des paramètres quelconques (au choix), et les conditions initiales suivantes :

$$A(s, 0) := \gamma s,$$

avec :

$$\gamma \in \mathbb{R}.$$

Alors, la solution exacte est la suivante :

$$A(s, t) = \gamma \left(-\frac{\sqrt{S_0} R(A(s, t))^{-1/3}}{r} \left(R(A(s, t)) + \frac{2}{3} \frac{bA(s, t)}{P(A(s, t))^2} \right) t + s \right).$$

$$\Leftrightarrow \boxed{\varphi(A) = 0}, \quad (4.4)$$

avec :

$$\varphi(A) := A - \gamma(-c(A)t + s).$$

Commentaires

- Lorsque Q suit la loi de Manning, la solution exacte reste implicite et nécessite la résolution d'équations algébriques non linéaires.
- On utilisera la méthode de Newton-Raphson pour approcher cette solution exacte, et on considérera que la solution obtenue est exacte.

Proposition

$$\frac{d}{dA}(\varphi(A)) = 1 + \frac{2}{3}\gamma b \left(\frac{\sqrt{S_0} R(A)^{-1/3}}{r P(A)^2} + \frac{2}{3} A^{-1/3} P(A)^{-8/3} \left(P(A) - 5 \frac{A}{b} \right) \right) t.$$

Démonstration

$$\begin{aligned} \frac{d}{dA}(\varphi(A)) &= \frac{d}{dA}(A - \gamma(-c(A)t + s + 1)) = \frac{d}{dA}(A) + \frac{d}{dA}(-\gamma(-c(A)t + s + 1)) \\ &= 1 - \gamma \frac{d}{dA}(-c(A)t + s + 1) = 1 - \gamma \left(-t \frac{d}{dA}(c(A)) + \frac{d}{dA}(s + 1) \right) = 1 + \gamma t \frac{d}{dA}(c(A)), \end{aligned}$$

et :

$$\begin{aligned} \frac{d}{dA}(c(A)) &= \frac{d}{dA} \left(\frac{\sqrt{S_0}}{r} \left(R(A)^{2/3} + \frac{2}{3} \frac{bA^{2/3}}{P(A)^{5/3}} \right) \right) \\ &= \frac{\sqrt{S_0}}{r} \frac{d}{dA} \left(R(A)^{2/3} \right) + \frac{d}{dA} \left(\frac{2}{3} \frac{bA^{2/3}}{P(A)^{5/3}} \right) = \frac{\sqrt{S_0}}{r} d_1 + d_2, \end{aligned}$$

avec :

$$d_1 := \frac{d}{dA} \left(R(A)^{2/3} \right), \quad \text{et} \quad d_2 := \frac{d}{dA} \left(\frac{2}{3} \frac{bA^{2/3}}{P(A)^{5/3}} \right).$$

D'une part (déjà démontré) :

$$d_1 = \frac{d}{dA} \left(R(A)^{2/3} \right) = \frac{2}{3} \frac{bR(A)^{-1/3}}{P(A)^2}.$$

D'autre part :

$$d_2 = \frac{d}{dA} \left(\frac{2}{3} \frac{bA^{2/3}}{P(A)^{5/3}} \right) = \frac{2}{3} b \frac{d}{dA} \left(\frac{A^{2/3}}{P(A)^{5/3}} \right) = \frac{2}{3} b \frac{\frac{2}{3} A^{-1/3} P(A)^{5/3} - A^{2/3} \frac{d}{dA} (P(A)^{5/3})}{P(A)^{10/3}},$$

et :

$$\frac{d}{dA} (P(A)^{5/3}) = \frac{5}{3} \frac{d}{dA} (P(A)) = \frac{10}{3} \frac{P(A)^{2/3}}{b}.$$

Donc :

$$d_2 = \frac{2}{3} b \frac{\frac{2}{3} A^{-1/3} P(A)^{5/3} - A^{2/3} \frac{10}{3} \frac{P(A)^{2/3}}{b}}{P(A)^{10/3}} = \frac{4}{9} b A^{-1/3} P(A)^{-8/3} \left(P(A) - 5 \frac{A}{b} \right).$$

Donc :

$$\frac{d}{dA} (c(A)) = \frac{2}{3} \frac{b\sqrt{S_0}R(A)^{-1/3}}{rP(A)^2} + \frac{4}{9} b A^{-1/3} P(A)^{-8/3} \left(P(A) - 5 \frac{A}{b} \right).$$

Donc :

$$\frac{d}{dA} (\varphi(A)) = 1 + \frac{2}{3} \gamma b \left(\frac{\sqrt{S_0}R(A)^{-1/3}}{rP(A)^2} + \frac{2}{3} A^{-1/3} P(A)^{-8/3} \left(P(A) - 5 \frac{A}{b} \right) \right) t.$$

Implémentation

Soient $\varepsilon > 0$ suffisamment petit, et $\varepsilon \in \mathbb{N}^*$ suffisamment grand. Alors, un algorithme qui applique la résolution du problème (4.4) est le suivant :

```

function RESOUDRE_NEWTON_RAPHSON_CINEMATIQUE_EXACTE(( $A_{k,n}$ )depart,  $s_k$ ,  $t_n$ ,  $b_k$ ,  $r_k$ , ( $S_0$ )k)
   $A_{k,n} \leftarrow (A_{k,n})_{\text{depart}}$ 
  arret  $\leftarrow$  0
  compteur  $\leftarrow$  0
  while compteur <  $\varepsilon$  and arret = 0 do
     $\varphi(A_{k,n}) \leftarrow \varphi(A_{k,n}, s_k, t_n, b_k, r_k, (S_0)_k)$ 
     $\varphi'(A_{k,n}) \leftarrow \varphi'(A_{k,n}, t_n, b_k, r_k, (S_0)_k)$ 
    ( $A_{k,n}$ )suivant  $\leftarrow A_{k,n} - \frac{\varphi(A_{k,n})}{\varphi'(A_{k,n})}$ 
    if  $|\varphi(A_{k,n})| < \varepsilon$  or  $|\varphi'(A_{k,n})| < \varepsilon$  or  $|(A_{k,n})_{\text{suivant}} - A_{k,n}| < \varepsilon$  then
      arret  $\leftarrow$  1
    end if
     $A_{k,n} \leftarrow (A_{k,n})_{\text{suivant}}$ 
    compteur  $\leftarrow$  compteur + 1
  end while
  return  $A_{k,n}$ 

```

end function

Commentaires

- On utilise $\frac{d}{dA}(\varphi(A))$ pour la méthode de Newton-Raphson.
- La méthode de Newton-Raphson permet de résoudre $\varphi(A) = 0$.
- On applique l'algorithme de la méthode de Newton-Raphson au niveau discret, pour tout k, n .

Remarque

D'après *Vatankhak et al. 2013 [12]*, la loi de Manning peut être approchée de la manière suivante :

$$Q(A) \simeq \alpha A^\beta,$$

avec :

$$\alpha := \frac{\lambda \sqrt{S_0} \psi(b)^{5/3-\beta}}{r \left(b + \frac{2\psi(b)^{2/3}}{b} \right)}, \quad \beta := \frac{5}{3} \left(1 - \frac{\frac{4}{5}\psi(b)}{b^2 + 2\psi(b)} \right),$$

$$\lambda := \begin{cases} 0.958 & \text{si } 4 \leq b < 10 \\ 0.968 & \text{si } 10 \leq b \leq 400 \end{cases}, \quad \text{et} \quad \psi(b) := \begin{cases} 1.064b^{0.928} & \text{si } 4 \leq b < 10 \\ 0.274b^{1.608} & \text{si } 10 \leq b \leq 400 \end{cases}.$$

On ne choisira pas cette approximation, mais la loi de Manning exacte.

4.1.3 Solutions approchées

Introduction

On va utiliser des schéma aux différences finies et aux volumes finis, uniquement appliqués à l'équation de la conservation de la masse. Comme la vitesse de l'onde est strictement positive (les ondes se transmettent uniquement de l'amont vers l'aval), alors on va utiliser des approximations arrières. Q reste déterminé par la l'expression exacte.

Notation

On note $c_k := c(A(s_k))$.

4.1.3.1 Schéma aux différences finies arrière

D'après les approximations (2.9) et (2.11), on obtient :

$$\begin{aligned} & \frac{\partial}{\partial t}(A_{k,n}) + \frac{\partial}{\partial s}(Q_{k,n}) = q_{k,n} \\ \Leftrightarrow & \frac{1}{(\Delta t)}(A_{k,n+1} - A_{k,n}) + \frac{1}{(\Delta s)_{k-1}}(Q_{k,n} - Q_{k-1,n}) \simeq q_{k,n} \\ \Leftrightarrow & A_{k,n+1} - A_{k,n} + \frac{(\Delta t)}{(\Delta s)_{k-1}}(Q_{k,n} - Q_{k-1,n}) \simeq (\Delta t) q_{k,n} \\ \Leftrightarrow & \boxed{A_{k,n+1} \simeq A_{k,n} - \frac{(\Delta t)_n}{(\Delta s)_{k-1}}(Q_{k,n} - Q_{k-1,n}) + (\Delta t)_n q_{k,n}}. \end{aligned} \tag{4.5}$$

Commentaires

- Le schéma numérique obtenu est le schéma arrière.
- Le schéma numérique obtenu est explicite.

Proposition (complexité algorithmique)

La complexité algorithmique du schéma (4.5) est égale à $O(KN)$.

Proposition (condition de stabilité, admise)

Le schéma (4.5) est stable lorsque :

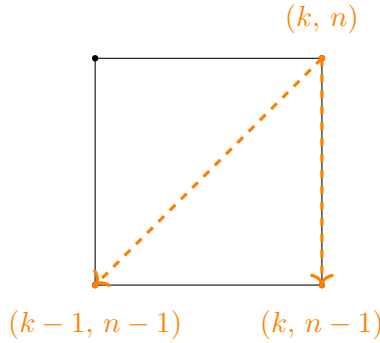
$$\forall n \in \{1, \dots, N\} : (\Delta_t)_n \leq \text{CFL} \cdot \min \left\{ \frac{(\Delta_s)_1}{c_1}, \min_{k \in \{2, \dots, K\}} \left\{ \frac{\min \{(\Delta_s)_k, (\Delta_s)_{k-1}\}}{c_k} \right\}, \frac{(\Delta_s)_K}{c_{K+1}} \right\},$$

avec :

$$\text{CFL} \in]0, 1].$$

Illustration

Schéma de dépendances du maillage espace-temps :



4.1.3.2 Schéma aux volumes finis arrière

On fait les approximations suivantes (arrières) :

$$Q_{k+1/2,n} \simeq Q_{k,n}, \quad \text{et} \quad Q_{k-1/2,n} \simeq Q_{k-1,n}.$$

Donc (d'après l'approximation (2.14)) :

$$\begin{aligned} & \frac{\partial}{\partial t} (A_{k,n}) + \frac{\partial}{\partial s} (Q_{k,n}) = q_{k,n} \\ \Leftrightarrow & \frac{\partial}{\partial t} (A_{k,n}) + \frac{1}{(\Delta_s)_k} (Q_{k,n} - Q_{k-1,n}) = q_{k,n}. \end{aligned}$$

D'après l'approximation (2.11), on obtient :

$$\begin{aligned} & \frac{\partial}{\partial t} (A_{k,n}) + \frac{\partial}{\partial s} (Q_{k,n}) = q_{k,n} \\ \Leftrightarrow & \frac{1}{(\Delta_t)_n} (A_{k,n+1} - A_{k,n}) + \frac{1}{(\Delta_s)_k} (Q_{k,n} - Q_{k-1,n}) \simeq q_{k,n} \\ \Leftrightarrow & (A_{k,n+1} - A_{k,n}) + \frac{(\Delta_t)_n}{(\Delta_s)_k} (Q_{k,n} - Q_{k-1,n}) \simeq (\Delta_t)_n q_{k,n} \end{aligned}$$

$$\Leftrightarrow A_{k,n+1} \simeq A_{k,n} - \frac{(\Delta_t)_n}{(\Delta_s)_k} (Q_{k,n} - Q_{k-1,n}) + (\Delta_t)_n q_{k,n}. \quad (4.6)$$

Commentaires

- Le schéma numérique obtenu est le schéma arrière.
- Le schéma numérique obtenu est explicite.

Proposition (complexité algorithmique)

La complexité algorithmique du schéma (4.6) est égale à $O(KN)$.

Proposition (condition de stabilité, admise)

Le schéma (4.6) est stable lorsque :

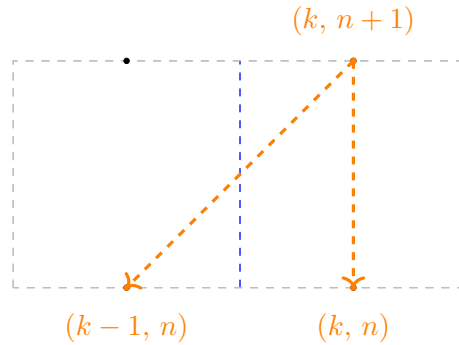
$$\forall n \in \{1, \dots, N\} : (\Delta_t)_n \leq \text{CFL} \cdot \min_{k \in \{1, \dots, K\}} \left\{ \frac{(\Delta_s)_k}{c_k} \right\},$$

avec :

$$\text{CFL} \in]0, 1].$$

Illustration

Schéma de dépendances du maillage espace-temps :



Implémentation (résumé, voir [lien cliquable](#))

Les signatures et corps de fonctions qui appliquent le schéma (4.6) sont les suivantes :

```
static inline __attribute__((always_inline))
void calculer_P_n(
    int K,
    int n,
    double *b,
    double *A,
    double *P_n
);
```

```
static inline __attribute__((always_inline))
void calculer_R_n(
    int K,
    int n,
    double *P_n,
    double *A,
```

```

    double *R_n
);

static inline __attribute__((always_inline))
void calculer_Delta_t_n(
    double CFL,
    int K,
    int n,
    double *Delta_s,
    double *b,
    double *r,
    double *S_0,
    double *P_n,
    double *R_n,
    double *A,
    double *Delta_t
){

    if (CFL <= 0.0){
        return;
    }

    double Delta_t_n_local;
    double Delta_t_n;
    double c_k_n;

    Delta_t_n = DBL_MAX;

    for (int k = 1 ; k < K + 1 ; k ++){

        c_k_n = (
            (S_0[k] / r[k]) *
            (
                pow(R_n[k], 2.0 / 3.0) +
                (2.0 / 3.0) * (
                    b[k] * pow(A[idx(k, n)], 2.0 / 3.0) /
                    pow(P_n[k], 5.0 / 3.0)
                )
            )
        );

        Delta_t_n_local = Delta_s[k] / c_k_n;
        if (Delta_t_n_local < Delta_t_n){
            Delta_t_n = Delta_t_n_local;
        }

    }

    Delta_t[n] = CFL * Delta_t_n;

    return;
}

```

```

static inline __attribute__((always_inline))

```



```

void appliquer_schema_A(
    int K,
    double *Delta_s,
    double *Delta_t,
    int n,
    double *q,
    double *A,
    double *Q
){

    for (int k = 1 ; k < K + 1 ; k ++){

        A[idx(k, n + 1)] = (
            A[idx(k, n)] -
            (Delta_t[n] / Delta_s[k]) * (Q[idx(k, n)] - Q[idx(k - 1, n)]) +
            Delta_t[n] * q[idx(k, n)]
        );

    }

    return;

}

```

```

static inline __attribute__((always_inline))
void appliquer_schema_Q(
    int K,
    double *Delta_s,
    double *Delta_t,
    int n,
    double *b,
    double *r,
    double *S_0,
    double *q,
    double *A,
    double *Q
){

    double P_k_n;
    double R_k_n;

    for (int k = 1 ; k < K + 1 ; k ++){

        P_k_n = b[k] + 2 * A[idx(k, n + 1)] / b[k];
        R_k_n = A[idx(k, n + 1)] / P_k_n;
        Q[idx(k, n + 1)] = (
            (sqrt(S_0[k]) * A[idx(k, n + 1)] * pow(R_k_n, 2.0 / 3.0)) /
            r[k]
        );

    }

    return;

}

```

```
static inline __attribute__((always_inline))
void appliquer_condition_limite(
    int type_condition_limite_A,
    int type_condition_limite_Q,
    int K,
    int n,
    double *A,
    double *Q
);
```

```
void resoudre_volume_fini_rusanov(
    int type_condition_limite_A,
    int type_condition_limite_Q,
    double CFL,
    int K,
    double *Delta_s,
    int N,
    double L_t,
    double *Delta_t,
    double *t,
    double *b,
    double *r,
    double *S_0,
    double *q,
    double *A,
    double *Q,
    double *t_final_ptr
);
```

Remarques

- On a 3 versions de l'implémentation des fonctions qui appliquent le schéma (4.6) :
 - Séquentielle : [lien cliquable](#)
 - Parallèle avec OpenMP : [lien cliquable](#)
 - Parallèle avec MPI : [lien cliquable](#)
- Chaque version donne des résultats numériques identiques (à non-associativité des nombres flottants près).
- Les fonctions pour gérer la parallélisation avec MPI sont identiques à celles du paragraphe 2.1.4.7.

Résultats (*erreurs numériques*)

On teste ces implémentations du schéma (4.6), en comparant les résultats numériques avec la solution exacte (4.2) (pour les paramètres suivants : $A_0(s) := \gamma(s+1)$, $\alpha := \frac{1}{2}$, $\gamma := 1$, $\Omega_s :=]0, 1[$, $\Omega_t :=]0, 10[$, $b \equiv 1$, $S_0 \equiv 0.01$, et $r \equiv 0.035$), et on obtient les résultats suivants :

$$\|A_{\text{approche}}(\cdot, 10) - A_{\text{exact}}(\cdot, 10)\|_{L_2} \text{ en fonction de } K \text{ et } N :$$

$\downarrow K \quad N \rightarrow$	100	1000	10000	50000
5	0.00342	0.00318	0.00316	0.00316
10	∞_f	0.00387	0.00385	0.00385
100	∞_f	∞_f	0.00448	0.00448
1000	∞_f	∞_f	∞_f	0.00457

$\|Q_{\text{approche}}(\cdot, 10) - Q_{\text{exact}}(\cdot, 10)\|_{L_2}$ en fonction de K et N :

$\downarrow K \quad N \rightarrow$	10	100	1000	10000
5	0.00043	0.00040	0.00039	0.00039
10	∞_f	0.00049	0.00049	0.00049
100	∞_f	∞_f	0.00053	0.00053
1000	∞_f	∞_f	∞_f	0.00053

Interprétations

- Le schéma (4.6) semble consistant en espace et en temps.
- Les erreurs sont satisfaisantes.

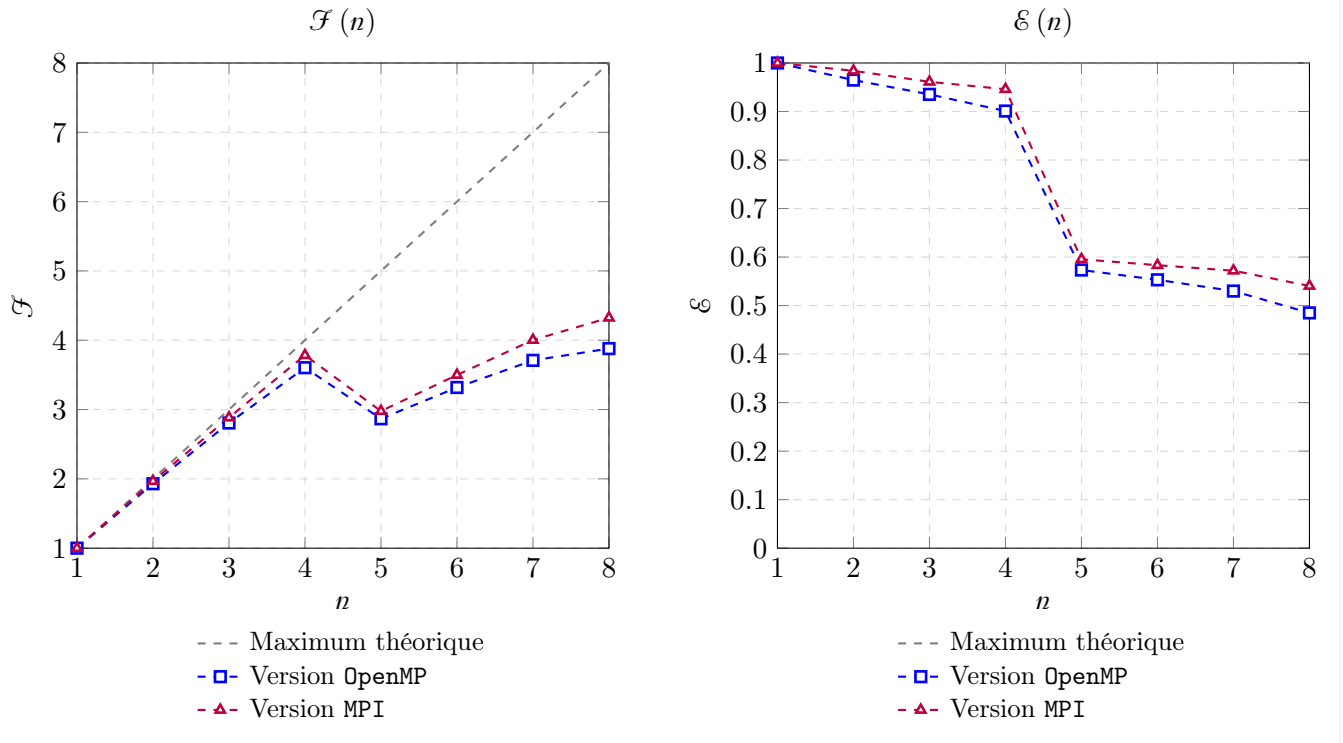
Résultats (*instabilités numérique*)

On teste ces implémentations du schéma (4.6), en recherchant des instabilités numériques, et on obtient les résultats suivants :

- Aucun paramètres tels que les résultats numériques divergent (lorsque $\text{CFL} \leq 1$) n'a été trouvé.
- Aucune oscillation en damier selon l'espace-temps n'a été trouvée.

Résultats (*parallélisation*)

On teste ces implémentations du schéma (4.6) (pour les paramètres suivants : $K, N := 15000$), en mesurant et comparant les facteurs d'accélération et efficacité de parallélisation entre les versions, et on obtient les résultats suivants (moyennés sur 10 exécutions) :



Interprétations

- Pour les versions `OpenMP` et `MPI`, l'efficacité de parallélisation est excellente lorsque $n \in \{2, 3, 4\}$ (entre 90% et 96%).
- Pour la version `OpenMP`, si $n = 4$, alors l'efficacité de parallélisation est égale à 90%.
- Pour la version `MPI`, si $n = 4$, alors l'efficacité de parallélisation est égale à 95%.
- Pour les versions `OpenMP` et `MPI`, l'efficacité de parallélisation est strictement décroissante.
- Pour les versions `OpenMP` et `MPI`, l'efficacité de parallélisation chute entre $n = 4$ et $n = 5$ (perte de 33% à 35%). On peut expliquer cela par l'architecture de la machine qui a 4 cœurs physiques et 8 threads logiques (avec 2 threads par cœur).
- L'efficacité de parallélisation est la meilleure pour la version `MPI` (2% à 6% de plus par rapport à la version `OpenMP`).
- Les résultats sont prometteurs en vue d'une parallélisation sur un cluster de calcul.

Résultats (temps d'exécution)

On teste l'implémentation du schéma (4.6) en version séquentielle, en mesurant les temps d'exécution, et on obtient les résultats suivants (moyennés sur 10 exécutions) :

Temps d'exécution (en s) en fonction de K et N :

$\downarrow K \quad N \rightarrow$	3000	6000	9000	12000	15000
3000	0.40	0.81	1.22	1.62	2.03
6000	0.81	1.63	2.45	3.24	4.03
9000	1.22	2.44	3.63	4.84	6.05
12000	1.63	3.25	4.87	6.50	8.09
15000	2.03	4.05	6.09	8.14	10.13

Coefficient du modèle de temps d'exécution en fonction de K et N :

$\downarrow K \quad N \rightarrow$	3000	6000	9000	12000	15000
3000	4.44×10^{-8}	4.50×10^{-8}	4.52×10^{-8}	4.50×10^{-8}	4.51×10^{-8}
6000	4.50×10^{-8}	4.53×10^{-8}	4.54×10^{-8}	4.50×10^{-8}	4.48×10^{-8}
9000	4.52×10^{-8}	4.52×10^{-8}	4.48×10^{-8}	4.48×10^{-8}	4.48×10^{-8}
12000	4.53×10^{-8}	4.51×10^{-8}	4.51×10^{-8}	4.51×10^{-8}	4.49×10^{-8}
15000	4.51×10^{-8}	4.50×10^{-8}	4.51×10^{-8}	4.52×10^{-8}	4.50×10^{-8}

Interprétations

- La matrice des temps d'exécution est pratiquement symétrique.
- La complexité algorithmique est vérifiée.
- Pour l'architecture de la machine, on peut établir la loi suivante :

$$\mathcal{T}(K, N) \simeq 4.50 \cdot 10^{-8} \cdot KN.$$

Résultats (opérations arithmétiques flottantes, accès mémoires, et temps d'exécution des fonctions)

On teste l'implémentation du schéma (4.6) en version séquentielle, en mesurant les opérations arithmétiques flottantes, les accès mémoires et le temps d'exécution des fonctions (avec le paquet **gprof**), et on obtient les résultats suivants (moyennés sur 5 exécutions) :

Propriété → ↓ Fonction	Nombre d'opérations arithmétiques flottantes									Nombre d'accès mémoire	Intensité arithmétique	Temps d'exécution (en %)
	+	-	*	/	fabs	sqrt	pow	<, >, ==, ...	Total			
calculer_Delta_t_n	1		3	7			3	1	15	7	0.3	14.3
calculer_P_n	1		1	1					3	4	0.1	9.6
calculer_R_n				1					1	3	< 0.1	8.1
appliquer_schema_A	1	2	2	1					6	8	0.1	40.8
appliquer_schema_Q			2	1		1	1		5	5	0.1	27.2
appliquer_condition_limite								5	5	8	0.1	0.0
autres	1								1	2	0.1	0.0
itération complète	4	2	8	11		1	4	6	36	37	0.1	100.0

Interprétations

- Les fonctions qui ont le temps d'exécution le plus élevé sont **appliquer_schema_A** (40.8%), et **appliquer_schema_Q** (27.2%). Ces 2 fonctions représentent 68.0% du temps d'exécution.
- Les fonctions qui ont le nombre d'opérations arithmétiques flottantes le plus élevé sont

`calculer_Delta_t_n` (15), et `appliquer_schema_A` (6).

- Les fonctions qui ont le nombre d'accès mémoire le plus élevé sont `appliquer_schema_A` (8), et `calculer_Delta_n` (7).
- On estime que les fonctions `appliquer_schema_A`, et `appliquer_schema_Q` représentent une grande partie du temps d'exécution.
- On estime que les fonctions `calculer_lambda_star_n`, `calculer_P_n`, et `calculer_R_n` représentent une petite modérée du temps d'exécution.
- On estime que la fonction `appliquer_condition_limite` représente une partie négligeable du temps d'exécution.
- L'intensité arithmétique est très faible. Le temps des accès mémoire représente probablement une grande partie du temps d'exécution.

4.2 Problème stationnaire, et conditions initiales

4.2.1 Présentation du problème

Si $\frac{\partial}{\partial t}(A) := 0$, $\frac{\partial}{\partial s} := \frac{d}{ds}$, $q := 0$, et $S_f := S_0$, alors :

$$\begin{aligned}\frac{\partial}{\partial t}(A) + \frac{\partial}{\partial s}(Q) &= q \\ \Rightarrow \frac{d}{ds}(Q) &= 0 \\ \Leftrightarrow Q &=: Q_C \in \mathbb{R},\end{aligned}$$

et :

$$\begin{aligned}S_f &= S_0 \\ \Leftrightarrow \varphi(A) &= 0,\end{aligned}$$

avec :

$$\varphi(A) := \frac{r^2 Q_C^2}{A^2 R(A)^{4/3}} - S_0.$$

Soit le problème $(P_{SV1r\star\star}^{\text{initial}})$ suivant :

Trouver A telle que :

$$\boxed{\begin{cases} \varphi(A, s) = 0 & \forall s \in \Omega_s \\ Q_C \in \mathbb{R} \end{cases}}. \quad (P_{SV1r\star\star}^{\text{initial}})$$

Commentaire

Le problème $(P_{SV1r\star\star}^{\text{initial}})$ est un ensemble d'équations scalaires indépendantes non linéaires.

4.2.2 Solutions approchées

Introduction

- En général, pour des paramètres quelconques, il n'existe de solution exacte connue de $(P_{SV1r\star\star}^{\text{initial}})$. Pour

utiliser le modèle, il faut calculer une solution approchée avec une méthode d'analyse numérique.
— L'objectif de cette section est de présenter des solutions approchées.

Implémentation

Soient $\varepsilon > 0$ suffisamment petit, et $\varepsilon \in \mathbb{N}^*$ suffisamment grand. Alors, un algorithme qui applique la résolution du problème $(P_{SV1r**}^{\text{initial}})$ est le suivant :

```
function RESOUDRE_DICHOTOMIE_CINEMATIQUE( $k, b, r, S_0, Q_C, A_{\text{minimum}}, A_{\text{maximum}}$ )
   $\varphi_{\text{minimum}} \leftarrow \varphi(k, b, r, S_0, A_{\text{minimum}}, Q_C)$ 
   $\varphi_{\text{maximum}} \leftarrow \varphi(k, b, r, S_0, A_{\text{maximum}}, Q_C)$ 
  if  $\varphi_{\text{minimum}} \varphi_{\text{maximum}} > 0$  then
    return erreur
  end if
  arret  $\leftarrow 0$ 
  compteur  $\leftarrow 0$ 
  while compteur  $< \varepsilon$  and arret = 0 do
     $A_{\text{milieu}} \leftarrow \frac{1}{2} (A_{\text{minimum}} + A_{\text{maximum}})$ 
     $\varphi_{\text{milieu}} \leftarrow \varphi(k, b, r, S_0, A_{\text{milieu}}, Q_C)$ 
    if  $|\varphi_{\text{milieu}}| < \varepsilon$  or  $|A_{\text{maximum}} - A_{\text{minimum}}| < \varepsilon$  then
      arret  $\leftarrow 1$ 
    else if  $\varphi_{\text{minimum}} \varphi_{\text{milieu}} \leq 0$  then
       $A_{\text{maximum}} \leftarrow A_{\text{milieu}}$ 
       $\varphi_{\text{maximum}} \leftarrow \varphi_{\text{milieu}}$ 
    else
       $A_{\text{minimum}} \leftarrow A_{\text{milieu}}$ 
       $\varphi_{\text{minimum}} \leftarrow \varphi_{\text{milieu}}$ 
    end if
    compteur  $\leftarrow$  compteur + 1
  end while
   $A \leftarrow A_{\text{milieu}}$ 
  return  $A$ 
end function
```

Remarques

Par la simplicité de ce type de problème, on n'a pas fait de tests rigoureux de cette implémentation. Les résultats ont été jugés très satisfaisants.

5 Résumé des modèles

Introduction

L'objectif de cette section est de résumer les modèles étudiés.

5.1 Caractéristiques

On résume les caractéristiques des modèles étudiés :

Modèle → ↓ Propriété	Équations de Saint-Venant cinématiques	Équations de Saint-Venant diffusives	Équations de Saint-Venant dynamiques
Géométrie	Hyperbolique	Parabolique	Hyperbolique
Type	Transport d'ondes	Diffusion-advection d'ondes	Propagation d'ondes caractéristiques
Conservation de la masse	oui	oui	oui
Conservation de la quantité de mouvement	non	partielle	oui
Sens de l'onde	amont → aval	amont ↔ aval	amont ↔ aval
Pente du fond de la rivière S_0	$\in \mathbb{R}_+$	$\in \mathbb{R}$	$\in \mathbb{R}$
Inertie	non	non	oui
Pression	non	oui	oui
Pente de frottement S_f	$:= S_0$	définition	définition
Modèle de condition initiale	Équations scalaires	Problème de Cauchy	Problème de Cauchy
Nombre de conditions aux limites	1 (amont)	2 (1 amont, 1 aval)	2 (dépend de Fr)
Nombre de variables indépendantes	1 (A)	1 (A)	2 (A et Q)

5.2 Erreurs

On résume les erreurs des modèles étudiés (pour le problème stationnaire) :

Modèle → ↓ Propriété	Équations de Saint-Venant cinématiques	Équations de Saint-Venant diffusives	Équations de Saint-Venant dynamiques
Erreurs	Excellent	/	Correct
Stabilité	Excellent	Catastrophique	Catastrophique

On résume les erreurs des modèles étudiés (pour le problème instationnaire) :

Modèle → ↓ Propriété	Équations de Saint-Venant cinématiques	Équations de Saint-Venant diffusives	Équations de Saint-Venant dynamiques
Erreurs	Excellent	/	Excellent
Stabilité	Très bonne	Correct	Catastrophique

Interprétations

- La stabilité du problème instationnaire est la meilleure pour le modèle par équations de Saint-Venant cinématiques.
- Le modèle par équations de Saint-Venant dynamique n'est actuellement pas utilisable en production.
- On utilisera les conditions initiales obtenues par l'implémentation de la résolution du problème ($P_{SV1r\star\star}^{\text{initial}}$) comme conditions initiales des problèmes (P_{SV1r}) et ($P_{SV1r\star}$). Physiquement, elles représentent un régime permanent cinématique et non diffusif ou dynamique, mais pour les cas d'utilisations de *Bluemapping*, on juge ce régime permanent cinématique utilisable pour ces problèmes.
- En pratique, dans les cas d'utilisations de *Bluemapping*, si on injecte un régime permanent cinématique comme conditions initiales pour les problèmes (P_{SV1r}) et ($P_{SV1r\star}$), alors il y a une période dite de *régime transitoire*, où l'état du maillage va tendre vers un pseudo régime permanent diffusif ou dynamique pendant quelques centaines voir milliers d'itérations temporelles. Lorsque le pseudo régime permanent est atteint, on le redémarre la simulation en utilisant ce pseudo régime permanent comme conditions initiales. On juge ce pseudo régime permanent satisfaisant.
- D'après l'état de l'art, le modèle par équations de Saint-Venant dynamique est réputé comme instable, à cause des termes S_0 et S_f .
- Pour résoudre les problèmes de stabilité du modèle par équations de Saint-Venant dynamique, des explorations sur des schémas plus stable et semi-implicites ont été démarrées (voir [2] [4] [5]).

Remarque

Selon la situation physique, il existe des recommandations rigoureuses sur quels modèles utiliser (voir [9]).

5.3 Performances

On résume les performances des modèles étudiés (pour le problème stationnaire) :

Modèle → ↓ Propriété	Équations de Saint-Venant cinématiques	Équations de Saint-Venant diffusives	Équations de Saint-Venant dynamiques
Complexité algorithmique	$O(K)$	$O(K)$	$O(K)$
Temps d'exécutions	Négligeable	Négligeable	Négligeable

On résume les performances des modèles étudiés (pour le problème instationnaire) :

Modèle → ↓ Propriété	Équations de Saint-Venant cinématiques	Équations de Saint-Venant diffusives	Équations de Saint-Venant dynamiques
Efficacité de parallélisation pour la version OpenMP (pour $n = 4$)	90%	92%	88%
Efficacité de parallélisation pour la version MPI (pour $n = 4$)	96%	96%	95%
Complexité algorithmique	$O(KN)$	$O(KN)$	$O(KN)$
Modèle de temps d'exécution (en s)	$4.50 \cdot 10^{-8} \cdot KN$	$5.71 \cdot 10^{-8} \cdot KN$	$3.59 \cdot 10^{-8} \cdot KN$
Fonctions goulots d'étranglement	appliquer_schema_Q, et appliquer_schema_A	appliquer_schema_Q, et appliquer_schema_A	appliquer_schema_Q, et appliquer_schema_A
Nombre d'opérations arithmétiques flottantes par itération	36	85	82
Nombre d'opérations arithmétiques flottantes de type pow par itération	4	3	1
Nombre d'accès mémoire par itération	37	71	75
Intensité arithmétique	0.1	0.1	0.1

Interprétations

- Pour chaque modèle, le temps d'exécution du problème stationnaire est négligeable.
- Pour chaque modèle, l'efficacité de parallélisation pour les versions **OpenMP** et **MPI** est prometteuse.
- Pour chaque modèle, l'efficacité de parallélisation est la meilleure pour la version **MPI**, et la plus mauvaise pour la version **OpenMP**.
- Pour chaque modèle, la complexité algorithmique du problème instationnaire est $O(KN)$.
- Pour chaque modèle, la complexité algorithmique du problème stationnaire est $O(K)$.
- Le modèle de temps d'exécution est le meilleur pour le modèle par équations de Saint-Venant dynamique ($3.59 \cdot 10^{-8} \cdot KN$), suivi du modèle par équations de Saint-Venant cinématique ($4.50 \cdot 10^{-8} \cdot KN$), et le plus mauvais est le modèle par équations de Saint-Venant diffusives ($5.71 \cdot 10^{-8} \cdot KN$).
- Le nombre d'accès mémoire par itération et le nombre d'opérations par itération est le meilleur pour le modèle par équations de Saint-Venant cinématiques.
- Pour chaque modèle, l'intensité arithmétique est petite.

Remarques

- En pratique, le modèle de temps d'exécution n'est pas le meilleur pour le modèle par équations de Saint-Venant cinématique. Cela peut sembler surprenant, mais on peut expliquer cette observation par le fait que ce modèle possède le nombre d'opérations arithmétiques flottantes de type `pow` par itération le plus élevé (4).
- Par rapport aux opérations arithmétiques flottantes de type `+`, `-`, `*`, `fabs`, ou comparateurs logiques (qui consomment généralement 1 à quelques cycles d'horloge C. P. U.), ou aux opérations arithmétiques flottantes de type `/` ou `sqrt` (qui consomment généralement quelques dizaines de cycles d'horloge C. P. U.), l'opération arithmétique flottante de type `pow` consomme généralement quelques centaines de cycles d'horloge C. P. U.. On conjecture que l'opération arithmétique flottante de type `pow` représentent une très grande partie du temps d'exécution de la partie calcul.

5.4 Conclusion

- Tous les modèles sont performants grâce à la nature mathématique du problème.
- Tous les modèles sont parallélisables pour un petit nombre de threads ou de processus. Il reste à confirmer cette affirmation pour un nombre plus grand de threads ou de processus.
- Pour chaque modèle, avec l'architecture de la machine, en exploitant tout l'espace mémoire en R. A. M., les temps d'exécution sont de quelques secondes.
- Avec les temps d'exécution très faibles, un facteur limitant est l'espace mémoire en R. A. M.. Si $K, N := 100000$, alors l'espace mémoire nécessaire alloué dynamiquement pour les tableaux $\mathbf{A}$, $\mathbf{Q}$, $\mathbf{q}$, et $\mathbf{v}$ est égal à $(100000 + 2) \cdot (100000 + 2) \cdot 8 \text{ o} \simeq 80\text{G o}$.
- En production, si le modèle par équations de Saint-Venant dynamique n'était pas instable, alors ça serait ce dernier à utiliser, car on juge que c'est le plus performant. En pratique, le meilleur compromis entre le réalisme physique, la stabilité et la performance est le modèle par équations de Saint-Venant diffusives.
- Finalement, on obtient 2 modèles de rivière utilisables, et 1 modèle de rivière dont le calcul de la solution approchée est à rendre stable.

6 Veille bibliographique, et couplage de modèles

Avant l'étude des modèles, on a étudié la bibliographie des logiciels concurrentiels suivants :

ANUGA Hydro	Basilisk 2D	BreZo	CAFLOOD	CaMa-Flood	Cartino 1D
CityCAT	Delft 3D	DHFM	DIVAST	DIVAST-TVD	DSM2
EFDC	ExZeco	FDM	FHM	FINEL 2D	Flo-2D
Flood Modeller	Flood4cast	FloodFX	FloodMap	Floodos	Flutor
GeoFlood	GUFIM	HEC-HMS	HEC-RAS	Hybrid Inundation Model	Hydra
HydroDEAL	Hydrocal	Iber	InfoWorks ICM	IRIP	ISIS-1D
ISIS-2D	ISIS-FAST	ISIS-FREE	JFLOW	LISFLOOD-FP	MASCARET
MHYST	MIKE FLOOD	MIKE HYDRO	MIKE+	MIKE-11	MIKE-21
MODFLOW	OpenLISEM	P-DWave	PRIMo	Rana	RFSM
RMA2	RuiCells	SCALGO	SCHISM	SFINCS	SMASH
SOBEK	SUNTANS	SWMM	TELEMAC 2D	TELEMAC 3D	TRENT
TRIM	Triton	TUFLOW	UIM	UnTRIM	UPFLOOD
WaterSed	XP2D				

La bibliographie comporte 1 fiche (au format `markdown`) par logiciel, avec les informations suivantes :

- Disponibilité (disponible, non disponible, libre, propriétaire, gratuit, payant, académique, industriel, ...).
- Modèles (rivière, plaine, ruissellement, ...).
- Fondements mathématiques et informatiques (équations, méthodes numériques, schémas numériques, ...).
- Dimension des modèles (1D, 2D, 3D).
- Lien vers des sites.
- Lien vers des documents (manuels, articles, ...).
- Paragraphes de description (conditions initiales, conditions aux limites, topographie, équations, modèles, données d'entrée, ...).
- Commentaires personnels.

Remarque

La bibliographie est sous forme d'un *vault* du logiciel **Obsidian**. Ce logiciel permet de stocker des fichiers `markdown` avec une arborescence structurée, de faire des liens entre les fichiers, de mettre des tags, de faire des recherches automatiques, ...

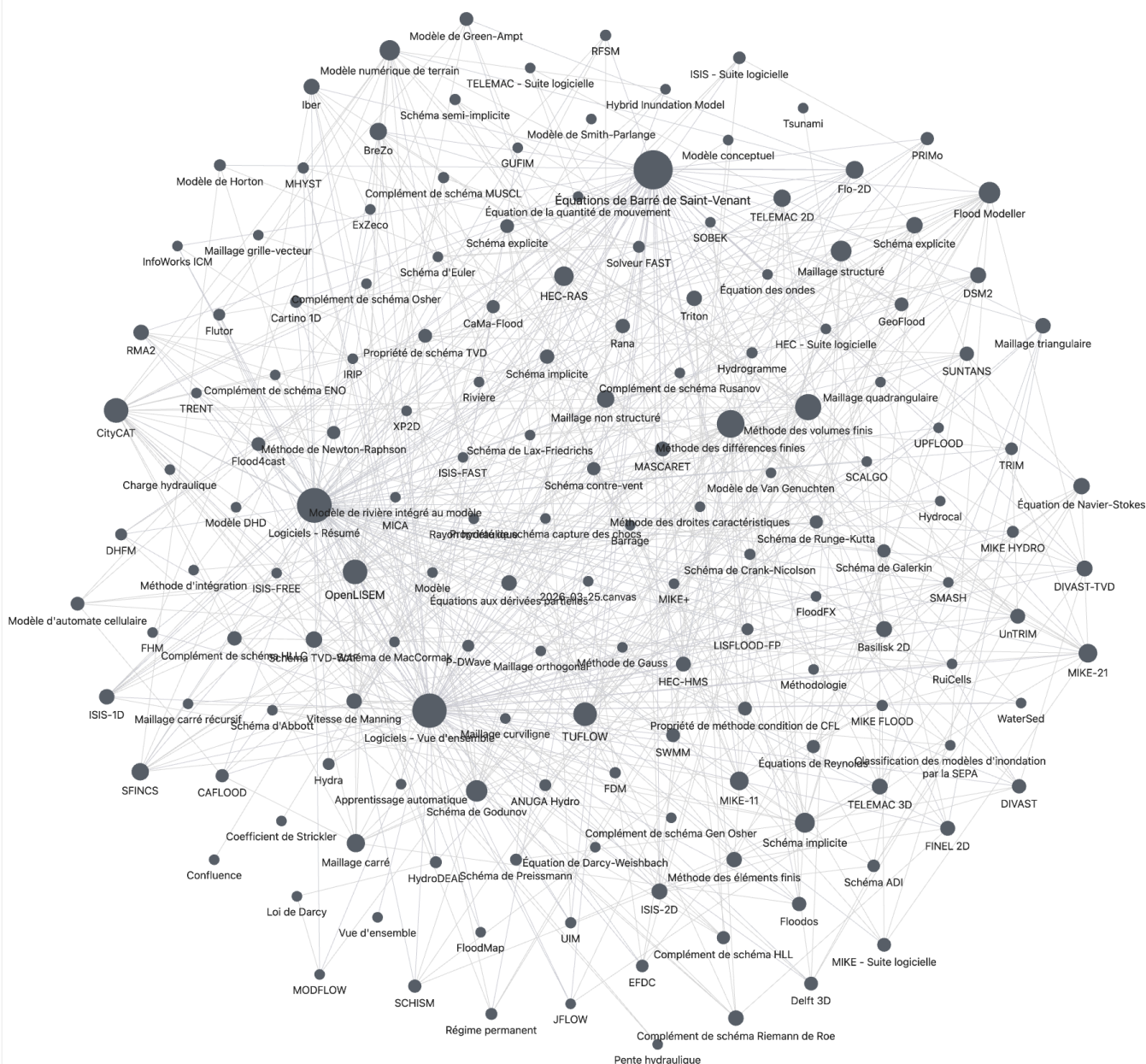
On résume quelques catégories de cette bibliographie :

Automate cellulaire	Modèle numérique de terrain	Équations de Saint-Venant	Équations de Navier-Stokes	Équations de Reynolds
CAFLOOD	BreZo	ANUGA Hydro	Basilisk 2D	EFDC
Hybrid Inundation Model	ExZeco	CAFLOOD	Delft 3D	SUNTANS
Hydrocal	FHM	Cartino 1D	SCHISM	TRIM
RFSM	Flutor	Basilisk 2D	SUNTANS	UnTRIM
RuiCells	GeoFlood	BreZo	TELEMAC 3D	
	Hybrid Inundation Model	CaMa-Flood		
	ISIS-FAST	CityCAT		
	MHYST	DIVAST		
	WaterSed	DSM2		
		Flo-2D		
		Flood4cast		
		Floodos		
		Flood Modeller		
		GeoFlood		
		HEC-HMS		
		HEC-RAS		
		Hydra		
		Iber		
		InfoWorks ICM		
		ISIS-1D		
		ISIS-2D		
		JFLOW		
		LISFLOOD-FP		
		MASCARET		
		MIKE-11		
		MIKE-21		
		OpenLISEM		
		P-DWave		
		PRIMo		
		Rana		
		RFSM		
		RMA2		
		SCALGO		
		SFINCS		
		SWMM		
		TELEMAC 2D		
		TUFLOW		
		Triton		
		UIM		
		UPFLOOD		

Méthode des différences finies	Méthode des volumes finis	Méthode des éléments finis
DIVAST	ANUGA Hydro	FINEL 2D
DSM2	Basilisk 2D	RMA2
EFDC	BreZo	SCHISM
Flo-2D	CityCAT	TELEMAC 2D
Flood Modeller	HEC-RAS	TELEMAC 3D
HEC-HMS	Hydra	
HEC-RAS	Iber	
ISIS-1D	MIKE-21	
LISFLOOD-FP	OpenLISEM	
MASCARET	PRIMo	
MIKE-11	RMA2	
MIKE-21	SCHISM	
MODFLOW	SUNTANS	
P-DWave	TELEMAC 2D	
SFINCS	Triton	
TELEMAC 2D	TUFLOW	
TUFLOW		
TRIM		

Illustration

Graphique des liens entre les pages de la bibliographie :



- Chaque disque représente une page de la bibliographie.
- Chaque segment représente un lien entre 2 pages.
- La taille du disque d'une page représente le nombre de lien qui pointent vers cette page.

Remarques

- Une attention particulière a été menée pour étudier les logiciels qui possèdent un couplage en espace et en temps entre un modèle de ruissellement et un modèle de rivière, car un objectif futur de *Bluemapping* est de coupler les modèles de rivière étudiés avec leur modèle de ruissellement actuel.
- Le couplage en espace consiste à échanger de l'eau entre le modèle de ruissellement et le modèle de rivière (ruissellement de la plaine vers la rivière, ou débordement de la rivière vers la plaine).

- Le couplage en temps consiste à échanger de l'eau à intervalle de temps fixes ou dynamiques.
- Des algorithmes de couplage en espace (avec projection entre le maillage 1D des modèles de rivière vers le maillage 2D du modèle de ruissellement) ont été conçus.

7 Application en ligne de commande de la résolution des modèles de rivière

Introduction

L'objectif de cette section est de décrire l'application des modèles de rivière.

7.1 Utilisation

- Les modèles de rivière fonctionnent par l'utilisation d'une application en ligne de commande, écrite en langages C et C++, et qui fonctionne avec **docker**.
- Les étapes pour utiliser l'application sont les suivantes :
 - Se placer dans un répertoire dédié à stocker l'application.
 - Cloner le dépôt :

```
git clone https://github.com/gaillot18/Stage-M2-CHPS-Modeles-riviere
```

- Se placer dans le répertoire de l'application :

```
cd Application
```

- Construire l'image **docker** associée à l'application :

```
docker build -f Dockerfile -t modeles-riviere .
```

- Créer et lancer un conteneur associé à l'image précédente :

```
docker run -it --rm -v ./Repertoire_racine modeles-riviere \
/bin/bash -c "./Script.sh"
```

- Dans l'image **docker**, les paquets sont les suivants :
 - **gcc** (paquet **apt**).
 - **g++** (paquet **apt**).
 - **make** (paquet **apt** ou **homebrew**).
 - **libgomp1** (paquet **apt** ou **homebrew**).
 - **mpich** (paquet **apt** ou **homebrew**).
 - **libmpich** (paquet **apt** ou **homebrew**).

Remarques

- Cette application est à visée académique, elle ne sera pas utilisée pour la mise en production.
- Dans le répertoire **Problèmes**, il y a un sous-répertoire par modèle. Ce sous-répertoire contient les fichiers de bibliothèques, de sources, un **Makefile** et un script. Il est possible de changer les paramètres de l'exécution dans le script.
- La solution exacte 1 permet de tester le schéma sur les erreurs. La solution exacte 2 n'est pas une vraie solution exacte, mais un problème quelconque dont on extrait les conditions initiales, elle permet de tester le schéma sur les performances.

7.2 Contraintes

- *Bluemapping* n'utilise pas des clusters de calcul traditionnels, mais utilise le cloud d'*Amazon Web Services*, où ils payent leur temps d'exécution à la minute. À l'heure actuelle, ils utilisent le cloud uniquement pour les simulations demandées par un client, ou des tests. Lors du stage, par manque de moyens, il n'a pas été possible d'inférer l'outil sur le cloud, donc tous les tests et toutes les expériences ont été faits sur des machines ordinaires, dont les configurations sont les suivantes :

↓ Propriété Machine →	Développements	Tests
Type	Macbook Air M3	Serveur Hetzner
C. P. U.	Apple ARM M3 (8 cœurs, 8 threads)	AMD EPYC (4 cœurs, 8 threads)
R. A. M.	16G o	32G o
Système d'exploitation hôte	macOS	linux
Système d'exploitation conteneurisé	linux	linux

7.3 Propriété intellectuelle

- Avant le stage, il n'existait pas de modèles de rivière chez *Bluemapping*.
- Tous les calculs intermédiaires (application des approximations de Lax-Friedrichs, de Rusanov, simplification des problèmes, ...) de ce document ont été réalisés à la main.
- Le sujet s'est déroulé sur une durée de $\frac{1}{2} \cdot 6 = 3$ mois.
- Le code source de l'application est libre.
- Il existe une autre application (en langage **python**, avec traitement de données d'entrée, algorithmes de couplage de modèles, ...), dont le code source n'est pas libre (appartient à *Bluemapping*).

Références

- [1] Muhammad Salihi Abdul Hadi, Mohd Zaini Mustapa, and Shahbudin Saad. Complete derivation of 2d shallow-water model from the primitive equations governing geophysical flows. In *Proceedings of the 8th International Conference on Marine Technology*, Terengganu, Malaysia, October 2012. 1, 2
- [2] Emmanuel Audusse, François Bouchut, Marie-Odile Bristeau, Rupert Klein, and Benoit Perthame. A fast and stable well-balanced scheme with hydrostatic reconstruction for shallow water flows. *SIAM Journal on Scientific Computing*, 25(6) :2050–2065, January 2004. 77
- [3] Olivier Delestre, Carine Lucas, Pierre-Antoine Ksinant, Frédéric Darboux, Christian Laguerre, Thi Ngoc Tuoi Vo, François James, and Stéphane Cordier. Swashes : a compilation of shallow water analytic solutions for hydraulic and environmental studies. *International Journal for Numerical Methods in Fluids*, 72(3) :269–300, October 2012. 40
- [4] Enrique D. Fernandez-Nieto, Jose Garres-Diaz, Emanuele Macca, and Giovanni Russo. A third-order finite volume semi-implicit method for the shallow water-exner model. *Journal of Scientific Computing*, 105(3), October 2025. 77
- [5] Irene Gomez-Bueno, Sebastiano Boscarino, Manuel J. Castro, Carlos Pares, and Giovanni Russo. Implicit and semi-implicit well-balanced finite-volume methods for systems of balance laws. *Applied Numerical Mathematics*, 184 :18–48, February 2023. 77
- [6] Justin Hudson. Numerical techniques for the shallow water equations. Technical Report Numerical Analysis Report 2/99, The University of Reading, Department of Mathematics, 1999. Consulté le 5 août 2026. 13, 20, 33, 38
- [7] Arthur T. Ippen, editor. *Estuary and Coastline Hydrodynamics*. Engineering Societies Monographs. McGraw-Hill Book Co., New York, 1966. 14
- [8] Keval H Jodhani, Dhruvesh Patel, and N. Madhavan. A review on analysis of flood modelling using different numerical models. *Materials Today : Proceedings*, 80 :3867–3876, 2023. 1
- [9] Victor Miguel Ponce, Daryl B. Simons, and Ruh-Ming Li. Applicability of kinematic and diffusion models. *Journal of the Hydraulics Division*, 104(3) :353–360, March 1978. 77
- [10] Jin Teng, Anthony .J. Jakeman, Jai Vaze, Barry F.W. Croke, Dushmanta. Dutta, and S. Kim. Flood inundation modelling : A review of methods, recent advances and uncertainty analysis. *Environmental Modelling and Software*, 90 :201–216, April 2017. 1
- [11] Eleuterio F. Toro and Svetlana A. Tokareva. Rusanov-type schemes for hyperbolic equations : Wave-speed estimates, monotonicity and stability. *Computers and Fluids*, 309 :107000, April 2026. 22
- [12] Ali R. Vatankhah and Said M. Easa. Depth-independent kinematic wave parameters for trapezoidal and power-law channels. *Ain Shams Engineering Journal*, 4(2) :173–183, June 2013. 65
- [13] Jean Paul Vila, Florent Chazel, and Pascal Noble. 2d versus 1d models for shallow water equations. *Procedia IUTAM*, 20 :167–174, 2017. 1